

BDL Design and Formalization Monograph

A Behavior Design Language for Industrial Designers:
Design Position, Mechanically Derived Kernel, Environment Boundaries,
Production Toolchain, Correspondence, Rejected Alternatives and Open Agenda

ZHU ZHEHAO

Living technical record — revision of 2026-09-20

Formal development KCN-judu/BDL_FV — this working tree, last pushed commit af25567 (Phase 13)
Production KCN-judu/BDL at de8154f5153495de2ad8a09f3ca3166c3678dc93 (2026-09-20, protocol 0.21)

This document is the authoritative narrative design record of BDL and a bridge between its formal model and its production system. It is not a paper, not a submission and not written to a page limit; it records what the language is, why each construct exists or was rejected, which theorem, counterexample or executed example supports each claim, how production implements it, where production deviates from the formal model, and what remains open — including the alternatives that failed. Every claim carries a strength label (formally proved; formally characterized under restricted hypotheses; mechanically executed example; counterexample; informed by FV; production implemented and tested; production architecture decision; proposal / not implemented; design recommendation; open empirical question). No usability claim in it has been tested with users.

Contents

About this document	6
Purpose, current result, scope	6
Claim strength	7
Identifiers	7
Revision log	7
Part I — Purpose and Design Position	9
Introduction	9
The Representation Problem	10
The target user is not a programmer with fewer syntax skills	10
A flow graph is a dependency view, not a program counter	11
Cognitive budget	11
Progressive disclosure is a semantic property	11
Scope and non-goals	11
Part II — The Designer-Facing Language	13
Concepts	13
Relationships and the three roles	13
Incomplete designs are legal	14
Formulas	14
Time	15
Physical outputs	15
Behaviors, components and systems	16
The Standard Library	16
Supplied computation — designed, not built	16
A representative scenario	17
What the workspace says	17
Looking underneath	18
Part III — The Core Declaration Model	19
Method	19
Scale and trust base	19
From Surface to Kernel: Architecture	19
Declarations, Interfaces, and Refinement	20
Typing through the type view	20
Satisfaction, well-formedness, and refinement	20
Client stability	21
Refinement versus edit	21
What persistent identity is	22
Where the rest of the kernel is described	22
Part IV — Semantic Identity and Physical Quantities	23
Nominal concepts	23
Representation binding and the grant	23
Physical dimensions	24
Part V — The Data and Equation Language	26
The question and the hypothesis	26
What entered the kernel, and why each item could not be derived	26
Polymorphism: rank-1, by families, no kernel type variable	27
Capabilities: the audit that reverted a decision	27
The definitional library	28
Enumerations, records, sets: what was encoded and what was deferred	29
The expressiveness ceiling	29
The natural expression surface (Phase 11)	29

Part VI – Units, Coordinates and Charts	31
Three notions kept apart	31
Linear units: no kernel construct	31
The numeric domain, in the open	31
Presentation, and the realistic formula	32
Unit ownership in authoring	32
Affine units: the failure of scale-only units, and the revision	32
Charts: coordinate erasure and conversion functoriality (Phase 10b)	32
The production chart model	33
The Formula Composer’s formal basis	34
Part VII – Reactive Semantics	35
A Minimal Reactive Semantics	35
One primitive	35
Causality	35
Derived operators	36
Clock Domains	36
Identity, not rate	36
One transport primitive	37
Strictly before	37
Cross-domain occurrences and lossless buffering (Phase 9a)	37
Part VIII – Behavior Systems	40
Behavior components (Phase 8a)	40
Behavior groups (Phase 8b)	41
Hardware interaction	43
Part IX – The Environment and the Physical Boundaries	44
The canonical type of a relationship that reads nothing	44
Source, Rule, Value: production’s reading of the realization state	45
Physical outputs without arbitration	46
The model	46
One final driver	46
What was removed	47
Why A -> () is not a sink	47
Provision: how a Source gets its value on a product	47
Two remarks across Parts III–IX	50
Why the signature is a design object	50
Nominal identity in three places	50
Part X – Validation and Deployment	51
Three verdicts that must not be confused	51
Target-specific hardware validation	51
Resources, capabilities, requirements	51
A sound and complete solver	52
Case study: an Arduino Nano	52
Two kinds of evidence	52
Deployment capacity	53
Provision as deployment architecture	53
The generated-code boundary	53
Open hardware questions	53
Part XI – Production Compiler, Runtime and Daemon	54
Four trust layers	54
Crates and semantic ownership	54
The compiler as a pipeline of explicit passes	55
Executable IR and lowering	56
The generated core	56

Embedded execution model	56
Differential testing	57
Project persistence and the daemon	57
The IDE service, the daemon and the protocol	58
The elaboration passes the kernel implies	58
Part XII — Studio and the IDE: The User Interface as a Semantic Projection	60
Three information levels	60
The canvas	60
Source, Rule and Value on the canvas	61
Semantic actions	61
Colour in the Code view and the formula field	61
The Library	61
Groups and components on one canvas	61
Design, Code and Split: views of one project	62
The definition editor and the Formula Composer	62
Partial expressions: ?	63
Simulate, Deploy, Library, Monitor	64
What Studio does not decide	64
Part XIII — Formal $\leftrightarrow$ Production Correspondence	65
Construct by construct	65
Deviations, stated once with their reasons	69
The snapshot	70
Part XIV — Minimality and the Rejected Alternatives	71
Two kinds of minimality	71
Kernel constructs retained	71
Rejected constructs	72
Decisions that changed	74
Part XV — The Open Agenda	76
Formal limits, stated once	76
Open formal and engineering problems	76
Empirical questions, explicitly	77
Expected cognitive advantages, as hypotheses	78
Comparative study	78
Field study	79
Risks of the design	79
Intellectual Context and Related Work	80
Industrial design tools and physical prototyping	80
Visual programming and end-user programming	80
Synchronous languages and functional reactive programming	80
Model-based design, statecharts and systems engineering	80
Typed holes, live and structured editing, projectional editing	81
Dimensional typing and units	81
Effects	81
Resource allocation	81
Formal verification, verified compilation and differential testing	81
Embedded DSLs and toolchains	82
Appendix A — Formal notation	83
Appendix B — Theorem and result index, by concept	85
Refinement and client stability (Core/Interface, Core/Decl, Core/Satisfaction, Core/Env)	85
Typing, unfolding and dependency (Core/Typing, Core/Dependency)	85
Semantic identity, representation and the grant (Core/Base, Experiments/SemanticTypeAlternatives, Experiments/ RepresentationBindingAlternatives)	85
Quantities and dimensions (Core/Base, Experiments/DimensionAlternatives)	86

Reactive semantics (Core/Reactive, Experiments/ReactiveAlternatives)	86
Clock domains and transport (Core/Clock, Experiments/ClockAlternatives)	86
Lists, the buffer and capacity (Core/ListData, Surface/Buffer, Validation/Capacity, Experiments/ BufferAlternatives)	87
Products, the recursor, equality and order (Core/Base, Surface/Poly, Experiments/PolyAlternatives)	87
The equation library and rank-1 polymorphism (Surface/Poly, Surface/Stdlib, Surface/Generic, Experiments/ EquationExamples)	88
Natural surface (Surface/Natural, Experiments/NaturalExamples)	88
Units, coordinates and charts (Surface/Units, Surface/Composer, Surface/Affine, Surface/Rational, Surface/ Charts)	89
Physical outputs (Core/Output, Experiments/OutputAlternatives)	90
Unit domain and the source role (Surface/UnitDomain, Experiments/UnitDomainExamples)	90
Source provision (Surface/Provision, Experiments/ProvisionExamples)	90
Hardware validation (Validation/Hardware, Experiments/HardwareAlternatives)	91
Behavior systems and groups (Behavior/*, Experiments/BehaviorAlternatives, Experiments/GroupAlternatives) .	91
Appendix C — Decision index	93
Appendix D — Evidence-strength ledger	105
Appendix E — Migration map from the old decision numbers	108
Open items	114
Appendix F — Production snapshot	116
Milestones of the week before the snapshot, newest first	116
Protocol	116
Implementation status, by area (production’s own words, abridged)	117
Facts this document relies on that could move	117
Appendix G — Development chronology	118
Appendix H — Revision log	120
Closing	121
References	122

About this document

This is the *BDL Design and Formalization Monograph*: a living technical record of the Behavior Design Language. It is the authoritative narrative design record of BDL — the one document that says, for every construct, why it is there, what was tried instead, which theorem or counterexample decided it, how production implements it, where production differs from the model, and what remains open. It is a bridge between the formal model (KCN-judu/BDL-FV, Lean 4) and the production system (KCN-judu/BDL, Rust and Flutter); a record of successful *and* rejected design alternatives; an evidence ledger; and a reference for future implementation and research.

It is **not** a conference submission, a page-limited paper, a polished retrospective in which the design emerged fully formed, or a marketing document. It is not written to a page count; completeness, provenance, negative results, design evolution and correspondence to production matter more than compactness. Where an earlier draft of BDL took a position that later evidence broke, the record keeps both and says what broke it. The directory the document lives in is still called *paper/* because renaming it would break history; reader-facing text says *monograph* or *record*.

Purpose, current result, scope

Purpose. Industrial products increasingly combine physical form with sensing, computation and control, and their designers have no medium for behavior with the immediacy that CAD has for geometry. BDL asks whether product behavior can be a *design material*: a language whose primary object is the typed product relationship — $\text{Tilt} \rightarrow \text{Brightness}$ — which may be declared before it is defined, checked while incomplete, simulated, and eventually realized on hardware, with the implementation machinery elaborated underneath rather than authored.

Current result. A small kernel, derived by a mechanized design-space exploration in Lean 4 and stated in Parts III–IX: an environment of named declarations with frozen types, monotone public commitments and write-once realizations; nominal concepts constructed only where a signature announces them; dimensions in the types of primitive operators; one temporal primitive that reads a clock domain at its previous activation; nominal clock domains and physical sinks with one explicit driver each. Above it, proved constructions rather than kernel constructs: list and product data with one recursor, a definitional equation library with rank-1 instantiation by matching, units as coordinates with exact affine charts, natural binder syntax as conservative desugaring, behavior components and groups that flatten into the same kernel, the canonical interface type $() \rightarrow B$ whose kernel value is B , and the provision of a Source at deployment by a raw reading and a pure transducer, proved transparent to the design. A separate validation layer decides hardware feasibility and collection capacity. Production implements the language as a Rust toolchain — model, elaborator, checker, reference evaluator, `no_std` code generator, daemon, IDE service — and a Flutter authoring environment, Studio, whose every semantic verdict is a projection from the compiler.

Formal scope. Thirteen phases, 55 Lean modules, no sorry, propositional extensionality and quotient soundness as the only axioms, no classical choice. Every theorem is about the *model*; none is about the Rust or Dart code.

Production scope. Production is described as of commit `de8154f5153495de2ad8a09f3ca3166c3678dc93` of KCN-judu/BDL, 2026-09-20 (after the syntax-highlighting milestone; protocol 0.21). The formal development is described as of the working tree that contains this revision of the document, whose last pushed commit is `af25567` (Phase 13). Every sentence about production is a sentence about that commit; volatile details are gathered in the production snapshot (Part XIII and Appendix F) so that the conceptual chapters do not go stale with the next milestone. Where a statement was only checked at an earlier snapshot, the text says so.

How to use this document. Read Parts I–II for the problem and the language as a designer meets it. Parts III–X are the formal development organized by concept, not by the order in which it was done: the declaration model, semantic identity and quantities, the equation language, units, reactive semantics, behavior systems, the environment and physical boundaries, validation and deployment. Parts XI–XII describe production: the compiler, runtime and daemon, then Studio and the IDE. Part XIII is the formal $\leftrightarrow$ production correspondence, construct by construct, with every deviation and its reason. Part XIV is minimality and the rejected alternatives. Part XV is the open agenda, with the empirical questions kept apart from the formal ones. The related-work chapter places BDL against the traditions it borrows from. The appendices are the reference apparatus: notation, the theorem index by concept, the decision index, the evidence-strength ledger, the migration map from the old decision numbers, the

production snapshot, the development chronology and the revision log. Phase numbers (Phase 0 ... Phase 13) appear throughout as *provenance* — the order in which hypotheses were tested — never as the structure of the exposition.

Claim strength

Every substantive claim carries one of the following labels, used in the text, in the ledgers of the appendices, and — with production’s own words — in production’s records (docs/project/formal-correspondence.md there, and the fv field of each ADR). The labels are never flattened into “BDL guarantees”.

label	means
formally proved	a named Lean theorem proves the stated property of the formal model; it never proves the Rust or Dart code
formally characterized under restricted hypotheses	a named theorem proves the property for a stated fragment (a wiring design, direct bindings, one domain); the restriction is part of the claim
mechanically executed example	a concrete design run through the proved-sound interpreter or solver inside the proof checker (decide, #eval); evidence for that input, not a theorem
counterexample / rejected by model	a candidate construct or claim was formalized and a theorem or executed case shows it wrong
informed by FV	a formal result or counterexample bounded an engineering choice; the choice itself is production’s
production implemented and tested	present at the production snapshot and exercised by a named test (differential, golden, property, end-to-end)
production architecture decision	an accepted production ADR; may or may not have formal backing, and the row says which
proposal / not implemented	a production proposal (PRP) or a formal construction with no implementation
design recommendation	guidance to production with no theorem behind it
open empirical question	a claim about designers or usability; no study has been run, and nothing in this document is evidence for it

“Minimal” is never claimed globally. Where the formal development says minimal it means *minimal among the tested candidates*, or *the smallest design found that supports the required cases*, and the text says which.

Identifiers

Formal design decisions are FVD-NNNN and formal open items FVI-NNNN (docs/decisions/, docs/issues/ in the formal repository); production’s records are ADR-NNNN, PRP-NNNN and ISS-NNNN (docs/decisions/, docs/proposals/, docs/issues/ in the production repository). Where a formal decision and a production record concern the same architectural question the text cites the two together — ADR-0032 (FVD-0118) — and no competing number is minted for a fact production already names. Until 2026-09-20 the formal decisions were numbered D-01 ... D-130 in a single ledger; Appendix E is the permanent map, and archived documents keep the old numbers. Theorem names are semantic identity and never change to track a document id: provision_transparent is that theorem whether its record is cited as PRP-0001, FVD-0126 or a section of this document.

Revision log

The full log is Appendix H; the entries that changed what the record is:

revision	what the record gained
2026-09, conference manuscript (Phases 0–7)	motivation, designer vocabulary, interaction model, the kernel through hardware validation, the claim-strength discipline; archived in paper/archive/
2026-09-18, the monograph rewrite	the monograph form: production architecture, correspondence and deviations, ledgers, the open agenda
2026-09-20, this revision	the conceptual restructuring (by concept, not by phase); Phase 13 integrated as the environment boundary; production at de8154f — Source / Rule / Value as one derived role, Sources driving outputs, applied_by and rule.apply, the generalized Standard Library, the unified project, the Formula Composer’s

revision	what the record gained
	structured forms, semantic highlighting, protocol 0.20/0.21; the FVD / FVI identifiers replacing the ledger numbers; the theorem, decision and evidence indexes; the two kinds of minimality; the publication-era residue removed

Part I — Purpose and Design Position

Introduction

Industrial design is increasingly concerned with products whose behavior is determined not only by geometry, materials, and mechanisms, but also by sensing, logic, timing, software, and networked control. A cup may infer that it has been lifted, a lamp may adapt to ambient conditions, a medical device may gate an action on multiple safety conditions, and a consumer robot may continuously map sensor estimates to actuator behavior. In such products, behavior is no longer a late implementation detail. It is part of the product concept.

Yet the dominant design media remain asymmetrical. Industrial designers have mature media for shape, layout, appearance, and physical assembly, while product behavior is usually externalized through prose, storyboards, flowcharts, state diagrams, interactive mock-ups, or ad hoc embedded code. The first four are easy to sketch but weak as executable specifications; the last is executable but forces the designer to adopt implementation-oriented concepts such as mutable variables, callbacks, polling loops, and device APIs. Existing physical prototyping systems such as Phidgets and d.tools substantially lowered the cost of building interactive prototypes [1], [2], but their behavioral representations still inherit important assumptions from programming and state-machine formalisms.

A concrete instance of the resulting cost appeared in a two-day introductory hardware workshop that the author designed and taught for ten participants with no prior embedded experience. Two of the available sensors behaved in opposite ways: the ambient-light sensor reports larger values as illumination increases, while the distance sensor reports smaller values as the target recedes. Participants lost track of which was which, repeatedly and across the whole group. This pattern suggests a representational problem rather than merely a syntactic one. Whether a reading rises or falls with the quantity it measures is a stable fact about a device, and in the code they were writing there was nowhere to record it. It survived only as a sign buried inside an expression and had to be reconstructed each time it was needed. The obstacle was not syntax. A piece of semantic information had no place to live.

BDL draws a different boundary. The goal is not to make engineering representations merely easier for designers to use. The goal is to define a *native representation of product behavior for design itself*, while retaining enough formal structure for static checking, simulation, and eventual implementation.

The system is **BDL**, a Behavior Design Language. BDL is organized around a working hypothesis: when a behavioral relationship is first specified, *what kind of relationship should exist* is frequently settled before its exact implementation is. A designer may know that **Tilt influences Brightness** before deciding the transfer function; that **CupPickedUp** should activate a behavior before deciding which sensor and threshold detect pickup; or that a safety condition should suppress an actuator before choosing the device driver. Therefore the primary design artifact should be the *typed relationship*, not the procedure that computes it.

The central example is a relationship. Instead of decomposing a design into procedural steps such as “read tilt,” “calculate brightness,” and “set the LED,” BDL represents one mapping:

$$? f : \text{Tilt} \rightarrow \text{Brightness} .$$

The mapping may exist before its body. A formula, curve, examples, or a fitted function can later be attached as a definition of the same block. Once a formula is supplied, for example

$$f(\theta) = \text{clamp} \left(0.2 + 0.8 \frac{\theta}{60^\circ}, 0, 1 \right),$$

it inhabits the previously declared signature. The claim attached to this **signature-first** model is deliberately narrow. It is not that designers universally think in signatures before bodies, nor that they should be trained to. It is that an unresolved typed relationship is a legal, statically meaningful state of the design, so that stopping between declaring a relationship and realizing it costs nothing. Whether designers make use of that position, and at what level of task complexity, is an empirical question that remains open; Part XV states it as such and lists the studies that would answer it.

Signature first; formula attached later

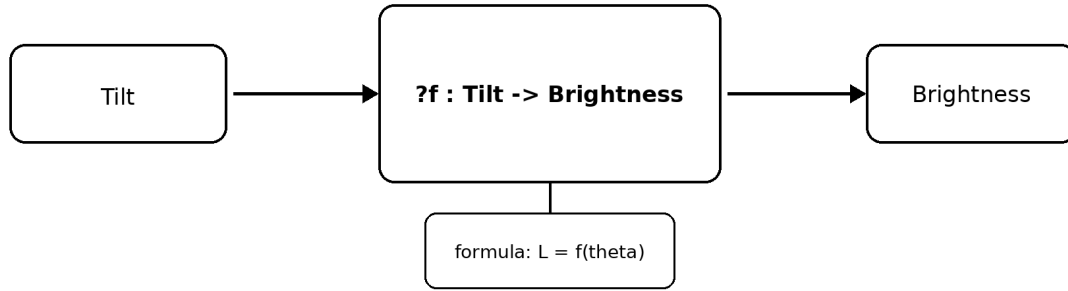


Figure 1: A signature-first relationship. The flow graph contains one semantic relationship, `Tilt -> Brightness`; the formula is attached to the block rather than represented as an additional execution step.

BDL separates a designer-facing surface language from a small formal kernel. The kernel presented here was not designed on paper and then implemented. It was derived by a mechanized design-space exploration in the Lean 4 proof assistant [3]: each candidate construct from an earlier draft of the language was formalized, attacked with counterexamples, reduced to other constructs where possible, and kept only when the tested alternatives failed for a stated reason. The result is smaller than the draft it replaced. Clock-indexed signal types, a separate event type, effect rows, action requests, and runtime actuator arbitration were all removed, each for a reason recorded in the accompanying development. What remains is an environment of named declarations with frozen expected types, monotone public commitments, and write-once realizations; nominal semantic types whose values can be constructed only inside a declaration whose own signature announces the concept; physical dimensions carried in the types of primitive operators; one temporal primitive that reads a clock domain at its previous activation; nominal clock domains with a separate domain judgment; and nominal physical outputs with a single explicit driver each. A validation layer outside the kernel decides whether a design can be placed on a declared target board. Later phases added, each for a reason recorded in Parts IV–VII, list and product data with one recursor, behavior components and groups, an exact model of units and charts, and an interface-level account of the canonical type $() \rightarrow B$ that leaves the kernel without a unit; nothing that was removed has returned.

The interaction model of Part II and the formal architecture of Parts III–X are two views of one language, and a third — the production system of Parts XI–XII — now exists: an elaborator, a compiler to an executable IR and a `no_std` Rust core, a daemon, an IDE service and the Studio authoring environment with its Formula Composer. What has *not* changed since the first draft is that no user study has been run: every statement in this document about what designers find natural is a hypothesis (Part XV).

The Representation Problem

The target user is not a programmer with fewer syntax skills

Many low-code and visual programming systems reduce textual syntax while preserving the underlying computational ontology: variables, assignments, loops, callbacks, functions, transitions, and scheduling. For software developers this can be convenient. For industrial designers, however, the dominant difficulty is often not syntax but *semantic translation*. The designer begins with a product statement such as “while the cup is held, brightness follows tilt” and must translate it into implementation machinery.

BDL therefore follows a stronger criterion: a surface primitive should be exposed only when it corresponds to a concept that is independently meaningful in the design task. A mutable accumulator used to count samples is generally not such a concept; “three pickup events within ten minutes” is. A polling loop is generally not; “while the product is held” is. A callback is not; “when the button is pressed” is.

The language should directly expose concepts, time-varying quantities, discrete occurrences, typed mappings, conditions, behavioral contexts, temporal relations, physical outputs, and safety constraints. By default it should hide program counters, threads, callbacks, continuations, clock variables, and bus transactions. These may remain inspectable in an expert or debugging view, but they are not the primary design medium.

A flow graph is a dependency view, not a program counter

BDL uses a flow-like canvas because causal and functional paths are useful visual structures. The arrows do **not** mean “execute the left node and then the right node.” They mean that the target relationship depends on the source relationship or value.

Consider the procedural decomposition:

```
[Cup picked up]
|
[Read tilt]
|
[Compute L = f(theta)]
|
[Set LED brightness]
```

This decomposition is rejected as a primary design representation because the “read,” “compute,” and “set” nodes are artifacts of an execution model. The corresponding BDL view is a `held` context containing a mapping from `Tilt` to `Brightness`. The formula is a property of that mapping, and the realization layer later binds the mapping’s value to a physical light output.

The distinction matters because it changes what the designer edits. In a procedure-centric editor, changing an implementation may require rewriting steps and intermediate state. In BDL, changing a transfer function edits the definition of one relationship while preserving the surrounding product logic. This is not only a matter of presentation. Realizing or refining one declaration leaves every other declaration’s typing untouched, and leaves every commitment that was already discharged in place, provided the evidence for those commitments does not depend on what was still unknown.

Cognitive budget

The surface language must remain intentionally small. Each additional primitive has a cost in learnability, visibility, consistency, and error-proneness, all familiar concerns in the Cognitive Dimensions tradition [4]. BDL therefore uses a *cognitive budget*: a new visible construct is justified only when it captures a distinct design concept that cannot be expressed cleanly as a property of an existing construct.

The same budget was applied to the kernel, under a stricter test. A kernel construct earns its place only if removing it makes some design either unrepresentable or ambiguous, and the argument for each is a theorem or a counterexample rather than a preference. Several constructs that the surface exposes as distinct concepts turned out, under that test, to be derived forms. Every temporal modifier reduces to one delay primitive. Every cross-domain policy reduces to one transport primitive plus ordinary data. Every output-selection policy reduces to ordinary computation upstream of a single drive edge.

Progressive disclosure is a semantic property

A design tool can be visually simple and still force premature decisions. BDL instead treats progressive disclosure as part of the language semantics. A design may contain a declaration whose realization is intentionally absent. Later steps may refine the same declaration with mathematical properties, a body, a clock domain, an output binding, and a target board. Each step is either a *refinement*, which preserves everything previously established, or an *edit*, which is permitted but reopens the validation of dependents. That distinction is made precise in the section on declarations, and it is the organizing principle of the interaction model.

Scope and non-goals

In scope. A designer-facing language for the behavior of physical products — concepts, relationships, timing, outputs, components — with a formal kernel small enough to reason about and a production toolchain that follows it: authoring in a graph and in text, checking while incomplete, simulation, target-relative deployment validation, and code generation for a deterministic embedded core. The formal development’s job is to say what the kernel must contain and to reject what it need not; the production system’s job is to build everything the kernel deliberately omits — the elaborator, the tooling, the execution path — without redefining the language.

Out of scope, by decision. BDL is not a general-purpose programming language: there is no general recursion, no user-defined type abstraction, no effect system, and the equation language is total and first-order in its data (Part V).

It is not a continuous-time or hybrid modelling language: time is a global tick with named domains, and continuous dynamics are outside the model (Part VII). It is not a systems-engineering framework: requirements, verification of physical properties beyond typing and dimension, and electrical or thermal budgets are not modelled (Part X). It is not a runtime with scheduling policy: the strictly-before rule removes the scheduler from the semantics, and the generated core has no tasks (Part XI). It does not verify its own compiler: the production code is held to the model by tests, and the refinement proof is an open item (Part XV).

Not claimed. That designers think signature-first; that Studio is usable; that the kernel is minimal in any absolute sense; that any theorem says anything about Rust or Dart. Each of these is either an open empirical question or a restriction on a claim, and the text says which wherever it arises.

Part II — The Designer-Facing Language

This Part is the language as a designer meets it in Studio and in a .bdl file at the production snapshot, with the vocabulary the product uses. Every construct here is a surface form; Parts III–X state which forms are kernel primitives and which are derived, and Part XII describes how Studio draws them. Two kinds of surface construct are kept apart throughout: those that exist in production (**production implemented and tested**) and those that were designed, reduced to the kernel in the formal development, and are not yet offered by the product (**design recommendation**; production’s ISS-0010 tracks the largest of them). The Part ends with one end-to-end scenario and the states a design passes through.

For every mechanism the same five questions are answered, here or in the Part that owns it: what problem the designer experiences, what concept the language exposes, what formal mechanism supports it, what implementation owns it, and what complexity is hidden.

Concepts

The first-class object is a **concept**, not a raw scalar: Tilt, Brightness, Temperature, CupContact, MotorAngle. A concept has a **value form** — a quantity with a physical kind and a unit (Tilt : Angle), on/off, a count, a collection of another concept’s values, a grouped value, an optional value — or none yet (*decide later*); it may carry a description, a preferred display unit and, for a quantity, a declaration that it is **ordered**. Its identity is what the language tracks: Tilt and MotorAngle may both be angles and are never interchangeable, and a link between sockets of different concepts is refused before any formula exists.

Designer problem: the workshop cost of Part I — a stable fact about a value with no place to live. *Language concept*: the named concept with its value form. *Formal mechanism*: the nominal type semantics and the write-once representation binding Θ (Part IV). *Owner*: bdl-model (identity, value form), bdl-elab (Θ), bdl-check. *Hidden*: the type constructor, the grant, the representation; the canvas shows a hue and a socket shape.

In text:

```
concept Tilt : Angle
concept Brightness : Scalar
ordered concept Severity : Scalar
concept Readings : List<Tilt>
```

Relationships and the three roles

A **relationship** is created from a **signature**: what it reads and what it produces.

```
mapping dimByTilt : Tilt -> Brightness      // reads Tilt, produces Brightness
mapping brightness : () -> Brightness        // reads nothing, produces Brightness
mapping tilt : () -> Tilt                    // reads nothing, no formula: a Source
```

The keyword is mapping; the product word is *relationship*. A relationship has a stable identity, a display name, a signature, an optional **formula** (its realization) and, when it lives in a timing domain, a domain (@interaction). Every relationship has one canonical type domain(inputs) -> B, with the empty product () as the domain of a relationship that reads nothing; mapping f : B is accepted as legacy shorthand with a hint and a fix, and f, f() and f(()) are one reference (ADR-0029, Part IX).

What a relationship *is* is one of three **roles**, derived from two authored facts — whether it reads anything, and whether it has a realization — never persisted, never chosen, never an identity (ADR-0032 (FVD-0118), production’s bdl_model::RelationshipRole):

role	when	has a value at a tick?	how a formula uses it	may drive an output?
Rule	it reads something (its type is an arrow)	no — it is a function	applied: dimByTilt(tilt)	no — its type is an arrow
Value	it reads nothing and has a formula	yes	by reference: brightness	yes, when its concept and domain are the output’s
Source	it reads nothing and has no formula	yes, once the environment supplies it	by reference: tilt	yes, on the same condition

The rule has one home, the model, and is stated on every projection (protocol 0.20); Studio maps the enum and re-derives nothing. *States* — declared, open, invalid, valid, not applied, driven, bound, clocked — vary within a role and never move it. A Source is complete, not unfinished: nothing is missing from it, the environment provides its value once per activation, and it is drawn with a boundary bar on the environment side and the word *Source*, never dashed. The word *sensor* is not used: a button state and an external configuration value are Sources too, and what realizes a Source is deployment’s business (Part IX).

Designer problem: stating that brightness depends on tilt before deciding how; distinguishing a value the product computes from one it observes. *Language concept*: signature, relationship, the three roles. *Formal mechanism*: a declaration with an interface and an optional realization (Part III); the unit-domain normalization and the source role as a realization state (Part IX). *Owner*: bdl-model (MappingBlock::role), bdl-compiler (MappingAnalysis.role), the daemon (MappingView.role). *Hidden*: currying, unit elimination, the kernel’s single input stream.

Incomplete designs are legal

Stopping between declaring a relationship and defining it costs nothing, and the product has several such stopping states, each drawn and explained rather than flagged as an error:

- a **declared** rule — a relationship that reads something and has no formula — drawn dashed with the one word *declared*; the design around it is checked as far as it can be, and other relationships may already apply it;
- a Source with **no value yet** in the simulator — the input control is drawn empty with a dashed outline; nothing is defaulted to false or 0, and the run does not step until every Source has a value;
- an **open** formula — one that waits on a concept whose value form is not yet chosen — a solid node with a hollow socket and the sentence *checked once Temperature’s value is decided*;
- a rule **nothing applies** — reactive.rule_unapplied, an informational finding, never an error — drawn with a hollow output socket and the words *not applied*, with the fix rule.apply offered: *Add a value that applies dimByTilt*, which creates `brightness : () -> Brightness = dimByTilt(tilt)` when each read concept has exactly one producing value, offers a choice when one has several, and is blocked with the reason when one has none;
- a formula with a **slot** ? — an expression not yet written, ordinary draft text that is saved with the file and refused by elaboration until filled (Part XII);
- a file that **does not build yet** — the text is kept as typed, the graph shows the last version that did build, and the reasons are listed (ADR-0023, ADR-0030).

Designer problem: conventional tools have no legal position for an undefined relationship, so a signature-first strategy cannot be adopted even when it would help. *Formal mechanism*: an unresolved declaration is a declaration whose realization is none and clients are typed against its interface (Part III). Whether designers use these positions is an **open empirical question** (Part XV).

Formulas

A formula is written in a small expression language: arithmetic and comparison, && / || / ! (drawn as *and*, *or*, *not* in the Composer), if ... then ... else, match over on/off, optional and numeric values, let, calls of the **equation library** (clamp, min, max, inRange, any, all, map, filter, sum, zip, contains, getOrElse, ...), rules `x => ...` as arguments, membership `x in [a, b, c]`, list and pair literals, quantity literals with units (90 deg), memory delay(init, e), carrying across domains sync(domain, init, e), and the natural forms:

```
all reading in readings: reading in 10 deg .. 45 deg
map r in readings: r / 90 deg
tilt ?? 0 deg
```

The formula’s inputs are the concepts of its signature by their names; a formula attached to a relationship that produces Brightness is written as a scalar expression and the elaborator supplies the wrapping — there is no constructor to type, and no other concept can be manufactured inside it. Units belong to literals: 90 deg is the designer’s, tilt is a Tilt and carries no unit of its own; changing the unit of a literal with the quantity kept is a spelling change, editing its coordinate is a change of meaning (Part VI). The natural forms are one-way spellings of library equations: a binder local is the equation’s lambda parameter, a range is inRange, ?? is getOrElse, and nothing new is evaluated (Part V).

The **Formula Composer** shows the same formula as the compiler reads it — reference chips with socket glyphs, literals as a coordinate and a unit pop-up, dashed slots, operators, calls, binders over an indented body, a choice as if with its condition over then and else — with, for the selected component, the compiler’s expectation and its reason (*Expected: an angle, because an angle ÷ an angle = a dimensionless quantity*) and the candidates of that kind. Every structured action is a text edit of the same formula. Richer forms — let, match, blocks, rules as arguments, collection literals — are drawn as text inside the Composer and edited as text (Part XII).

Time

The designer sees two kinds of temporal thing: a **quantity** that has a value whenever its domain is active, and an **occurrence** that may or may not be present at an activation. Underneath, the distinction is not one of type: every relationship is a stream under the tick semantics, and an occurrence is an optional value (Part VII).

Memory. `delay(init, e)` is last activation’s value of `e`, and `init` before there was one. Every remembered value has an explicit first value, and the tool asks for it — what brightness does the lamp show before it has ever been tilted? — because it is a product decision, not an implementation default. A relationship that depends instantaneously on itself is reported at the loop, in the names of the relationships, with the suggestion to remember one of them.

Timing domains. “Contact and orientation move with the interaction; temperature moves with the environment” is a statement about which values update together, not about how often. It is recorded by naming a **timing domain** on a relationship (`@interaction`), with no rate attached. The first time a relationship reads across domains the tool stops at that reference: the two values update in different rhythms, and it asks how the reader should see the source — as the last value the source produced, with a stated value to use before the source has ever reported (`sync(environment, init, e)`, chosen in the *Starts at* sheet), or by moving the reader into the source’s domain. Both are legitimate designs; the initial value of a safety interlock is exactly the decision worth writing down. Rates belong to deployment.

Temporal modifiers and contexts — designed, not offered. The earlier draft of BDL (archived in `paper/archive/`) described temporal phrases as qualifiers on relationships — `p` for 300 ms, `after e` by 2 s, `while p`, `until e`, `since e`, `once e`, `every 1 s`, `rise p`, `previous x`, `count e`, `hold x e` — and **contexts** (StateHandlers): named product situations, activated by a condition or entered and left by occurrences, containing a local flow graph and nested contexts, so that “while the cup is held, brightness follows tilt” is a region rather than a transition table. The formal development elaborated and executed each of these over the one memory primitive (Part VII): `previous`, `hold`, `count`, `since`, `once`, `every` and `rise` as self-delayed declaration shapes; a context as an activation declaration, an entry edge, gated and reset local state, an inactive default, and a conditional in the one relationship that drives an output. The reduction is **formally characterized for the tested cases** — condition-scoped activation, entry, reset-on-entry state, inactive default, event-latched activation with exit-wins, state-local output selection, nested selection with an output — and contexts with their own timing domain and independently clocked nesting were not examined (FVI-0009). Production offers neither the phrases nor contexts at the snapshot: a designer writes `delay` and `sync` directly, and the shapes above are written out by hand (ISS-0010). Their surface is a **design recommendation** carried here so that when they are built they are built as elaborations and not as kernel constructs.

Designer problem: history without explicit state; different rhythms without a scheduler. *Language concept:* memory with a first value; named timing domains; carrying across domains with a first value. *Formal mechanism:* `delay`, `sync`, `strictly-before`, the domain judgment (Part VII). *Owner:* `bdl-reactive`, `bdl-lower`. *Hidden:* the tick, the schedule, the two-phase step.

Physical outputs

A design computes values; it does not move hardware. Physical effect happens only through an **output** — `output light : Brightness @interaction` — and one **drive** connecting it to exactly one relationship that produces its concept in its domain: a Value, or a Source (a value the environment supplies may be passed straight through to a light); never a Rule, whose type is an arrow. Where several behaviors would influence one output — a safety override and an interaction — they are combined by an ordinary relationship that becomes the one driver, and the combination rule (`priority`, `blend`, `maximum`, `clamp`) is written in the design where it can be read. A second drive on the same output is refused as *contested*, with the message on the output, in the product’s words: *this output already has a driver; combine the two brightness values before connecting it*. An output with no driver is reported until the design is executable.

Below the output is the **device** that realizes it on a board (device pwmLight : pwm_channel for light), and below that the board's pins; neither enters the behavior model (Part X).

Designer problem: two behaviors reaching for one light, and a runtime rule chosen silently. *Formal mechanism:* nominal sink identity, the write-once drive edge, DriveWF, SingleDriver, CompleteOutputs (Part IX). *Owner:* bdl-output. *Hidden:* nothing — the point is that the arbitration is on the canvas.

Behaviors, components and systems

Relationships can be gathered into a **behavior** — a named group, drawn as a box, collapsible to its boundary sockets, moved, merged and split with no change to the design and no re-check (ADR-0019 (FVD-0074 ... FVD-0082)). *Package as Component* turns a behavior into a **component**: a reusable design with a **promise** — what it requires, provides and takes as a parameter, which timing domains it is written against, which concepts it shares with the system — inferred from the boundary and widened by the designer, never narrowed below what the members read. An **instance** is one use of a component with fresh identity; a **binding** connects a provided port or a value to a required port or an open relationship, directly in one domain or carried across domains with a first value; a **version** is a copy with the same promise. Every project is a **system** — a design with no components is the degenerate one — and the flat design every check runs on is derived, never persisted (ADR-0021, ADR-0022 (FVD-0064 ... FVD-0073)).

```
component AdaptiveLamp {
  use concept Tilt
  use concept Brightness
  param clock main
  requires tiltValue : Tilt @main
  mapping dimByTilt : Tilt -> Brightness
  dimByTilt(t) = t / (90 deg)
  provides brightness : Brightness @main
  brightness() = dimByTilt(tiltValue)
}
instance lamp : AdaptiveLamp { main = interaction }
bind lamp.tiltValue = tilt
```

Inside a component, the relationship behind a required port is a Source of the body — provided through the port — and Studio wears the port's word for it; an unbound required port of an instance is a Source of the system, a simulation input. The role is a fact of one design, and the same declaration can have a different role in the body, in the flat design and at the top level (production's relationship-roles matrix; Theorem H of Part VIII).

Designer problem: reuse without copying; organizing a large design before deciding its interfaces. *Formal mechanism:* groups as authoring metadata proved transparent; components as templates instantiated with fresh identity and bound by ordinary realization steps; flattening (Part VIII). *Owner:* .bdl/authoring.json (groups), bdl-system. *Hidden:* renaming, the origin map, the flat design.

The Standard Library

The Library is an **authoring catalogue**: items that create ordinary objects in the design in one transaction. A *Concept* item creates a concept with its value form; a *Source* item creates a concept and a () -> concept relationship with no formula — a Source by the derived rule, exactly as one made by hand. The library carries no role and no flag; instantiating an item is the same edit sequence a designer could perform, undone as one step. Item names and descriptions are localized by Studio; identifiers never are. The Library is not a device catalogue: what realizes a Source on a board is a deployment concern (Part IX), and the device catalogue that concern needs does not exist at the snapshot.

Supplied computation — designed, not built

Some behavior is easier to state as code than as a diagram: recursive filters, estimators, self-tuning controllers. The design admits computation blocks written in the host language behind a signature the designer places first — smooth : Distance -> Distance as the specification an engineer works against — with the properties the kernel cannot derive for supplied code (determinism, totality, range, state size, the rate it was designed for) declared as validation obligations, and with one rule the kernel does enforce: supplied code may not drive an output (ADR-0005 (FVD-0050 ... FVD-0056)). The trust boundary is designed (docs/architecture/component-boundary.md in production); no implementation exists.

A representative scenario

A table lamp responds to being handled: when picked up and tilted its brightness follows the tilt; when set down it holds the last brightness. A heater in its base keeps a drink warm, with a temperature sensor beside it; above a warning temperature the lamp pulses, above a critical temperature the heater must switch off regardless of anything else. The product is to run on an Arduino Nano, chosen last. The scenario is small and exercises everything the model offers: relationships known before their formulas, history without explicit state, a safety condition competing with an interaction for one output, two rhythms, and a board met last. The common path below is what Studio does at the snapshot; the two places where the designed vocabulary goes beyond the product are marked.

Starting from intent. The designer names *Tilt*, *Brightness*, *Temperature*, *Held* — none a sensor or a number — and adds from the Library the *Tilt Sensor* and *Temperature Sensor* items, each of which creates its concept and a relationship with no formula in one step — renamed here to `tilt : () -> Tilt` and `temperature : () -> Temperature` — each a Source by the derived rule, each complete, each provided by the environment. The first relationship is drawn from *Tilt* to *Brightness*; it becomes `dimByTilt : Tilt -> Brightness`, a declared rule, dashed, not an error. The design now says that brightness depends on tilt and nothing about how. A property such as *increasing in tilt* cannot yet be recorded — commitments are a kernel slot with no production authoring (Part III) — but a link from *Tilt* to a motor-angle concept would already be refused.

Refining locally. Opening `dimByTilt` shows the Formula Composer. The designer chooses *Function* → *clamp*, fills the slots — `clamp(tilt ÷ 90 deg, 0, 1)` — with the expectation and the units offered at each slot, and the rule checks. The canvas still shows one arrow. The rule is *not applied* until something applies it; the offered fix creates `brightness : () -> Brightness = dimByTilt(tilt)`, a Value, which is what the simulator shows and what an output can read.

History. (*designed vocabulary*) The phrase *hold while not Held* on the wire is the temporal-modifier surface, not offered at the snapshot; the designer writes the shape the phrase would elaborate to — `brightness() = if held then dimByTilt(tilt) else delay(0, brightness)` — and is asked for the first value.

Contexts. (*designed vocabulary*) A *Held* region containing `dimByTilt` is the context surface, not offered; the activation condition is an ordinary on/off relationship and the gating is written in the driver, as above.

One output, one driver. `output light : Brightness @interaction` is driven by *brightness*. The warning pulse, *warmPulse*, cannot be connected to *light* as well; the repaired design has one more relationship, *lampTarget*, whose formula says whether the pulse takes priority while warm, or the maximum is shown, and *lampTarget* drives *light*. The heater's cutoff is the same shape: *heaterTarget* forces the demand to zero above the critical temperature, visibly, upstream of the output.

Two rhythms. *temperature* updates with the environment, *tilt* and *held* with the interaction: two timing domains, `@environment` and `@interaction`, no rates. *heaterTarget @interaction* reads the critical condition from `@environment`; the tool stops at the reference and the designer chooses *carry across, starting at released* — `sync(environment, false, critical)` — writing down what the interlock does before the first temperature reading.

Choosing a board. The outputs are given devices — the light a PWM channel, the heater a switched load — and the Deploy page, on selecting a Nano, derives what the design needs and proposes a placement, or reports on the requirement that could not be placed and names what occupies each candidate pin. Nothing on the canvas changes; a larger board places the same design; a manual pin is a constraint on placement, not a change to the design (Part X).

Simulating. The Simulate page lists the Sources — *tilt*, *temperature*, *held* — as the inputs, refuses to step until each has a value, and shows every Value and every output per tick, with a probe naming for each concept what carries it and, for a rule, where it is applied.

What the workspace says

Figure 2 lists the states a design passes through, as the product reports them; each answers what is unresolved, what is settled and what can be done next.

State	What is settled	Where
declared	a rule is named and typed; others may apply it; no formula yet	<i>canvas</i>
open	a formula waits on a concept whose value form is not yet chosen	<i>inspector</i>
invalid / valid	the formula does not check / produces what the signature promises	<i>inspector</i>
not applied	a rule no value applies; the value that would is offered	<i>canvas, Simulate</i>
temporally valid	every remembered value has a first value; no instantaneous loop	<i>canvas</i>
clock-consistent	every read across timing domains has a transport and a first value	<i>canvas</i>
executable	every needed relationship valid; every required output driven exactly once	<i>outputs</i>
feasible	the chosen board can carry every derived requirement, and every collection fits	<i>Deploy</i>

Figure 2: States a design passes through, as the workspace reports them. All but the last are properties of the design alone; the last is a property of the design together with a board, and is re-established whenever either changes.

A relationship that is declared but not defined is shown as open, and the design around it is checked as far as it can be. A definition that does not check is marked at the definition, in the vocabulary of the relationship. A loop or a missing first value is marked at the wire. A missing driver is marked at the output. A conflict on the board is marked on the Deploy page, at the requirement that could not be placed. *Feasible* is separate from everything above it because it is the only state that depends on something other than the design: six PWM channels fit a Nano, a seventh may not, and nothing about the first six changes.

Looking underneath

Most of the time the compact surface is all a designer sees. **Explain**, a collapsed disclosure at the end of the inspector, opens the selected object to what it became: the canonical type and the kernel interface, `role: Source | Rule | Value`, the Core term a formula elaborated to, the memory cell behind a `delay`, the transport and the tick at which a source was last read, the drive edge and the requirements derived from a device. It is where the kernel vocabulary is allowed to appear, for the engineer who receives the design and wants to know exactly what was generated, and it is the answer to the hidden-elaboration risk of Part XV: the surface stays simple because elaboration does real work, and the work is inspectable rather than mysterious.

Part III — The Core Declaration Model

Method

The kernel was obtained by a method, and the method is part of the record. Each phase of the development took a family of candidate constructs from the earlier draft, formalized the smallest plausible version and its alternatives in Lean 4 without external libraries, and attacked each with the same questions. What does it reject that the others accept? What does it accept that it should not? Is it a special case of another? Which operations on a design are refinements under it, and which are edits? Constructs were promoted from the experiment modules to the kernel only after surviving, and the theorems about them were re-proved in the promoted form.

The formal development labels each conclusion with one of six strengths — formally proved, formally rejected by counterexample, reduction by proof, tested formulation redundant, engineering preference, not ruled out — and this document maps them onto the five labels of the front matter: the first three are **formally proved** (a counterexample and a reduction are theorems), the fourth and fifth are **informed by FV** or **design recommendation** according to whether a mechanized example bounded the choice, and the last is stated as open. Nothing in the development is a proven impossibility, and “minimal” always means minimal among the tested designs.

Scale and trust base

The development builds with Lean 4.33.1 with no sorry. The axioms used by every theorem are propositional extensionality and quotient soundness, the latter only through function extensionality and the choice-free rational quotient of Part VI; classical choice is absent, and each phase re-audited the whole development for it. As of Phase 13 the sources are 55 modules: 11 in Core, 12 in Behavior, 12 in Surface, 2 in Validation, and 18 experiment modules holding alternatives, counterexamples and executed examples. Every trace, assignment, unsatisfiability result and executed example reported here was obtained by running a proved-sound interpreter or solver inside the proof checker. Several theorems are recorded as trivial by definition — the typing half of client stability is a one-liner, and the well-formedness of a refinement target is unused because the invariant was moved into the definition — and they are reported as such rather than presented as content.

From Surface to Kernel: Architecture

The system has three layers, and the boundary between them is the main architectural result of the formal development.

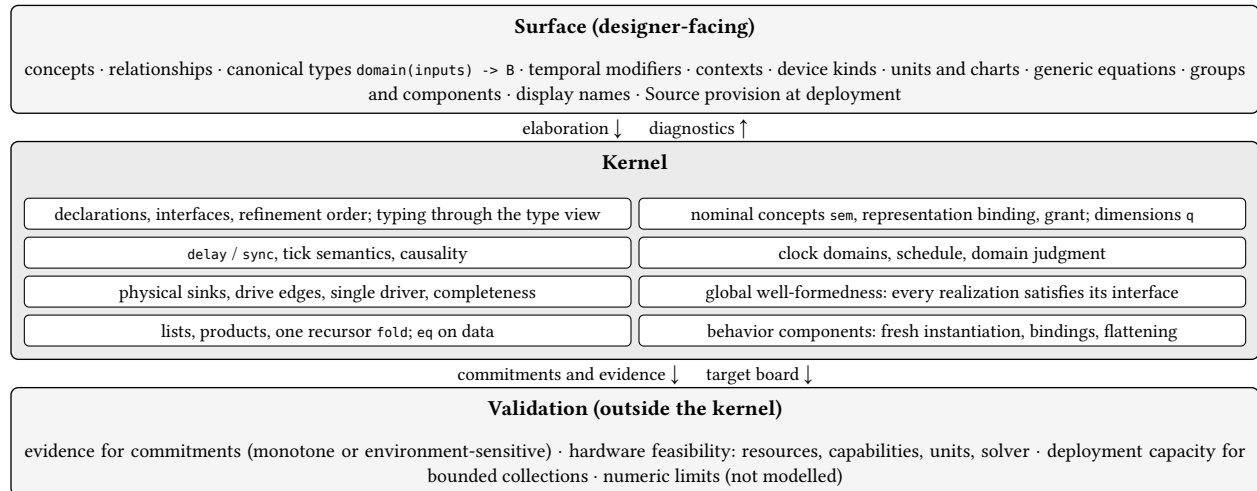


Figure 3: The three layers. Typing consults only the type view of declarations and the representation view of concepts; validation may consult commitments, evidence, and the target; the surface is derived forms over the kernel.

The **surface** is what the designer authors: concepts, relationships, temporal modifiers, contexts, device bindings, units, generic equations, groups and components, and display names. Everything in it elaborates to kernel objects, and the elaboration is one-directional: the kernel never needs to recover surface structure.

The **kernel** is the formal object of Parts III–IX. It consists of an environment of declarations, a typing judgment, a tick-indexed evaluation relation over one or several clock domains, a domain judgment, and a small number of global well-formedness conditions: every realization satisfies its interface, the instantaneous dependency graph is acyclic, every reference respects domains, and every physical output has at most one driver. The typing judgment reads only the *type view* of declarations — their expected types — and the *representation view* of concepts. It does not read realizations, commitments, evidence, clocks, or bindings.

The **validation layer** is everything that may depend on more than types. It discharges the commitments a declaration makes, and it decides whether a design fits a target board. Two kinds of evidence live here and are kept apart. Evidence that is meant to survive refinement — a monotonicity commitment discharged compositionally through the commitments of other declarations — must be stable under monotone extension of the environment, and the kernel imposes that condition. Evidence that is not meant to survive refinement — the existence of a pin assignment on a particular board — is re-established after every change and is never merged with the first kind.

Figure 3 shows the layers as they stand after Phase 13; the kernel band also holds list and product data with one recursor (Part V) and the behavior-component constructs (Part VIII), the surface band holds the deployment construction of Part IX, and the validation band holds deployment capacity (Part X). What is notable about the arrangement is how much of the earlier draft of BDL is absent from the kernel band. Reactive types, event types, effect rows, action requests, policy transformations, and a five-phase tick with a resolve step were all part of the draft kernel. Each was removed because it either added no rejection the smaller kernel lacked, or made a design decision on the designer’s behalf that should have been visible in the design. The states of Figure 2 are the designer-facing face of the same structure: *declared* and *type-valid* are the typing judgment and satisfaction; *temporally valid* is causality; *clock-consistent* is the domain judgment; *output-complete* is the single-driver and completeness conditions; *hardware-feasible* is the validation layer’s solver. Each is decidable for finite designs.

Declarations, Interfaces, and Refinement

The foundational kernel object is a **design declaration**,

$$\text{DesignDecl} = \langle id : \text{DeclId}, \\ interface : \text{DeclInterface}, \\ realization : \text{Option Expr} \rangle,$$

where an interface is an expected type together with a set of public commitments:

$$\text{DeclInterface} = \langle expectedType : \text{Ty}, \\ commitments : \text{PropertyId}^* \rangle.$$

A design is an environment $\Delta : \text{DeclId} \rightarrow \text{Option DesignDecl}$. An unresolved declaration is one whose realization is none; nothing else distinguishes it. Display names are not part of the kernel.

Typing through the type view

Terms refer to declarations by identity, $\text{declRef } d$. The typing judgment $\Theta; \Delta; G; \Gamma \vdash e : \tau$ takes a concept environment Θ and a grant G , both introduced in the next section, and a declaration environment Δ which it consults through exactly one projection, the type view $\Delta^{\text{ty}}(d)$, the expected type of d if declared:

$$\frac{\Delta^{\text{ty}}(d) = \text{some } \tau}{\Theta; \Delta; G; \Gamma \vdash \text{declRef } d : \tau}.$$

This is the only rule that reads Δ . A client of d is therefore typed against d ’s interface and never against its body, whether or not a body exists. Inference is syntax-directed and decidable, and an inference function is proved sound, complete, and unique against the judgment.

Satisfaction, well-formedness, and refinement

A realization e **satisfies** an interface S when it has the expected type under the grant of that type and discharges every commitment:

$$\begin{aligned}
& \text{Satisfies } ev \ \Theta \ \Delta \ \Gamma \ e \ S := \\
& \quad \Theta; \Delta; \text{Grant.of}(S.ty); \Gamma \vdash e : S.ty \\
& \quad \wedge \forall p \in S.commitments. \ ev \ \Delta \ e \ p,
\end{aligned}$$

where $S.ty$ abbreviates the expected type. Here $ev : \text{DeclEnv} \rightarrow \text{Expr} \rightarrow \text{PropertyId} \rightarrow \text{Prop}$ is an abstract **evidence** relation supplied by the validation layer. It takes the environment as an argument because compositional discharge needs it: “ A is monotone because B is committed to be monotone” consults B ’s interface. A design is **globally well formed** when every stored declaration sits under its own identity and its realization, if any, satisfies its interface in that design.

Interfaces are ordered by monotone refinement: $S \sqsubseteq S'$ when the expected type is unchanged and the commitments of S are contained in those of S' . A declaration takes a refinement step in one of three ways. An unresolved declaration may have its interface refined; an unresolved declaration may be realized with a satisfying body; and a realized declaration may have its interface strengthened, provided the body is re-verified against the new interface. The reflexive-transitive closure of these steps is exactly the structural order together with well-formedness of the target, where the structural order DeclLeq requires the same identity, interface refinement, and a write-once realization, and $\text{EnvRefines } \Delta_1 \ \Delta_2$ lifts it pointwise while permitting new declarations.

Client stability

Can a declaration be refined or realized without editing its clients, and without invalidating what was previously established about them? The answer has two halves with deliberately different hypotheses.

The typing half needs only the structural order. If B is declared in Δ and $\text{DeclLeq } B \ B'$, then every typing judgment in Δ holds in $\Delta[B']$. This is a one-line consequence of typing references through the type view, and it should be read as such: its content is that the decision to let clients see interfaces and never bodies is *sufficient* for client stability. It is also necessary. Change B ’s expected type while keeping its identity, and every client breaks.

The commitment half needs more. If Δ is globally well formed, B takes a refinement step whose side conditions are checked in Δ , and the evidence relation is **monotone** — stable under EnvRefines — then $\Delta[B']$ is globally well formed, and the same holds for a whole lifecycle of B checked against the original environment. The monotonicity hypothesis was not part of the original design. It appeared when the theorem was attacked. An evidence relation that consults the *absence* of information — one that discharges a property because a dependency is still unresolved, say — is destroyed by a perfectly valid realization step, and the non-monotonicity of that evidence can be derived from the failure of preservation. Any discharge mechanism intended to survive refinement must therefore be positive in the environment.

Refinement versus edit

The preservation theorems cover refinement only. The table lists the operations examined and their classification; each row is witnessed by an example on a two-declaration design in which $A : \text{nat} \rightarrow \text{bool}$ is realized through an unresolved $B : \text{nat} \rightarrow \text{nat}$.

Operation on B	Kind	Effect on A
add a public commitment	refinement	typing and commitments preserved
realize	refinement	preserved; A now unfolds to a closed program
strengthen a realized interface	refinement, with re-verification	preserved
change the expected type, keep identity	edit	typing broken
drop a commitment	edit	typing silent; A ’s commitment broken
replace B by a new identity	edit	dangling reference
add a new B' beside B	edit	A still depends on the stale B ; never executable
detach or replace the realization	edit	evidence that consulted the body is void
assign or change a clock domain	edit	domain judgment on clients broken
retarget an output binding	edit	completeness or single-driver may break

Two rows deserve comment. Dropping a commitment changes no type, so the type checker is silent, yet A ’s own commitment was discharged through B ’s and is now unsupported. Commitments are therefore part of the interface

in the same load-bearing sense as the expected type. That is a stronger position than the earlier draft of the language took when it described properties as merely attaching to a name. Detaching a realization, which that draft permitted as an ordinary operation, is an edit for the same reason: clients' typing is unaffected, but any evidence that consulted the body is void. The kernel does not forbid edits. It declines to promise anything about them, and the tool must reopen the validation of transitive dependents.

What persistent identity is

The earlier draft left open whether a persistent, referable identity for an unresolved relationship is a novel abstraction. It is not. Every theorem that mentions identity uses it only to make an update land on the slot a reference resolves to, which is what a name does in any environment-based semantics. The non-trivial content lies in the environment order, and in the two facts that clients depend on it only through the type view for typing and only through monotone evidence for commitments. This is the familiar interface/implementation separation of module signatures, of a parameter later given a definition, or of a metavariable context with write-once assignment and fixed types, together with a Kripke-style stability condition on evidence. What is slightly non-standard is that the interface carries a growable commitment set whose growth is a first-class operation on a declared-but-undefined name, and that the kernel imposes a stability condition on the validation layer. Neither is a new type-theoretic mechanism, and no such claim is made.

Where the rest of the kernel is described

The declaration model above is the frame every later result is stated in. Semantic identity, representation binding and dimensions are Part IV; the data core, the equation library and the natural surface Part V; units and charts Part VI; the reactive semantics, clock domains and buffering Part VII; behavior components and groups Part VIII; the environment boundaries — the canonical type $() \rightarrow B$, the source role, physical sinks and provision — Part IX; hardware and capacity validation Part X. The minimality verdict for each construct is given where the construct is discussed and gathered in Part XIV.

Part IV — Semantic Identity and Physical Quantities

The kernel of Part III types declarations and lets clients depend on interfaces; it says nothing yet about what a type *means*. This Part adds the two things a product concept carries that a number does not — an identity that survives representation, and a physical dimension — and shows, by the counterexamples that decided each, why structural numeric identity is insufficient, why observation and construction of a concept’s representation must be treated differently, and why dimensions live in the types of operators rather than in a separate judgment. The capability audit that fixed what *equality* and *order* mean on concept values is in Part V, because it was forced by the equation library; its conclusion — equality on data, order on quantities and declared-ordered concepts only — belongs to this Part’s picture of a concept and is restated there.

Nominal concepts

Suppose concepts were represented only by their representation types, so that `Tilt` and `MotorAngle` are both numbers. Then the wire `motorTarget := tiltSensor` is well typed and the design is globally well formed, because nothing in the model records the distinction. This baseline was built, the wire was accepted, and three ways of recording the distinction were then tried against it.

The one that survived is a single nominal type constructor over an internal identity:

$$\text{SemanticId}, \quad \text{Ty} \ni \text{sem } s.$$

Two distinct identities are distinct types regardless of representation, so the invalid wire is rejected by the ordinary rules of the simply typed calculus with no additional judgment. An explicit relationship between concepts, `tiltToMotor : Tilt -> MotorAngle`, is an ordinary declaration of arrow type — a design relationship that is itself signature-first and may remain unresolved — and it appears in the term wherever a crossing occurs. It is not a cast, coercion, or conversion; the kernel has no such mechanism.

Semantic identity is independent of declaration identity, of display name, of dimension, and of hardware, and each independence was tested rather than assumed. A rename preserves identity, whereas a model in which the name *is* the identity makes renaming destructive. Treating a concept as an ordinary declaration admits two category errors: the concept becomes usable as a value, and it can be realized by a number. Keeping identity out of the type as interface metadata, checked by a direct-wire rule, is evaded by η -expansion, since $(\lambda x. x) \text{ tilt}$ has the same flow with no direct wire. A compositional role judgment strong enough to close that gap has the rule shapes of typing over types-with-sem, and the one such formulation examined duplicated nominal typing without benefit. Flow-sensitive or relational semantic analyses were not formalized and are not ruled out; nominal *sem* is the smallest mechanism among the designs that were tried, not the only one possible.

Erasing all semantic identities is sound — a well-typed semantic term is well typed at its representation — and the baseline is exactly what erasure leaves. Generated code is thus ordinary code; the semantic layer has no runtime residue.

Representation binding and the grant

Nominal identity alone leaves semantic values opaque. Without a way to observe a representation and construct a value, no mapping can be realized by a formula; under nominal typing alone a semantic value can only originate from a declaration of semantic type. That is the right state before representation is added. The question is how to add it without destroying what identity just bought.

The obvious form — global $\text{rep}_s : \text{sem } s \rightarrow R$ and $\text{mk}_s : R \rightarrow \text{sem } s$ available to every term — destroys it immediately. $\lambda x. \text{mk}_{\text{Motor}}(\text{rep}_{\text{Tilt}} x)$ is a well-typed `Tilt -> MotorAngle` in the empty environment, with no declaration and no mapping; a motor angle can be manufactured from a literal; and the crossing can hide inside a body whose signature mentions no motor. Observation alone, with no construction, is safe but cannot realize a mapping. The surviving model separates the two:

- a **concept environment** $\Theta : \text{SemanticId} \rightarrow \text{Option Ty}$ binds each concept, write-once, to a representation that mentions no semantic type and contains no function type;
- $\text{rep } e$ is typed at R whenever $e : \text{sem } s$ and $\Theta s = \text{some } R$, everywhere;

- $\text{mk } s \ e$ is typed at $\text{sem } s$ whenever $e : R$, $\Theta \ s = \text{some } R$, and the grant permits s .

$$\frac{\Theta \ s = \text{some } R \quad \Theta; \Delta; G; \Gamma \vdash e : \text{sem } s}{\Theta; \Delta; G; \Gamma \vdash \text{rep } e : R}$$

$$\frac{G \ s \quad \Theta \ s = \text{some } R \quad \Theta; \Delta; G; \Gamma \vdash e : R}{\Theta; \Delta; G; \Gamma \vdash \text{mk } s \ e : \text{sem } s}$$

The grant G is a predicate on concepts. Client code is typed under the empty grant. The realization of a declaration is typed under $\text{Grant.of}(\tau)$, the concepts in result position of its own signature τ . A value of `MotorAngle` can therefore be constructed only inside a declaration that announces `MotorAngle` in its signature, which is exactly where a reader of the design would look for it. A well-typed term constructs s only where granted s ; the hidden crossing above is rejected under the grant of an unrelated declaration and becomes legal, and visible, once `tiltToMotor` is declared; binding an unbound concept is monotone for typing, satisfaction, and global well-formedness; and rebinding a concept to a different representation is an edit that breaks existing realizations.

Two constraints on the representation were not anticipated. Representation types must be free of semantic types, because if `Tilt` may be represented *by* `MotorAngle` then `rep` itself is a hidden mapping under every policy, including observation-only. And they must be data types, a requirement that arrived later from the reactive semantics: a semantic value may be delayed, and a function-typed representation would carry a closure across ticks.

The grant is a known shape — a capability attached to a definition site, or equivalently the private constructor of an abstract type exported only to the module that declares it. What is specific here is where the capability comes from: the signature the designer already wrote, so no annotation is added. It is not presented as a capability calculus. One consequence should be stated plainly. After all bodies are inlined into one executable program, that program is checked under the universal grant, because each construction was authorized at its own declaration. Semantic isolation is a property of the design graph and survives inlining as provenance, not as a type property of the executable.

Physical dimensions

Physical quantities have type $q \ d$ for a dimension d , an exponent vector over a small set of base dimensions. There is no dimension-specific typing rule; the algebra lives entirely in the types of registered primitive operators,

$$\begin{aligned} \text{add}_d &: q \ d \rightarrow q \ d \rightarrow q \ d, \\ \text{mul}_{d_1 d_2} &: q \ d_1 \rightarrow q \ d_2 \rightarrow q \ (d_1 + d_2), \\ \text{div}_{d_1 d_2} &: q \ d_1 \rightarrow q \ d_2 \rightarrow q \ (d_1 - d_2), \end{aligned}$$

and an application is checked by ordinary function application. Erasing every dimension to the zero vector is sound and accepts `length + time`, so the untyped numeric baseline is the erasure of dimensional typing in the same sense that the representation baseline is the erasure of nominal typing. Dimensions might instead have been validation metadata; the argument against is that multiplication and division *produce* dimensions, so any checker recomputes the same inference, but that family was argued against rather than excluded.

Dimension and semantic identity are orthogonal. `Tilt` and `MotorAngle` both bound to $q \ \text{Angle}$ remain distinct types. A mapping realized by the dimensioned formula $\lambda x. \text{mk } \text{bright} \ (\text{rep } x \cdot \text{gain})$ with $\text{gain} : q \ (\emptyset - \text{Angle})$ is typed; a dimension error inside the formula is caught by the same typing; and the formula cannot manufacture a `MotorAngle` despite the shared dimension. The association between a concept and its dimension lives in Θ , not in the identity and not in the type constructor. The earlier draft's two-index $\text{Sem}[n, d]$ becomes $\text{sem } s$ together with $\Theta \ s = \text{some } (q \ d)$.

Units are surface (Part VI in full). A literal `n u` elaborates to a dimensioned literal scaled by the unit's factor; changing the unit changes the value, never the type, and mixed-unit addition works after elaboration. Expressing a quantity in a unit — its *coordinate* — and building a quantity from a coordinate are the same arithmetic against a unit constant: `inUnit(q, u)` is q divided by the unit's scale and has dimension zero, `withUnit(x, u)` is x times the scale and has the unit's dimension, and a conversion between two units is their composition; each is elaborated, none is a kernel construct, and a unit choice never reaches a type (`1 m` and `100 cm` are equal values of one type). The unit laws — round trips, derived conversion, dimension safety — are proved over an abstract scalar domain and

instantiated by a symbolic group in which π is a generator, so that a degree is exactly $\pi/180$ radian; the executable kernel truncates to naturals and production approximates in floating point. A preferred display unit is presentation: it changes the number shown and no judgment of the design. Affine scales such as degrees Celsius are not linear (0°C is 273.15°K), yet their literals and coordinates elaborate exactly by adding an offset. Unit coordinates erase chart identity while preserving affine coordinate change: the conversion between two charts is an affine map, conversions compose and invert — compatible charts are isomorphic coordinate systems — and differences inherit the linear part of that transformation, so a difference of ten degrees Celsius is eighteen degrees Fahrenheit from any base point, while a conversion with a non-zero offset is not an additive homomorphism. The same abstraction covers sensor calibration and encoder offsets. These laws are proved over an abstract field and instantiated exactly at rationals; production floating point is held to a toleranced version of them. Whether a sum of two absolute temperatures should be permitted is a separate, optional validation question that the dimension does not decide and conversion does not need.

Part V — The Data and Equation Language

The kernel of Parts III–IV computes with booleans, counts, quantities, nominal concepts and optional values, and abstracts with lambdas over them. That is enough to state the theorems about identity, refinement, time and outputs, and it is not enough to write the equations designers actually write: a brightness clamped to a range, a mode tested against a finite set of modes, every sensor below a threshold, a pair of readings, a calibration mapped over a collection. Phases 9b and 9c asked for the smallest typed data/function basis that supports those equations without turning BDL into a general functional language, and Phase 11 asked whether the natural surface forms for them can be added with no semantic change at all. This Part records the whole investigation: the candidates, the counterexamples, the kernel additions, the definitional library, the capability audit that reversed one of Phase 9b’s own decisions, and the surface forms.

The question and the hypothesis

The design direction was stated before the experiments: a very small core, parametric polymorphism, ordinary structured data, a rich definitional standard library, and intent-oriented abstractions at the UI. The hypothesis to test was a core of roughly `bool`, `nat/q d`, `sem s`, `A → B`, `A × B`, `opt A`, `list A`, plus rank-1 polymorphism — with the instruction not to assume any proposed feature belongs in the kernel.

The required cases were fixed in advance and every one is executed in `BDL/Experiments/EquationExamples.lean`: (A) clamp a brightness; (B) a mode in a finite set of modes; (C) every sensor below a temperature threshold; (D) any fault above a severity threshold; (E) a pair of temperature and humidity; (F) a calibration mapped over sensor values; (G) two sampled collections zipped; (H) an optional fallback; (I) dimension-preserving `min`; (J) semantic-type-preserving generic functions; (K) range membership; (L) a piecewise rule combining a range, a collection predicate and a boolean; and (M), added when production’s `enum LampMode { Off, Automatic, Manual(Brightness) }` was inspected, an enumeration with a payload.

What entered the kernel, and why each item could not be derived

Four additions were made to the kernel; each was tested against a derivation first.

Products. `Ty.prod a b`, `Value.pair`, and the operators `pair : a → b → a × b`, `fst`, `snd`. A pair is data exactly when both components are (`Ty.prod_data`). The derivation that was tried and rejected is the Church encoding. It fails twice. First, a Church pair is an arrow, and arrows are not data: nothing of function type can be delayed or transported (`arrow_not_delayable`, **formally proved** by inversion of the `delay` and `sync` typing rules, which require `τ.Data`). Paired *state* — a delayed reading with its timestamp — therefore needs a data product. Second, a Church pair used as a first-class value needs rank-2 types: in the toy System F of `PolyAlternatives.lean`, the type of `fst` on Church pairs has rank 2 (`church_fst_rank`), and in the prenex fragment a pair instantiated at one result type serves only one projection (`church_pair_prenex_one_projection`). Products are value composition only. They are never a component interface, an output bundle or a system boundary — Phase 8b’s Counterexample 5 shows what a tuple-returning declaration does to the dependency graph.

The list recursor. `Expr.fold f z l`, with `fold f z [x1, ..., xn] = f x1 (... (f xn z))`, is a *term former*, not a registered operator. The kernel has no recursion, deliberately; a total language needs an eliminator for its inductive data, and this is the one construct in BDL that applies a function value in the course of evaluation. Registered operators still never apply closures. Its evaluation rule unrolls syntactically through the environment:

$$\frac{\Delta; I; t; \rho \vdash f \Downarrow v_f \quad z \Downarrow v_z \quad l \Downarrow []}{\text{fold } f \ z \ l \Downarrow v_z} \quad \frac{f \Downarrow v_f \quad z \Downarrow v_z \quad l \Downarrow x :: xs \quad [xs, v_z, v_f] \vdash \text{fold } \#2 \ \#1 \ \#0 \Downarrow r \quad [r, x, v_f] \vdash \#2 \ \#1 \ \#0 \Downarrow v}{\text{fold } f \ z \ l \Downarrow v}$$

The recursive premise evaluates the syntactic term `fold (var 2) (var 1) (var 0)` in an environment holding the three values; this keeps `Ev` an ordinary inductive relation with no mutual recursion, so every earlier proof by induction on `Ev` extends by one case. Totality is a separate lemma by induction on the list (`fold_total`, `mfold_total`), and the fundamental theorem’s `fold` case uses it. Every collection operation — `map`, `filter`, `any`, `all`, `contains`, `append`, `sum`, `zip`, and, through `toList`, the option eliminators — is a definition over it (below). The alternative of one primitive per operation was rejected because a primitive cannot apply a closure and each would need its own evaluation rule; the alternative of bounded unrolling was rejected because lists (the Phase-9a buffer) are unbounded.

Equality at every data type. $\text{eq } \tau \text{ (h : } \tau.\text{Data})$: structural equality on data values — booleans, numbers, none/some, pairs and lists componentwise, semantic values by concept and representation — with the proof of $\tau.\text{Data}$ carried *in the syntax*. This is the kernel’s only capability evidence: an equality on a function type is unwritable rather than ill-typed, which keeps the `prim` typing rule unconditional. Before Phase 9b, equality existed only at quantities, and production encoded boolean equality as $(a \wedge b) \vee (\neg a \wedge \neg b)$. On first-order values structural equality is equality (`Value.beq_iff`, **formally proved** by a mutual induction over the nested value type).

Two first-order operators. `drop` τ (the dual of `take`) and `toList` $\tau : \text{opt } \tau \rightarrow \text{list } \tau$. The second is the one that matters: without it an optional value has no eliminator that does not need a default — `getD` needs a fallback of the payload type, which a generic `mapOpt` cannot produce — and with it `fold` over a list of length at most one is the option eliminator (`optElimF`, `mapOptF`). `drop` lets `zip` be derived.

Every earlier theorem — determinism, totality in one and many domains, provenance, unfolding, the Phase-8 preservation results, the Phase-9a buffer — was re-established without change of statement. The logical relation gained the product clause; the fundamental theorem gained the `fold` case; the `Wiring` fragment gained a variant with variables for the recursor’s environment-passing sub-derivations (`Ev.noCloV`, `Ev.foldCons_move`).

Polymorphism: rank-1, by families, no kernel type variable

The five models compared, with the verdicts:

model	verdict	evidence
A — monomorphic STLC kernel	kept, unchanged	typing rules unchanged; <code>HasType.unique</code>
B — per-type duplication	what the kernel <i>sees</i> after elaboration	<code>instances_are_monomorphic</code> : three uses of <code>min</code> are three kernel terms
C — rank-1 parametric	adopted, as definitional families	every library entry is $\text{Ty} \rightarrow \text{Expr}$ (or $\text{Dim} \rightarrow \text{Expr}$); <code>*_typed</code> proves its scheme at every instance
D — System F $(\wedge, [\tau], \vee)$	rejected	the toy <code>FTy</code> with rank measures what it would add; its prenex fragment <i>is</i> instantiation of families
E — higher rank	rejected	every candidate use has rank ≥ 2 (<code>applyBoth_rank</code> , <code>existential_rank</code>) and a rank-1 replacement (<code>applyBoth_replacement</code>)

Why C needs no kernel support: a use site always has *closed* argument types. Every declaration’s expected type is frozen and closed since Phase 1, and `infer` is bottom-up, so finding the instance of a scheme is one-way *matching* of the scheme’s pattern against closed types — decidable, returning the unique substitution on the pattern’s variables (`matchTy_sound`, `matchTy_complete`, **formally proved** in `Surface/Poly.lean`). There is no unification of two open types, no let-generalization inside expressions, and no principal-type search: those problems arise when a definition’s type is inferred from its body, and BDL definitions carry their signature (mappings do; a helper `fn` would). Dimension polymorphism (`sum : list (q d) → q d`, `min at q d`) uses the same mechanism with dimension pattern variables; no kind system, no `Type + Dim` universe, because the dimension algebra already lives in the primitive table.

The scheme model: `PTy` is the kernel’s type formers over type variables and dimension patterns (`PDim.const d | PDim.dvar n`); `Subst` maps both kinds of variable; `Scheme = (pattern, caps)` where `caps : List (Nat × Cap)` names, per variable, what it needs; `Scheme.instantiate 0 0 τ` matches then checks the capabilities and is sound (`Scheme.instantiate_sound`). The two failure points have designer-level explanations — *no instance* (“expected a collection”) and *capability failed* (“cannot compare functions”; “Mode values can be compared for equality, but they have no default order”) — and a nominal mismatch is reported by the kernel’s unique typing as “Brightness and Opacity are different concepts”, never as a unification residue, because there is no unification.

Capabilities: the audit that reverted a decision

Phase 9b’s first form generalized both `eq` and `lt` to every data type through a structural order on values: booleans `false < true`, numbers, `none < some`, pairs and lists lexicographically, concepts by representation. It was formally consistent — `Red_prim` held, determinism held — and the closed capability vocabulary collapsed to `{Data}`. Phase 9c audited it and rejected it.

The audit asked, for each type, whether `==` and `<` have a domain-natural meaning for a behavior designer:

expression	meaning	verdict
<code>temperature1 == temperature2, <</code>	magnitude comparison of one quantity	Eq, Ord (q d; q Length < q Time still rejected)
<code>brightness1 < brightness2</code>	the concept is a magnitude the designer declared ordered	Ord <i>by declaration</i> , through the representation
<code>model == mode2</code>	same mode	Eq
<code>model < mode2</code>	none — any order would come from a code, a constructor tag or a SemanticId	rejected
<code>pair1 == pair2</code>	same reading	Eq
<code>pair1 < pair2</code>	lexicographic order is a mathematical convenience with no design meaning	rejected
<code>list1 == list2</code>	same collection in the same order	Eq
<code>list1 < list2</code>	none	rejected
<code>optional1 == optional2</code>	both absent, or both present and equal	Eq
<code>None < Some x</code>	a constructor-tag artifact	rejected

So `Data ≈ Eq` holds (extensionally on this type grammar: `Cap.eq_iff_data`), but `Eq ≈ Ord`. An implementation may need a total order on values for maps, canonical forms, serialization and tests; that is toolchain-internal and is not `<` in BDL. The kernel's structural order (`Value.bl`) was deleted and `lt` restored to `lt (d : Dim)` on quantities — the Phase-4 form — which made Phase 9b *smaller*. The surface vocabulary became `Cap = data | eq | ord (Poly.Cap)`: `data` is what delay/sync need and what the eq proof field checks; `eq` coincides with `data` today and is kept as a separate name because it answers a different question and is what diagnostics say; `ord` is a *surface* capability — a quantity, or a concept the designer declared ordered (`OrdDecl`) and represented by a quantity (`Ty.ordB`) — with no kernel counterpart, because an ordered concept compares as `lt d` on `rep`, a term the kernel already admits. Enumerations follow the same rule: equality is natural, declaration order is never silently behavioral order.

Ordering in the library is evidence-indexed: `Ordered τ` is `q d`, or `sem s d` for a concept declared ordered with representation `q d` (well-formed against Θ : `Ordered.WF`); `ltAt o a b` is `lt d a b` on a quantity and `lt d (rep a) (rep b)` on a concept, so `min`, `max`, `clamp`, `inRange`, `inInterval` return one of their arguments with identity intact. The comparator escape hatch `minBy/maxBy` needs no declaration and loses nothing: `minBy_recovers_min` (**formally proved**) shows the comparator $\lambda a\ b. a < b$ makes `minBy` compute exactly `min`. Negative examples on realistic concepts: `model == mode2`, `pair == pair`, `list == list`, `opt == opt` accepted; `mode < mode`, `pair < pair`, `list < list`, `None < Some`, `bool < bool` rejected at the surface (`lt_rejected`, `min_mode_rejected`) and unwritable in the kernel (`lt_only_on_quantities`).

Production adopted this as ADR-0025 and refined it as ADR-0026: two values of one concept compare as that concept (`==` always, `<` and the ordered equations only while the concept is declared ordered); two values of different concepts never compare, whatever their representations; a concept beside a *plain value of its representation* — `tilt < 10 deg` — is observed and compared as that representation, because the designer wrote a number. The remaining asymmetry is visible and intended: `min(o1, o2)` is refused while `min(o1, 0.5)` is not, and `Explain` shows the observation.

The definitional library

`Surface/Stdlib.lean` is the reference for production's bdl-equations. Each entry is a closed de Bruijn term indexed by types: `idF τ`, `constF τ σ`, `swapF a b`, `minF o`, `maxF o`, `clampF o`, `inRangeF o`, `inIntervalF o`, `minByF τ`, `maxByF τ`, `foldrF τ σ`, `anyF τ`, `allF τ`, `containsF τ h`, `mapF τ σ`, `filterF τ`, `appendF τ`, `sumF d`, `optElimF τ σ`, `mapOptF τ σ`, `getOrElseF τ`, `zipF a b`; `listLit`, `oneOfE` (a finite-set literal is `contains` over a list literal); records as right-nested pairs with positional projections (`recTy`, `recE`, `projE`). `zip` is derived through `fold`, `take 1`, `drop 1` and `reverse`, folding over the reversed first list with the remaining second list and the accumulated pairs in a pair — not pretty, and the point is that it is derivable.

Every entry is a **combinator**: variables, literals, lambdas, applications, registered operators, the recursor and, since 9c, `rep` — no reference, no state, no transport, no `mk`. For combinators, proved once: typing is independent of the design and the grant and reads Θ only through write-once representation bindings (`HasType.comb_irrelevant`); the value is the same in every design at every tick under every input (`lib_eval_context_free`, from the kernel lemma `Ev.pure`: a pure term in a pure environment is context-free); the term is clocked in every domain (`lib_clocked`); nothing

is constructed (Comb.noConstruct); and the four together as the inlining statement `lib_expansion`. This is what lets production inline an equation at each use without creating a declaration — a library entry as a declaration would be monomorphic and would enter the dependency graph (FVD-0094).

The evaluation specifications connect each entry to the mathematical function: `any_spec` (`List.any`), `all_spec`, `contains_spec` (`List.any (beq x)`), `map_spec`, `filter_spec`, `min_spec`, `max_spec`, `clamp_spec`, `inRange_spec` (by the compared magnitudes `Ordered.key`), each **formally proved** through a general `fold_spec`: the recursor computes `List.foldr g` whenever the step closure implements `g` on the reachable accumulators. From these, finite quantification is a fold: `forall_in_list` ($\forall x \in xs, P \times \text{iff all } xs \text{ } P \text{ evaluates to true}$) and `exists_in_list`; a finite-set literal means membership (`oneOf_mem`, via `beq_iff`), and duplicates do not change the answer (`oneOf_dup_irrelevant`), so no uniqueness convention and no `Set` type. An interval is a pair with a convention and membership is a function of the pair. A predicate is an ordinary $\alpha \rightarrow \text{bool}$.

Nominality survives all of it: `generic_preserves_identity` — *any* family typed at $\alpha \rightarrow \alpha \rightarrow \alpha$, instantiated at concept `s`, rejects an argument of concept `s'  $\neq$  s`, the representations never consulted; `generic_preserves_dimension` for `q d` vs `q d'`; `pair_projections_keep_concepts`; `map_keeps_concepts`; `eq_across_concepts_rejected`. Executed: `min` at `Brightness` typed and `Opacity` rejected though both are `q 0` underneath (`exJ`), and the same for lengths and times (`exI`).

Enumerations, records, sets: what was encoded and what was deferred

Production's textual syntax declares enumerations with payloads and matches on them. Phase 9b encoded `LampMode` as a tag paired with an optional payload, with `match` as conditionals on the tag (`exM`, executed), and deferred a kernel `sum` type: nothing tested needs more than the encoding gives, and the cost of a kernel `sum` would be one more eliminator term former, exactly like `fold`. Production keeps user enums open (ISS-0005). Records are positional nested pairs; labels resolve to positions at elaboration; row polymorphism was rejected because no case needs a function generic over record shapes. Sets are list literals with `contains`; set algebra is list definitions when needed; nothing tested observes a canonical form.

The expressiveness ceiling

Total, first-order-data computation over booleans, quantities, nominal concepts, options, lists and pairs, with higher-order functions and one list recursor; generic definitions instantiated at closed types; no general recursion, no type abstraction in terms, no sums yet (encoded), no unbounded quantification. This is a design recommendation backed by the executed cases and the proved library, minimal among the tested candidates; it is not a minimality theorem. The seven questions the phase was asked have the answers: rank-1 by families constrained by `{Data, Eq, Ord}` is the weakest useful polymorphism; products belong in the data core; no `Set` type; finite $\forall/\exists$ reduce to folds, proved; no existentials (Phase 8a components hide by fresh instantiation); no user typeclasses; and the ceiling above.

The natural expression surface (Phase 11)

Designers should be able to write

```
all reading in readings:
  reading in 10 deg .. 45 deg
```

while the semantics stays exactly the verified Phase-9 machinery. Phase 11 tested that binder syntax and closed ranges are *conservative* surface abstractions and proved it for a six-form fragment (`Surface/Natural.lean`). The surface model `NatExpr` has an embedded closed core term, a named binder `local`, application, the four binders `all/any/map/filter` `x in xs: body` with the element type chosen by local inference, a range `x in lo .. hi` with its `Ordered` evidence, and a coalesce `x ?? d`. Elaboration `desugar` is one-way, under a binder stack:

```
all x in xs: p      ↪ app2 (allF τ) (lam τ p') xs'      (any, map, filter alike)
x in lo .. hi       ↪ app3 (inRangeF o) x' lo' hi'
x ?? d              ↪ app2 (getOrElseF τ) x' d'
```

A binder `local` is the kernel's lambda parameter: `desugar` resolves a name to its de Bruijn index — nearest binder first — and adds nothing. **Formally proved**: nearest-binder scoping, shadowing, an unbound name is an error rather than a free variable, ambient references are untouched (`desugar_local_nearest`, `desugar_shadow`, `desugar_unbound`, `desugar_core`); alpha-equivalence — renaming a binder and its occurrences to a name fresh in the expression and the stack leaves the elaboration unchanged (`desugar_rename`, `alpha`), which is what a Formula Composer generating fresh locals

needs; no `mk` is introduced (`desugar_constructs`); typing of each form with the local *forced* to the collection's element type by inversion (`binder_local_type`) and range bounds forced to the value's nominal type (`range_bounds_forced`); evaluation exactly the library's (`binder_all_eval`, ... from `all_spec`, `any_spec`, `map_spec`, `filter_spec`; `natural_forall`, `natural_exists`; `range_eval = lo ≤ x ∧ x ≤ hi`); and clocks of the elaborated term are the operands' (`binder_clock`, `range_clock`).

Executed: the call form and the natural form elaborate to literally the same term; `angle in 10 deg .. 45 deg` after unit elaboration, with the boundary values checked; nested `all row in rows: any v in row: v > 0`; shadowing where the inner `x` ranges over `ys`; alpha renaming under shadowing; `Tilt` ranges with `Tilt` bounds accepted, `q Angle` bounds rejected, and accepted through `rep`; a `MotorAngle` predicate rejected on a `Tilt` local; the required negatives (`all x in 5: true, filter x in xs: 5, angle in 2 s .. 3 s`, an unordered concept in a range, a binder variable outside its body). Verdicts: binders, `BinderKind`, the range node and `??` are surface desugaring; an interval type, general comprehension (generators, `yield`, `where`) and a general quantifier are removed — nested binders cover every required case, and the forms are finite list equations.

Production implements the forms (P11; ADR-0028's second amendment; **production implemented and tested**): `bd1-elab::formula::binder` lowers the three forms once to the equation library, the parser keeps them as their own nodes and the formatter keeps the spelling authored, binder locals are lexically scoped to the body, and the tests `natural_forms_lower_to_the_same_core_as_the_call_forms`, `binder_locals_are_elements_scoped_to_the_body` and `natural_form_mistakes_are_named_in_their_own_words` discharge the Phase-11 claims by differential elaboration; the Composer draws binders and ranges with local chips and `??` as a comparison node (`ComposeAction.binder`, `range`). The production recommendations recorded with the phase, which the implementation followed: `all`, `any`, `map`, `filter` and `in` as contextual keywords only in the binder head, so existing names keep parsing; the local visible in the body only; the natural form kept as authored and never reconstructed from Core; `..` binding tighter than `in` and looser than arithmetic, legal only as the right operand of `in`; local inference exactly `binder_local_type`; the Composer representing a binder as a node with a fresh local backed by `desugar_rename`; diagnostics in concept language (“5 is not a collection”, “Mode values have no order, so `in lo .. hi` does not apply”).

Part VI — Units, Coordinates and Charts

Dimensions entered the kernel in Phase 3 (Part IV): a quantity $q\ d$ carries an exponent vector, the dimension algebra lives in the types of the primitive operators ($\text{mul} : q\ d_1 \rightarrow q\ d_2 \rightarrow q\ (d_1 + d_2)$), and dimensional typing rejects $\text{length} + \text{time}$ where the erased numeric baseline accepts it. Units were surface from the start (FVD-0032): a literal $n\ u$ elaborates to a scaled dimensioned literal; changing the unit changes the value, never the type. Phase 10 asked what more the same quantity in different units needs — extracting a coordinate, constructing from a coordinate, letting an editor infer dimensions for incomplete expressions, keeping units out of type identity — and Phase 10b revisited the one conclusion of Phase 10 that turned out to be too strong. This Part records both, the production realization, and the design decision about who owns a unit in the authoring surface.

Three notions kept apart

notion	formal object	layer
physical quantity	a kernel value of type $q\ d$: a canonical magnitude and a dimension	kernel
unit coordinate	$\text{inUnit}\ q\ u = q / \text{scale}(u), a$ dimensionless number	surface elaboration
display / authoring unit	$\text{Presentation.preferred} : \text{SemanticId} \rightarrow \text{Option Unit}$	presentation

Collapsing any two of these is the mistake the phase was designed to avoid: a quantity with a unit inside it splits one quantity into many types; a coordinate that remembers its unit is a runtime tag nobody needs; a display preference that enters elaboration changes semantics when a designer changes what they like to see.

Linear units: no kernel construct

Six models were compared. Literal-only elaboration (A) is insufficient alone — a coordinate is needed for display, for normalization ($\text{tilt in degrees} \div 90$), and for export. $\text{inUnit}(q, u) : \text{Scalar}$ by elaboration (B) and $\text{withUnit}(x, u) : Q[d]$ by elaboration (C) were adopted: inUnit is $\text{div}\ q\ (\text{lit}\ d\ \text{scale}(u))$, typed $q\ (d - d) = q\ 0$ (inUnitE_typed) and rejecting a quantity of another dimension by inversion (inUnitE_safe); withUnit is $\text{mul}\ x\ (\text{lit}\ d\ \text{scale}(u))$, typed $q\ (0 + d)$ (withUnitE_typed) and never a concept ($\text{withUnitE_is_quantity}$); a literal is withUnit . First-class runtime units (D) were rejected — no tested case delays, syncs, stores or compares a unit, and a UI dropdown is not a runtime value. Units in quantity types (E) were rejected — $1\ m$ and $100\ cm$ would differ in type while being one quantity (exA: same type, equal values). A kernel conversion primitive (F) was rejected — $\text{convert}\ x\ u\ v = \text{inUnit}\ (\text{withUnit}\ x\ u)\ v = x \cdot \text{scale}(u) / \text{scale}(v)$ (convert_eq , ev_convertE), transitive (convert_trans), identity on the same unit (convert_self), mismatch rejected never coerced (inUnit_mismatch). Unit operations construct nothing ($\text{unitOps_no_construction}$), and on a concept they go through rep — so $\text{inUnit}(\text{TiltValue}, \text{deg})$ obeys the representation discipline, and rebuilding a concept from a coordinate requires mk under the concept’s grant. Tilt and MotorAngle are both readable in degrees and stay distinct under every unit round trip (nominal_distinct); Brightness and Opacity likewise. Unit compatibility is not identity.

The numeric domain, in the open

The formal development says exactly where exactness holds. Unit laws are proved over an abstract scalar domain $\text{Scalars}\ K$ — a commutative monoid with a division that cancels multiplication by a non-zero scale — and instantiated by Sym , the free abelian group on the generators $2, 3, 5, 127, \pi$. Every registered scale is an element, exactly: $\text{deg} = \pi / 180\ \text{rad} = \pi \cdot 2^{-2} \cdot 3^{-2} \cdot 5^{-1}$, $\text{inch} = 127 / 5000\ m$. Round trips (inUnit_withUnit , withUnit_inUnit), conversion and mismatch hold with π symbolic: $90\ \text{deg}$ in radians is $\pi/2$ (exD), not $1.5707\dots$. The executable kernel’s magnitudes are naturals, so its registry uses integer scales relative to canonical sub-units — $0.1\ mm, ms, \text{arc-second}, g, K/180$ — under which $mm, cm, m, km, inch, deg, turn, ms, s, min, g, kg$ and K are exact; round trip 1 is exact for every positive scale ($\text{inUnit_withUnit_nat}$) and round trip 2 exact when the scale divides the magnitude ($\text{withUnit_inUnit_nat}$), the strongest law integer division admits; radians are *not* in that registry because their scale is not an integer in any degree-compatible basis. Production uses IEEE doubles with radians canonical (ADR-0011; bdl-elab::units): neither round trip holds exactly there, and the document says so rather than hiding it (Part XIII).

A unit itself is `(id, dim, scale)` in a registry; its symbol is not part of it — two units with equal identity are the same unit whatever they are spelled — and `unitsFor reg d` is sound and complete relative to the registry (`unitsFor_sound`, `unitsFor_complete`).

Presentation, and the realistic formula

Presentation is a separate object; a design with a presentation is a pair, and every kernel judgment of the pair — typing, evaluation, dependencies, clocks — is literally the judgment of the design (`presentation_irrelevant_*`, by construction, `rfl`); what changes is the displayed number (`presentation_changes_display`). Ordering compares canonical magnitudes through `ltAt`, which mentions no unit; comparing *displayed* coordinates would not be safe, since integer display can identify distinct magnitudes (`display_may_identify_distinct`).

The recurring formula `Tilt → Brightness: tilt / (90 deg)` and `inUnit(tilt, deg) / 90` are both dimensionless and evaluate equal for every tilt (`normalizations_agree`, `exF`). The representation-aware form is the second — it names the unit the designer thinks in and keeps the divisor a plain number — and neither changes the concept: both consume `rep tilt`, and the result is wrapped as `Brightness` by the block’s own signature (ADR-0013).

Design guidance, not a theorem, for persistence: the *source literal unit* `(90 deg)` is semantic source — its scale enters elaboration — and lives in the formula text; the *concept preferred display unit* (`Tilt` shown as `deg`) is authoring metadata beside the display name and never enters elaboration; a *simulation-input display unit* is session state.

Unit ownership in authoring

The production Formula Composer made this a rule (ADR-0028): in `tilt / 90 deg`, `tilt` is a semantic reference and `90 deg` is a designer-authored quantity literal. **A literal owns an editable unit; a semantic reference does not; a display boundary may carry a presentation unit.** The reason is semantic ownership. A literal’s unit is part of what the designer *wrote* — its scale enters elaboration, so switching it with the quantity kept (`180 deg → 3.141592653589793 rad`) is a change of spelling that preserves meaning, and editing the coordinate is a change of meaning; both are the designer’s to make on the literal. A reference’s kind is its declaration’s: `tilt` is a `Tilt`, represented as an angle, and no unit is part of that; offering a unit pop-up on a reference would invite a rewrite of the representation binding of a concept from inside one formula, which is exactly the representation escape hatch the grant discipline forbids. The display of a reference’s *value* — in simulation, in the inspector — may use the concept’s preferred unit, which is presentation. This decision is traceable to `unitOps_no_construction`, `withUnitE_is_quantity` and `presentation_irrelevant_*`, not to taste: the Composer draws a literal as two fields (coordinate and unit pop-up) and a reference as a chip with its concept’s socket glyph, because those are the two semantic situations.

Affine units: the failure of scale-only units, and the revision

Temperature scales are affine: `canonical = scale · x + offset`. Phase 10 showed, executably, that the linear model cannot represent °C or °F — `0 °C = 273.15 K`, which no scale produces from 0 (`celsius_not_linear`) — and that an affine *literal* and an affine *coordinate* nevertheless elaborate exactly with no kernel change: `n °C ↦ lit Temp (n · s + off)`, `inUnit °C q ↦ (q - off) / s`, in the `K/180` basis where °C and °F have integer scales and offsets (`affLitE_typed`, `affInUnitE_typed`, `affine_roundtrip_ev`; `0 °C = 32 °F`, `100 °C = 212 °F` in `scales_agree`). What breaks is arithmetic on *absolute* values: `20 °C - 10 °C` is a difference of 10 K (`delta_is_linear`), while `10 °C + 10 °C` is well typed at `q Temp` and reads 293 °C (`sum_of_points_is_not_a_point`, `sum_well_typed`). The kernel’s dimension cannot separate a point on the scale from a difference.

Phase 10’s conclusion, as first recorded: the missing information is a *sort* — point or difference — on top of the dimension (`AffSort`, with `affAdd/affSub` as the affine-space rules: `point + point` refused, `point - point = delta`), and a Formula Composer cannot offer only difference units for a delta slot from the dimension alone (`delta_candidates_need_sort`). Affine conversion was called “safe now”; affine *arithmetic safety* was said to need the sort and was deferred, with the sort treated as potentially necessary for future production.

Phase 10b tested the smaller hypothesis that for *conversion* chart-specific information is intentionally erased at coordinatization and only the affine transformation structure between coordinate systems must be preserved. It holds, and the conclusion was revised. This revision is recorded as research evidence, not smoothed over.

Charts: coordinate erasure and conversion functoriality (Phase 10b)

Over one physical dimension, a chart is `(scale, offset)`, valid when `scale ≠ 0`, with

$$\text{reconstruct}_u(x) = s_u x + o_u, \quad \text{coord}_u(q) = (q - o_u)/s_u.$$

The scalar domain is an abstract field (Field K: commutative, with inverses of non-zero elements, negatives and fractions) instantiated by `Q`, exact rationals built in the development as a quotient of `Int` fractions with every law proved from `Int`’s ring identities — because core Lean’s `Rat` proves its algebra with `Classical.choice`, which this development has kept out of every proof. Concrete equalities are decided by cross-multiplication.

Formally proved, over any field:

law	theorem
chart left inverse <code>coord u (reconstruct u x) = x</code>	<code>chart_left_inverse</code>
chart right inverse <code>reconstruct u (coord u q) = q</code>	<code>chart_right_inverse</code>
conversion is affine: <code>C(u,v)(x) = (s_u/s_v)·x + (o_u - o_v)/s_v</code>	<code>convert_is_affine</code> , <code>convertMap</code>
identity <code>C(u,u) = id</code>	<code>convert_identity</code>
composition <code>C(v,w) ∘ C(u,v) = C(u,w)</code> — from the chart laws alone, no algebra	<code>convert_compose</code> , <code>convertMap_compose</code>
inverse <code>C(v,u) ∘ C(u,v) = id</code> , both ways	<code>convert_inverse</code>
difference map <code>f(y) - f(x) = a·(y - x)</code>	<code>difference_map</code>
offset cancels <code>f(x + δ) - f(x) = a·δ</code>	<code>difference_offset_cancels</code> , <code>difference_converts_linearly</code>
linear-part functoriality <code>L(u,u) = 1</code> , <code>L(v,w)·L(u,v) = L(u,w)</code>	<code>linear_part_identity</code> , <code>linear_part_compose</code>
not additive when the offset is non-zero	<code>not_additive_of_offset</code>
erasure keeps conversion structure <code>coord v q = C(u,v)(coord u q)</code>	<code>unit_erasure_preserves_conversion_structure</code>
display switch preserves the quantity <code>reconstruct v (C(u,v)(coord u q)) = q</code>	<code>display_switch_preserves_quantity</code>
coordinate edit changes the quantity	<code>coordinate_edit_changes_quantity</code>

Compatible charts form a groupoid of affine isomorphisms (theorem-level; no category-theory framework was built). Unit coordinates erase chart identity while preserving affine coordinate change; differences inherit the linear part of that transformation. A conversion with a non-zero offset is *not* an additive homomorphism, and this document never calls Celsius-to-Fahrenheit one.

Celsius and Fahrenheit, exactly. Kelvin canonical, `celsius = (1, 27315/100)`, `fahrenheit = (5/9, 45967/180)`: `C(°C, °F)(x) = 9/5·x + 32` and `C(°F, °C)(x) = 5/9·(x - 32)` for every `x` (`CtoF_closed`, `FtoC_closed`); `0 °C = 32 °F`, `100 °C = 212 °F`, `-40 °C = -40 °F`, the `°C→°F→°C` round trip, `°C→K→°F` equal to `°C→°F`, `Δ10 °C = Δ18 °F` from two base points, `Δ°C = 5/9·Δ°F`, and the non-additivity counterexample `f(0+0) = 32 ≠ 64` (executed, `Experiments/AffineExamples.lean`).

Not a Celsius special case. A 10-bit ADC over 5 V, millivolts, and a calibrated reading with a 0.5 V zero instantiate the same theorems — composition through millivolts, inverse back to raw, a difference independent of base (`exH`); so does an encoder with home offset, `angle = 45/512·count + 30°` (`exI`). Unit conversion and sensor calibration are one affine-map abstraction.

The scalar coordinate is chartless. `32` is a Fahrenheit coordinate of `0 °C` and a Celsius coordinate of `32 °C` (`coordinate_needs_chart`); the destination chart supplied to `reconstruct/convert` interprets it; no runtime tag is needed, and no theorem requires a unit to be data, delayed, synced, stored or compared — Phase 10’s rejection of runtime units re-audited and confirmed.

AffSort, revised. Conversion never takes a sort and the sort checker never takes a chart (`sort_orthogonal_to_conversion`, `conversion_orthogonal_to_sort`, by construction). Two claims coexist: (A) unit conversion is complete and correct without point/delta; (B) a domain-specific checker may still use point/delta to reject `point + point`, as *optional physical-arithmetic validation*, orthogonal to A. The design decision FVD-0106 was rewritten from “affine conversion works, but point/delta is missing information” to “affine conversion is complete as coordinate-change semantics; point/delta is additional validation information for restricting physical arithmetic.” `Ty.q d` is unchanged; no conversion theorem needed a sort in the type.

The production chart model

Production (`crates/bdl-elab/src/units.rs`, ADR-0028 and its amendment) realizes the registry as `UnitDef { id, symbol, dim, chart }` with

```

pub enum Chart {
  Linear { factor: f64 }, // canonical = coordinate × factor
  Affine { scale: f64, offset: f64 }, // canonical = coordinate × scale + offset, scale ≠ 0
}

```

and coord, reconstruct, linear_part on the chart; `convert(x, from, to) = coord_to(reconstruct_from(x))` is the one conversion operation and hides the chart's shape. The Phase-10b laws are property-tested on f64 — identity, composition, inverse, display switch preserving the quantity, the difference map linear — within a few ulps, including the Celsius/Fahrenheit charts; the formal laws are exact. The affine charts exist as tested infrastructure and are **not** in the registry offered to formulas or the Composer's pickers (ISS-0004): conversion and display of an absolute temperature are safe, arithmetic on absolute temperatures is a separate validation concern, and the product does not offer 20 °C in a formula until the point/difference validation exists or a decision is taken not to need it. The symbol is presentation; identity is the id; inch is spelled out because in is the membership keyword.

The production contract for floating point, as recommended by the formal phase: for registered charts u , v , w and sampled x , $|C(u, u)(x) - x| \leq \epsilon |x|$, $|C(v, w)(C(u, v)(x)) - C(u, w)(x)| \leq \epsilon |x|$, $|C(v, u)(C(u, v)(x)) - x| \leq \epsilon |x|$, $|C(u, v)(x + \delta) - C(u, v)(x) - L(u, v)\delta| \leq \epsilon |\delta|$, and $|reconstruct_v(C(u, v)(coord_u(q))) - q| \leq \epsilon |q|$, with ϵ a few ulps scaled by the largest coefficient, plus exact-rational oracles — never a claim of exact f64 identity or composition.

The Formula Composer's formal basis

A structured composer needs, from the formal side, exactly three things: typed holes, expected-dimension propagation, and candidate-unit inference. `Surface/Composer.lean` gives them and no more.

`PEExpr` is holes, known operands by dimension (a literal, a reference or a known equation result), and `add/sub/mul/div`; it is never executed. `check` is bottom-up on the known parts. `solve` pushes an expected dimension down by the local rules of each operator in the `Dim` group — `add/sub`: both sides get the result; `mul`: the other side gets $r - d$; `div`: numerator $r + d$, denominator $d - r$ — deterministically and in one pass; a product of two holes is reported *unsolved*, never searched. **Formally proved** for the whole fragment: `solve_sound` (filling the holes as solved makes the expression check at the expected dimension) and `solve_complete` (any filling that checks agrees with the solution — a one-hole equation in an abelian group has one solution); `candidates_sound`, `candidates_complete` and `slot_sound` (the units offered for a hole are exactly the registered units of its solved dimension); `refCandidates_sound` for declaration references. Executed: `? / (1 s) : Speed` $\Rightarrow$ `? : Length` with candidates mm, cm, m, km, inch (exG); `Force × ? : Torque` $\Rightarrow$ `? : Length` (exH); the explanation `Speed = [Length] / [Time]` as slots (`speed_slot`). No unification, no symbolic algebra beyond the group laws.

The Composer does not need to make a unit part of an expression's type: an affine quantity literal needs the physical dimension, the chosen chart and the scalar coordinate, and changing the displayed chart is `coord_u(q) → coord_v(q)` with q preserved. What it *would* need beyond the dimension to offer only difference units for a delta slot is the point/difference sort — which is why that sort is recorded as optional validation information the editor could carry, not as anything in the kernel.

Part VII — Reactive Semantics

A Minimal Reactive Semantics

One primitive

The reactive kernel adds one expression form and no types:

$\text{delay } \text{init } e.$

Its meaning is given by a tick-indexed big-step evaluation relation $\text{Ev } \Delta \ I \ t \ \rho \ e \ v$: the value of e at tick t under a local environment ρ , where $I : \text{DeclId} \rightarrow \mathbb{N} \rightarrow \text{Value}$ supplies the inputs. Every declaration is a stream by interpretation. An unresolved declaration is an input and takes its value from I ; a realized declaration is evaluated from its body at the current tick; $\text{delay } \text{init } e$ evaluates e at tick t when read at tick $t + 1$, and init at tick 0:

$$\frac{\Delta^{\text{real}}(d) = \text{none}}{\text{Ev } \Delta \ I \ t \ \rho \ (\text{declRef } d) \ (I \ d \ t)}$$

$$\frac{\text{Ev } \Delta \ I \ 0 \ \rho \ \text{init } v}{\text{Ev } \Delta \ I \ 0 \ \rho \ (\text{delay } \text{init } e) \ v} \quad \frac{\text{Ev } \Delta \ I \ t \ \rho \ e \ v}{\text{Ev } \Delta \ I \ (t + 1) \ \rho \ (\text{delay } \text{init } e) \ v}.$$

There is no signal type in Ty. Under this semantics a signal type would be inhabited by exactly the terms of the underlying type and would reject nothing. The information a reactive type would carry in a multi-domain setting is *which clock*, and the next section places that information in a judgment rather than a type. There is likewise no event type. Within one domain an input delivers at most one value per tick by construction, so an occurrence is a stream of optional type, and the streams of type $\text{opt } \tau$ are exactly the streams of multiplicity at most one. What separates an occurrence from an optional value — two occurrences falling in one observation interval — can only be seen when a source ticks faster than its observer. That is a cross-domain question and is treated at the end of this part.

Two restrictions on delay were forced by the totality proof rather than chosen. The delayed type must be a **data** type, one with no function type inside, because a delayed closure would have to persist across ticks and the logical relation for closures is tick-indexed and cannot be transported. And delay may occur only at **top level**, under no binder, because a delay under a lambda would re-evaluate its operand at the previous tick in an environment created at the current tick. Temporal state therefore belongs to declarations, and mappings are pointwise. This is the arrangement of *pre* in Lustre, where it lives in nodes rather than in functions [5], and its practical consequence is that a reusable stateful component is instantiated into fresh declarations rather than abstracted over.

Causality

The dependency graph on declarations comes in three variants. Structural dependency, *DependsOn*, records every reference in a body. Instantaneous dependency, *InstDependsOn*, excludes references under the delayed operand of a delay (the initial value is read at tick 0 and counts as instantaneous). A design is **causal** when its instantaneous graph is acyclic, witnessed by a bounded rank:

$$\text{Causal } \Delta := \exists \text{rank}, R. (\forall d. \text{rank } d < R) \wedge \forall a \ b. \text{InstDependsOn } \Delta \ a \ b \rightarrow \text{rank } b < \text{rank } a.$$

On the delay-free fragment this coincides with structural acyclicity, so the earlier acyclicity condition is the timeless special case rather than a replaced requirement. A structural cycle every one of whose paths passes through a delayed operand — $A := \text{delay } 0 \ B$; $B := A$, or a self-delayed accumulator — is causal and runs at every tick. A cycle that is partly delayed is not causal. A strict cycle, one through neither a delay nor a lambda, has no value at any tick.

Evaluation is a partial function: one tick, one environment, one term, at most one value, with no hidden evaluation order, and this holds unconditionally. It is total on causal designs: if Δ is causal and globally well formed and the inputs are well typed, every declaration has a value at every tick, related to its type by a logical relation, by an

induction on tick, rank, and derivation. An executable interpreter is proved sound for the relation, and every trace reported in Parts III–X was obtained by running it.

One gap should be recorded. A cycle guarded by a lambda, $A := \lambda x. A \ x$, is rejected by Causal, yet `declRef A` does evaluate — to a closure; only applying it diverges. Causal is conservative for lambda-guarded cycles, and the negative theorem covers strict cycles only.

Derived operators

Every temporal operator of the surface language reduces to delay and the primitive operators. There is no independent kernel definition of `count` for the reduction to be proved equivalent to; what was done instead was to elaborate each operator, check its typing and causality, and run it on a concrete input trace. In each row the declaration refers to itself. Each is a self-delayed cycle, the class that structural acyclicity forbade and causality licenses.

Surface	Declaration shape
<code>previous x</code>	<code>delay init x</code>
<code>previous x without init</code>	<code>delay none (some x); the absence is pushed to consumers</code>
<code>hold init e</code>	<code>getD e (delay init self)</code>
<code>count e</code>	<code>ite (isSome e) (1 + delay 0 self) (delay 0 self)</code>
<code>since e</code>	<code>ite (isSome e) 0 (1 + delay 0 self)</code>
<code>once e</code>	<code>delay false self v isSome e</code>
<code>every n</code>	<code>modulo-<i>n</i> counter over delay</code>
<code>rise b</code>	<code>b $\wedge$ $\neg$ delay false b, as an optional Boolean</code>

State has no identity of its own. A cell is a delay in a declaration body, and nothing refers to it because consumers refer to the declaration. There is consequently no notion of two writers to one cell in this kernel.

State preserves semantic identity and dimension. The typing rule is $\text{delay} : \tau \rightarrow \tau \rightarrow \tau$ for data τ , so a delayed tilt is a tilt and a backward difference over a time step has dimension `Length – Time` with no derivative primitive. Provenance holds in the reactive setting too: if no signature announces a concept and no input carries it, no value at any tick carries it. Temporal state carries tags; it never creates them.

Initialization is semantic, not validation. Every delay carries an explicit initial value. Two toy relations without one show why: the first tick is either undefined or nondeterministic. Adding or removing a delay, or changing an initial value, is an edit.

Clock Domains

Identity, not rate

“Contact and orientation move with the interaction; temperature moves with the environment.” A designer can say this before any rate is known, and it is a statement about which quantities are updated together, not about how often. BDL records it as a nominal **clock domain**, `ClockId`, and treats rate as validation data that never enters the kernel.

The time model is one global base tick and a schedule $S : \text{ClockId} \rightarrow \mathbb{N} \rightarrow \text{Bool}$ saying at which global ticks each domain activates. A period n induces the schedule $t \bmod n = 0$; the schedule lives outside the design. Domain-local time is not a separate counter but the sequence of a domain’s activations.

Each non-agnostic declaration is assigned a domain by a **clock environment** $K : \text{DeclId} \rightarrow \text{Option ClockId}$; a declaration with no domain is a pure mapping that may serve any domain. The clock is interface data in every sense that matters — clients’ validity depends on it, it is frozen under refinement, and changing it is an edit — and it is stored as a projection beside the interface, exactly as a concept’s representation is stored in Θ rather than in the type. Whether to fold it into the interface record is churn rather than semantics.

Rate and identity are distinct. A clone of a domain with the identical schedule is a different domain, and a direct wire between them is rejected; a domain at the same rate but shifted in phase reads different values through a transport. Rate changes are validation-only. They change the induced schedule and hence the observed values, but no client’s well-formedness. This is where BDL departs from synchronous languages that recover clocks by inference [6]. The

domain is authored, because the information needed to infer it does not arrive until realization binding, which in this workflow is the point at which the designer is least able to make the decision.

One transport primitive

Cross-domain reading is the second and last temporal form:

$$\text{sync } c \text{ init } e,$$

the value of e , evaluated in domain c , at the last activation of c strictly before the current tick, and init if there has been none. The multi-domain evaluation relation $\text{MEv } S \Delta I c t \rho e v$ indexes evaluation by the domain in which it takes place, and its two transport rules are

$$\frac{\text{prevAct } S c' t = \text{none} \quad \text{MEv } S \Delta I c t \rho \text{init } v}{\text{MEv } S \Delta I c t \rho (\text{sync } c' \text{init } e) v}$$

$$\frac{\text{prevAct } S c' t = \text{some } t' \quad \text{MEv } S \Delta I c' t' \rho e v}{\text{MEv } S \Delta I c t \rho (\text{sync } c' \text{init } e) v}.$$

delay is sync at the expression's own domain: $\text{delay init } e \equiv \text{sync } c \text{init } e$ in domain c , as an equivalence of the two relations. The kernel therefore has one temporal primitive — read a domain at its previous activation — and the single-domain semantics of the previous section is its diagonal. Under the always-active schedule, MEv coincides with Ev in every domain, so the earlier results are the one-domain special case rather than a replaced machine. A delay in a slow domain reads three global ticks back where a delay in a fast one reads one, with the same syntax.

A **domain judgment** $\text{Clocked } K c e$ rejects every other cross-domain reference: a reference stays in its domain or is agnostic, a delay needs a domain, and $\text{sync } c'$ switches the domain of its operand. Typing is unchanged and is blind to domains; the direct wire between two domains at the same value type is well typed and rejected only by the domain judgment. Placing the domain in the type instead was tried and set aside. Every pure mapping would need clock polymorphism, and nothing the type rejects is missed by the judgment.

Strictly before

A transport sees only source activations strictly before the destination tick. That is a choice with an observable alternative, and the alternative was built. A transport that lets simultaneously active domains see each other's current values makes the scheduler order observable: two priorities between the domains give two outputs. The strictly-before rule has no such parameter, and multi-domain evaluation is deterministic with no order between simultaneously active domains appearing in the semantics. It also makes cross-domain causality free. A transport's operand is never instantaneous, so Causal Δ is unchanged and no cross-domain cycle can be instantaneous. Every crossing costs one destination-visible step; "synchronous sub-domains evaluated in one instant" are, in this model, the same domain.

Every sync carries an explicit initial value, used at a destination activation with no earlier source activation. Totality extends to the multi-domain case — a causal, globally well formed design with well-typed inputs has a value in every domain at every tick — so the first activation is deterministic with the stated initial values. Semantic identity and dimension pass through transport untouched, by the typing rule; a crossing from Tilt@fast to Tilt@slow authorizes neither $\text{Tilt} \rightarrow \text{MotorAngle}$ nor $q \text{ Length} \rightarrow q \text{ Time}$.

Cross-domain occurrences and lossless buffering (Phase 9a)

Phase 5 left one problem open, and it is worth recording in full because its resolution shaped the data language.

The problem. Within one domain an occurrence is a stream of optional type: an input delivers at most one value per tick, so $\text{opt } \tau$ streams are exactly the streams of multiplicity at most one. Across domains this fails. sync is a zero-order hold: a slow consumer of a fast event source sees the last value only. Phase 5's Counterexample C ($\text{opt_loses_multiplicity_under_sync}$, **executable**) has two fast events at ticks 1 and 2 and one event at tick 2 indistinguishable at the slow activation at tick 3, and a single event at tick 1 followed by a quiet fast tick *dropped* outright. The counterexample is against sync as an *event transport*, not against optional types.

The window model, on tick sets. What the destination should see is the source’s occurrences at the source activations since the destination’s own previous activation — the *window* $\text{windowTicks } S \text{ src dst } t = \text{srcTicks } S \text{ src } (\text{prevAct } S \text{ dst } t) \text{ } t$. Phase 5 proved (*buffer_from_log_and_cursor*) that this window equals the source’s accumulated log read at the current tick minus the log length read at the previous destination activation: two single-instant reads, a sync of a source-side accumulator and a delay of a cursor. Policies are functions of the window; *latest*, *count*, and *count with latest* each identify distinct windows (*policies_lose_information*), and only the list is injective. So multiplicity and order are observable, buffering is required to keep them, and buffering is a structured use of the existing state basis *plus list data* — which the kernel then lacked.

The failed abstractions, generalized. Phase 9a tested the models the brief named. Model A, *latest*; B, *count*; C, a fold such as a sum; D, a fixed tuple of the newest entries; E, the list. A–D are lossy, with witnesses (*latest_not_lossless*: [1,2] vs [2]; *count_not_lossless*; *sum_not_lossless*: [1,4] vs [2,3]; *modelD_not_lossless*). The general statement behind A and D is *bounded_summary_not_lossless* (**formally proved**): any summary depending only on the newest k entries, for any fixed k , identifies a k -entry window with a $(k+1)$ -entry window. A lossless summary is injective and therefore unbounded (*lossless_iff_injective*). The claim discipline is precise: not that the list is the only lossless representation, but that any lossless one must be injective on windows and hence unbounded in size, and that an ordinary list is the smallest general sequence representation tested.

The object-language buffer. With list τ (the `Ty.list` constructor, `Value.list`, and six registered operators `nil`, `cons`, `length`, `take`, `reverse`, `head`, with `length` yielding $q \ 0$), the Phase-5 construction is five ordinary declarations (`Surface/Buffer.lean`):

```
log      @src : cons src (delay nil log)      -- source-side accumulator
logD     @dst : sync src nil log              -- the log, transported
seen     @dst : length logD                  -- log length now
cursor   @dst : delay 0 seen                  -- log length at the previous activation
window   @dst : reverse (take (seen - cursor) logD) -- the new entries, oldest first
```

Nothing new is evaluated: *delay*, *sync* and registered operators. *logD* is the one cross-domain read, an explicit sync with the explicit initial value `nil`. **Theorem M** (*buffer_window_correspondence*, **formally proved**): for every schedule, every input, every destination domain and every tick,

$$\text{MEv } S \Delta I \text{ dst } t \text{ [] window } \Downarrow \text{list } (\text{map } (I \text{ src}) (\text{windowTicks } S \text{ src dst } t)),$$

whenever the six declarations are realized as above and *src* is an input. The strictly-before rule is untouched — the window at t contains source activations $< t$ only. The elaboration is well typed in any environment with the declared types (*buffer_elaboration_well_typed*) and well clocked with *src*, *log* in the source domain and the rest in the destination (*buffer_elaboration_well_clocked*). The list summary is lossless by injectivity (*buffer_lossless*); its entries are indexed by the window ticks in strictly increasing order (*window_to_list_preserves_order*), and for every predicate on values the number of entries satisfying it equals the number of window activations whose value does (*window_to_list_preserves_multiplicity*).

Executed (`Experiments/BufferAlternatives.lean`, on Phase 5’s multi-rate schedule with fast events at ticks 1 and 2): at the slow activation 3 the window is [none, some 1, some 2], at 6 it is [none, none, none], while *latest* sees some 2 at 3 (*buffer_trace*); the two histories Phase 5 could not tell apart under *latest* are told apart by the window (*buffer_distinguishes_what_latest_identifies*); the elaborated design passes the typing, clocking and causality checkers. The buffer is also a Phase-8a system with no new binding kind: a sensor component provides its event *and* its log, a consumer requires the log through an ordinary transported binding with initial value `nil` and computes the window (*transport_trace*), whereas binding the event itself through *sync* yields *latest* and loses the tick-1 event (*latest_transport_loses*). “Buffer or not” is a choice of *which provided port to bind*, visible in the interface.

Capacity is a validation obligation. The kernel model is the unbounded log. A deployment has finite memory and must show that no window exceeds its capacity. *CapacitySufficient* $S \text{ src dst cap } T$ states that every window up to horizon T fits; it is decidable for a finite horizon; *requiredCapacity* computes the least sufficient value with a proof (*requiredCapacity_sufficient*); and for periodic schedules the horizon is unnecessary — a window never holds more than one destination period of source activations (*periodic_window_bound*), so one period is a sufficient capacity at every horizon (*periodic_capacity_sufficient*). Overflow policies are explicit functions of the unbounded window — *dropOldest cap*, *dropNewest cap* — and under a sufficient capacity both are the identity (*sufficient_capacity_*

preserves, bounded_buffer_agrees); under an insufficient capacity they change the trace (negE, executed: the three-entry window at tick 3 becomes two entries under either policy, capacity 2 fails the check at horizon 6, the required capacity at horizon 30 is 3). Consequently the only overflow policy that preserves the kernel semantics is to *reject the deployment*; dropping is a semantic change and must be written by the designer as ordinary computation over the window if wanted.

What is still not a kernel type. An event stream is a data-typed declaration in a domain; an occurrence is its value at an activation; a lossless cross-domain view of it is the five declarations above; latest, count, coalesce, drop, sample are ordinary computations over window. None of this needs an Event type, a buffer primitive, a scheduler order, same-tick visibility, an implicit overflow rule or an effect system, and none was added. The negative examples of the phase also settle a subtler point: a clone of fast with the identical schedule is a different domain, and listing the value does not make the direct read legal (negF) — equal rate is not the same domain, as in Phase 5.

Production. Production follows this exactly, with two additions the formal model deliberately left to engineering (ADR-0024, ADR-0027; docs/spec/deployment-capacity.md). A collection in the generated core is alloc::vec::Vec<T> stored last element first, behind a collections feature of the runtime crate turned on exactly when the plan carries a list; a program without lists stays allocation-free and Copy. No capacity is imposed by the core and no core ever drops a value. *The design writes its bounds:* a remembered collection is bounded by the operators that bound it — take cap ... — and by nothing else, so the deployable window is the Phase-9a construction with take cap on the log and the count kept apart:

```
count @fast := 1 + delay 0 count
log    @fast := take cap (cons x (delay [] log))
logD   @slow := sync fast [] log
seen   @slow := sync fast 0 count
cursor @slow := delay 0 seen
window @slow := reverse (take (seen - cursor) logD)
```

With cap ≥ required(fast → slow) its window equals the unbounded construction's at every tick — the production reading of bounded_buffer_agrees, differentially tested across the reference evaluator, the executable-IR interpreter and the generated core (corpus bounded_buffer against buffer, 7 and 3 000 ticks); with cap smaller the oldest values of a window are the ones missing (corpus overflowing_buffer). *The toolchain computes the bound and the requirement:* a sound static bound per declaration and cell (bdl-exec-ir::bounds), the required window capacity per crossing under the deployment schedule (bdl-reactive::capacity, the production Capacity.lean), a CollectionsReport with readiness, byte estimates and per-crossing requirements, and deployment.* diagnostics; an unbounded state is reported on a host and **refuses the artefact on a bounded-memory target** (bdld compile --bounded-memory --period). The static bound is an engineering analysis with no theorem behind it — recorded as such in Part XIII. A ring buffer specialized for transport buffers was considered and deferred: the window is not a construct the compiler can recognize without a surface form (ISS-0001), and a bounded refinement must be visible in the design to be equivalent — take cap is exactly that.

Part VIII — Behavior Systems

Phase 8 asked whether behavior can be a first-class *design object*: packaged behind an interface, instantiated with fresh identity, bound to other behaviors, nested, and flattened into the same kernel — without any kernel change, without Event/Signal kernel types, without a StateHandler kernel, without general effects, and without name-based identity. Phase 8a answers for reusable components; Phase 8b for the authoring structure designers form before packaging. The two are kept apart on purpose, and the distinction is the main result of this Part.

Behavior components (Phase 8a)

The requirements report. The phase began with a requirements report (docs/notes/behavior-system-requirements.md) rather than a formalization: six capability questions — can a behavior be reused, instantiated twice, bound to other behaviors, nested, opened for later completion, substituted — and, for each, whether the answer is REQUIRED of the kernel, OPTIONAL, or DERIVABLE from what exists. The outcome was that everything is derivable from Phase-1 realization plus renaming, and the formal development proves it.

Renaming. Ren bundles four renamings — declaration identities, semantic identities, clock identities, output identities — and the kernel’s judgments are equivariant under it: HasType.rename (typing, given agreement of the three environments on the image; no injectivity needed), Satisfies.rename, Clocked.rename (the domain judgment, for a clock environment that agrees on the declared identities), and an abstract condition on evidence, Evidence.Equivariant. This is the machinery of Phase 8; nothing else is new.

Interfaces and components. A Port is a template declaration by local identity with the public part of its interface and its parameter clock. A BehaviorInterface has required ports (unresolved declarations a composer binds), provided ports (declarations offered to others), elaboration-time parameters (unresolved data-typed declarations bound to closed constants at instantiation), and clock parameters (the domains the template is written against). A BehaviorComponent is an interface, a template Design over local identities below a width, and a partition of its concepts and sinks into private (freshened per instance) and shared. Realizes $ev \vdash C$ is a predicate over the *existing* judgments: the template is a well-formed design; every required port is an unresolved declaration of the stated interface; every provided port is declared with it; parameters are unresolved, data-typed and clock-free.

Fresh instantiation. $fresh\ W\ k\ n = W \cdot (k+1) + n$ maps local identity n of instance k into the global space, with decode its inverse; the identities of two distinct instances never coincide (inst_decl_disjoint, Theorem A). The encoding is a device, not a semantics: any injective allocator would do, and production uses the same allocator its own ids come from. An instance is a component with a clock-parameter assignment; Ren.inst renames the template’s private identities into the instance’s range and its clock parameters to the assignment.

Bindings and flattening. A Binding realizes a destination port of one instance from a source — a port of another instance, or a closed constant — with an optional *transport*: none for a direct reference in the same or an agnostic domain, some init for sync from the source’s domain with an explicit initial value. A BehaviorSystem is a width, a list of instances, a list of bindings, the shared concept environment and the external sinks. flatten is the union of the renamed instances followed by the bindings applied as Phase-1 realization steps: the destination port is realized as declRef src or sync c init (declRef src). The result is an ordinary Design. Every existing pass consumes it unchanged.

Theorems B–I (Behavior/Preservation.lean, **formally proved**): under ComposeWF — every instance realizes its interface, every binding is well formed (BindingWF: types agree, a direct binding’s source is in the destination’s domain or agnostic, a transported binding’s source has a domain), external sinks are driven by at most one instance — the flattening is globally well formed (flatten_WF), causal under an acyclic inter-instance graph (flatten_causal), well clocked (flatten_wellClocked), single-driver (flatten_singleDriver); open ports stay open (open_port_stays_open). Evidence must be Monotone, Equivariant and PortSound (a discharged commitment survives when a port copy is realized by a reference to a declaration of the same interface); a concrete compositional evidence model would discharge these once and is still open.

Semantics (Theorem J, Behavior/Semantics.lean): for wiring designs with closure-free inputs and *direct* bindings, the value of a declaration in an instance evaluated alone with a consistent modular input equals its value in the flattened system (eval_flat_to_inst, eval_inst_to_flat, modular_iff_flat). The restriction is exact and recorded: transported

bindings under MEV need a domain-indexed input for the transported port, and higher-order bodies are not covered — the same obstacle in both directions.

Substitutability. `IfaceRefines A B`: B may replace A when every port A provides, B provides with the same type and clock, and every port B requires, A required. `substitute_composeWF`: replacing an instance’s component by a refining one preserves composition well-formedness. `toComponent` packages a whole system as a component of a chosen interface (the decidable side condition that the chosen interface is realized is not proved).

Counterexamples (`Experiments/BehaviorAlternatives.lean`): a name-based identity collides on double instantiation; a shared clock captured inside a template cannot be re-bound; a binding across domains without transport is rejected by the domain judgment; a binding of a required port to a wrong type is rejected; two instances driving one external sink violate single-driver. The lamp system — a source instance and two dimmer instances at different domains — is executed through the flattening.

Production (ADR-0021, ADR-0022; `bd1-system`; `docs/architecture/behavior-systems.md`): components, stored port contracts (“a component interface is a stored promise”: the contract is persisted and checked against the body, so an edit to the body that breaks the promise is reported at the contract), instances with fresh identity from the project’s allocator, direct and transported bindings, flattening with an *origin map* back to instances and ports so diagnostics land on the instance, composition validation, packaging, versions and substitution. There is one BDL: no system type checker, evaluator, clock judgment or code generator exists. Every project is a behavior system — a design with no components is the degenerate one — and the flat design is derived and never persisted. What production supports beyond the theorem fragment — transported bindings under the multi-domain evaluator, higher-order bodies — is *production-tested*, not proved (Part XIII).

Multi-clock limitations, exactly. Theorem J is proved for `Ev` (one domain) with direct bindings. The clock parameter list must cover the clocks of declarations and sinks; a `sync` clock appearing only inside a body is renamed to a fresh domain, harmless for `Ev` and `Clocked` but something an elaborator should collect. `InstAcyclic`, the inter-instance graph condition for causality, is too coarse for extraction (below); a port-level graph would subsume both results.

Behavior groups (Phase 8b)

Three things that must not be confused. A designer who selects several mapping blocks and chooses *Group as Behavior* has made an authoring decision, not a semantic one. A designer who later chooses *Package as Component* has made a semantic decision: from that point the unit has an interface, can be instantiated with fresh identity, and can be bound.

	mapping / declaration	BehaviorGroup	BehaviorComponent
identity	<code>DeclId</code>	<code>GroupId</code> and a member list	template identities, fresh per instance
public interface	its own <code>DeclInterface</code>	none — only projections	required / provided ports, clock parameters
instantiation	—	none	fresh identities
flattening	—	none needed	flatten
semantics	kernel	none : authoring metadata	elaborates to declarations

The hierarchy mapping $\rightarrow$ group $\rightarrow$ component $\rightarrow$ system has exactly one semantic step, packaging, and that step is an elaboration into the Phase-8a system model.

Transparency. A `BehaviorGroup` is a `GroupId` and a list of member `DeclIds`; a `GroupedDesign` is a `Design` with a list of groups beside it; `eraseGroups` forgets the list. Every group operation — `group`, `ungroup`, `addMember`, `removeMember`, `move`, `merge`, `split` — acts on the list and leaves the design untouched, so every kernel judgment of the erased design is the *same proposition* before and after (Theorems A–G, each proved by `rfl` or `Iff.rfl`). That is not a weakness of the theorems; it is the point. Dependency edges are untouched; nesting is a relation on the flat group list with no recursive structure. In the Phase-1 vocabulary this is a fourth class below “validation-only”: nothing is rechecked because nothing changed.

Boundaries as projections. Over a finite enumeration of the design’s declarations: `crossIn` lists the non-members some member depends on (the required ports), `crossOut` the members some non-member depends on (the provided ports), `openMembers` the unresolved members, `drivenMembers` the members driving a sink, `privateMembers` the rest (Theorems H–L: `mem_crossIn`, `mem_crossOut`, `internal_not_crossIn`, `internal_only_not_crossOut`). The aggregate input socket a collapsed group shows is `crossIn ++ openMembers`; the output socket is `crossOut`. None is a declaration. Theorem H, `socket_no_fanout`, is what makes the socket safe: $a \in \text{crossIn}$ says *some* member depends on a and nothing about the others. Counterexample 6 shows what goes wrong if the socket is realized as a declaration every member reads (a member acquires an instantaneous dependency it never had); Counterexample 5 shows the tuple-returning-declaration encoding (the consumer of one mapping comes to depend on the inputs of all of them). The kernel has no tuples for boundaries, and this is a reason not to add them for this purpose — the products of Phase 9b are values, not sockets.

Inference rules and their counterexamples. Required = `crossIn`; provided = `crossOut`; private = members neither provided nor driving a sink; clock parameters = every clock the design uses (a group owns no domain, so nothing is captured — Counterexample 3 shows the reconnecting binding failing its clock condition when a clock is captured); physical sinks are not ports — a member that drives a sink keeps its drive edge inside the component and the sink stays external (Counterexample 4: converting a sink into a provided port yields a behavior with no physical effect). Two naive rules are rejected: every reference of a member as required exposes internal producers and changes behavior (Counterexample 1); only the members’ own open declarations hides external dependencies and leaves the template ill-typed (Counterexample 2).

Extraction. Extract builds two *restrictions* of the design: the component keeps the members and holds an unresolved copy of each crossing-in declaration as a required port; the residual keeps the non-members and holds an unresolved copy of each crossing-out member. It forms a Phase-8a system — instance 0 the residual, instance 1 the component, clock parameters mapped identically — with one direct binding per crossing declaration. Each binding is a Phase-1 realization step: the port copy is realized as a reference to the home copy. No body is translated, copied across the boundary, or rewritten. **Formally proved:** both templates realize their inferred interfaces (`restrict_realizes`, needing Evidence.InterfaceLocal: a discharged commitment depends only on the interfaces of the referenced declarations); the system is a well-formed composition (`system_composeWF`) and its flattening a well-formed design (`flat_WF`, Theorems M–Q); causality needed its own argument — a group with both inputs and outputs is never `InstAcyclic`, yet the flattened instantaneous graph is the original graph with every crossing edge subdivided through a port copy, and doubling the original rank witnesses it (`flat_causal`: rank $2 \cdot r$ on home copies, $2 \cdot r + 1$ on port copies); open members stay open; private members are not provided ports, source no binding, and are never referenced by the residual (`private_unobservable`); each crossing-out member is its own provided port (`provided_iff`) — several independent mappings yield several independent ports; and behavior is preserved (Theorem R, `orig_iff_flat`): for a wiring design with closure-free inputs, an original declaration and its home copy in the flattened extraction evaluate to the same value at every tick.

Identity. Before packaging nothing is renamed. After packaging the templates keep the original identities, and the flattened system carries Phase-8a fresh identities: an original n lives at $W + n$ on the residual side and $2W + n$ on the component side, with a port copy on the other side where the boundary is crossed; a later second instance receives $3W + n$. The group’s identity never becomes a component identity.

Production (ADR-0019; `.bdl/authoring.json`; `ui/layout.json`). A group persists as an identity and its member list by identity, in the authoring sidecar — no types, clocks, formulas, outputs or sockets; anything derived is recomputed and not authoritative if stored. Collapsed/expanded state, position, size, colour, member ordering and socket placement are layout and affect no check. Group operations never trigger semantic invalidation: the workspace does not re-run typing, causality, clock, output or hardware checks and does not clear cached verdicts, because these are the same propositions (`group_is_identity_on_design`). *Package as Component* computes interface candidates from the member set and lets the designer widen them — promote a private member, add an open required port — but not narrow below the inferred required set or turn a sink into a semantic port; the flattened result is guaranteed well formed under the stated hypotheses, and the causality check passes even though the instance graph shows edges both ways, so packaging is not gated on `InstAcyclic`. What cannot be inferred and is asked: whether an unresolved member is the component’s input or an open internal; whether a private member should nevertheless be exported;

whether a member's physical sink is private to the component; display name and documentation. Packaging a group *inside* a component body is open (ISS-0007).

Hardware interaction

Components partition sinks into private and external. Private sinks are freshened per instance, so two instances of a component with a private status LED are two LEDs and two requirements; external sinks are shared and may be driven by at most one instance in a system (`ExternalSingleDriver`). Requirements generation (Part X) runs over the flattened design's sinks, so the number of instances is exactly what the board must accommodate — a fact the hardware validator sees only through the flat design, which is the intended separation.

Part IX — The Environment and the Physical Boundaries

Everything in Parts III–VIII happens *between* two boundaries. On one side the environment provides values — a tilt, a temperature, a button state — and the design observes them; on the other side the design’s values leave it through physical outputs. The kernel’s evaluation relation has no effect component: a derivation relates a tick, an environment, a term and a value, and both boundaries are named by things outside the relation — an input stream on the way in, a drive edge on the way out. This Part is the account of those two boundaries as the formal development settled them in three steps: the canonical type of a relationship that reads nothing and the derived *source role* (Phase 12, ADR-0029 and ADR-0032); the physical sink with its single explicit driver, and why the dual form $A \rightarrow ()$ is not a consumer (Phase 6 and Phase 12); and the provision of a Source at deployment by a raw reading and a device transducer, proved transparent to the design (Phase 13, PRP-0001). The last of these is the conceptual completion of the first: a Source is environment provision *at the concept’s type*, and provision is the construction that says how a value of that type comes to exist on a product without the design being able to tell.

The Part is organized by the boundary, not by the phase. The reader who wants the chronology has Appendix G.

The canonical type of a relationship that reads nothing

A relationship declared with no inputs — mapping `TempSensor : RoomTemp`, mapping `boost : Brightness` — is typed by the kernel at its output: `expectedType = B`. Production (ADR-0029) gives the same relationship a *canonical type* $() \rightarrow B$, with $()$ the empty product, so that every relationship has one type shape $\text{domain}(\text{inputs}) \rightarrow B$: $\text{domain}([]) = ()$, $\text{domain}([A]) = A$, $\text{domain}([A, B, \dots]) = A \times (B \times \dots)$. The surface spells mapping $f : B$ as shorthand for mapping $f : () \rightarrow B$; f , $f()$ and $f(())$ are one reference; $()$ is refused as a value anywhere else, as an output, and as a value form; hover and Explain show the canonical type and the sentence “its canonical domain is $()$, the empty product; the kernel encodes $() \rightarrow B$ as B ”. Production’s `bdl_ir::Ty` has a `Unit` constructor for this — a real IR type with the ordinary predicates — that never types a Core term, a representation or a runtime value. ISS-0014 asked whether the kernel and the canonical type agree by theorem.

Phase 12 (`Surface/UnitDomain.lean`) answers without adding a unit to the kernel. The canonical types live in an *interface layer* above it — `CTy` is the kernel’s `Ty` plus unit, interface arrows and domain products — and the kernel interface type is the value of a normalization function on them (**formally proved**):

$$\text{elim}(\text{canonical}(s)) = \text{encode}(s), \quad \text{encode}(\langle [], B \rangle) = B, \quad \text{encode}(\langle A_1, \dots, A_n; B \rangle) = A_1 \rightarrow \dots \rightarrow A_n \rightarrow B,$$

where `elim` performs unit elimination ($() \rightarrow B \mapsto B$) and currying ($(A \times B) \rightarrow C \mapsto A \rightarrow (B \rightarrow C)$), is total by a weight both steps decrease, and has no value on the bare unit — `elim () = none`: the unit is never the type of anything. The encodings are inverse over every signature whose output is not an arrow, hence over every concept signature (`decode_encode`, `canonicalOfKernel_encode`, `encode_injective`, `canonical_injective`), which is production’s inverse test as a theorem for all signatures. The categorical slogan $\text{Hom}(1, B) \cong B$ is thus, in this model, definitional at the kernel (the realization obligation of $() \rightarrow B$ is $\vdash e : B$ in the empty context, `zero_input_obligation`, by `Iff.rfl`), a theorem at the interface (the inverse pair), and a bijection denotationally (`homUnit : (1  $\rightarrow$   $\beta$ )  $\cong$   $\beta$ )`.

Why the unit is eliminated *before* the kernel is a theorem, not a preference. The literal alternative — realize $() \rightarrow B$ as a lambda over the unique argument and read it by application — is refused by typing: no term $\lambda x. \text{delay } i \ e$ has any type, for any binder domain, because `delay` and `sync` are typed only in the empty context (`delay_not_under_binder`, `sync_not_under_binder`), while `delay init e : B` is a legal realization of $() \rightarrow B$ under the kernel encoding (`zero_input_memory`). A kernel unit binder would forbid memory in every zero-input declaration; a counter `boost : = delay 0 (boost + 1)` would become untypable. `lams_typed` states the general case: a formula body checked in the context of its inputs is a realization of the encoded type under n binders, and for no inputs there is no binder.

The three spellings are one term (`refForms_agree`), so they share typing, evaluation and the clock judgment by reflexivity (`HasType.refForms`, `Clocked.refForms`). Two theorems say what the unique argument could have carried, and that it carries nothing: a reference’s value at a tick is independent of the local environment — a realized declaration evaluates in the empty environment, an unresolved one is read from the input stream (`Ev.declRef_env_irrelevant`, `MEv.declRef_env_irrelevant`) — and two readings in one tick agree under any two environments (`same_`

tick_same_value). There is no per-reference call to repeat; a zero-input declaration is evaluated as a declaration, once per activation, and the reader observes that value. No unit clock, no extra activation, no evaluation step.

The source role is a realization state, not a type shape. $() \rightarrow A$ describes the shape of a signature; a *source* is a declaration with no realization, whose value at every tick is provided by the environment: $\text{Source } \Delta d := \text{realizationOf } d = \text{none}$, and *source_value* gives that value as $I(d, t)$ in every domain and under every environment — indexed by the declaration and the tick alone, which is the formal sense in which the unit argument carries no temporal or environmental information. A resolved $() \rightarrow A$ never consults the input stream (*resolved_not_source*); it is not a sensor, and every mathematical $() \rightarrow A$ is not one either. Production’s *simulation input* — Studio offers a declaration as an input when it is unresolved and unit-domain — is $\text{SimulationInput} := \text{Source } \wedge \text{UnitDomain}$, a narrowing the kernel does not need (its input stream provides a value for every unresolved declaration, arrow-typed ones included) and that is recorded as surface policy over the same semantics: on a unit-domain source the two agree (*SimulationInput.value*). No source kind exists in either model; production’s $\text{Signature}::\text{is_unit_domain}$ is the one predicate, and the formal $\text{UnitDomain } \Delta d$ is the same predicate on the kernel type.

The two production consequences of the unit domain are corollaries of existing rules (**formally proved**). Only a data value — never a function — can be remembered or transported: a well-typed sync or delay of a reference forces the referenced type to be data, hence not an arrow, hence unit-domain (*transport_needs_unit_domain*, *delay_needs_unit_domain*) — production’s diagnostic *f has inputs, so its value cannot be carried across timing domains* is this theorem’s message. And a driver of a sink is a value of the accepted type, so when the sink accepts a concept the driver is unit-domain (*driver_is_unit_domain*, from *DriveWF*). The dual form $A \rightarrow ()$ is taken up below.

Executed (**executable example**, *Experiments/UnitDomainExamples.lean*): the lamp’s signatures round-trip through both encodings and *elim* computes them; $\text{boost} := \text{delay } 0 \text{ (boost + 1)}$ read as *boost*, *boost()* and *boost()* gives 0, 1, 3 at ticks 0, 1, 3, and the same under a non-empty local environment; the unresolved *TempSensor* reads the input while *boost* ignores it entirely; *infer* accepts $\text{delay } \dots : 00$ in the empty context and refuses a binder around it; sync of *boost* and of *TempSensor* is typed and sync of the one-input *dimByTilt* is refused.

Verdicts (**informed by FV**): unit in the canonical interface notation — keep, above the kernel; a kernel unit type, a unit runtime value, a unit term — remove; a source semantic kind — remove, derive from the realization state; the source surface role — keep in the surface; $A \rightarrow ()$ as a physical sink — remove (below); the output/drive boundary — keep. Production’s correspondence row for ADR-0029 reads *formally proved (model)* at the interface and *transcribed* at the kernel encoding, and ISS-0014 is resolved.

Source, Rule, Value: production’s reading of the realization state

Production reads the two kernel facts a declaration carries — does its type have inputs, does it have a realization — as one derived **relationship role** (ADR-0032, amended 2026-09-20; *bdl_model::RelationshipRole*):

```
role(x) = Rule    if x reads something           (its canonical type is an arrow)
        = Value   if x reads nothing and has a realization
        = Source  if x reads nothing and has none
```

The role is derived, never persisted, never authored, and distinct from every *state*: a declared rule, an invalid formula, a rule nothing applies, a driven value, a bound port, a clocked declaration are states that vary within a role and never move it. *Has a realization* means an attached definition — a formula whether or not it checks, the reference a binding made, memory, a constant, a parameter’s argument — and never a draft; the role follows the committed revision. The rule has one home, *MappingBlock::role*; the compiler states it in *MappingAnalysis.role*, the daemon on every *MapView.role* (protocol 0.20), and Studio maps the enum and re-derives nothing. The earlier arrangement, in which Studio derived a *Source* from the shape of the projection, was replaced by this one authority on 2026-09-20 and is not what the product does.

Only the *Source* case has a formal object: $\text{Source } \Delta d := \text{realizationOf } d = \text{none}$ on the unit domain (FVD-0118, *source_value*, *resolved_not_source*). *Rule* and *Value* are production’s names for the arrow type and the realized unit domain; the kernel has no need to distinguish a *Value* from a *Rule* beyond typing, and none was added. The role crosses the component boundary as a fact of *one design*: the relationship behind a required port is a *Source* of the body, provided through the port; a provided port realized inside is a *Value*; an unbound required port of an instance

is a Source of the system (Theorem H, Part VIII); a base relationship a binding realizes is a Value at the top level because its flattened copy carries the binding. Production’s `relationship-roles.md` is the normative matrix.

Two consequences the kernel already fixed, restated in the role vocabulary. **A Source may drive an output.** `DriveWF` (Part IX below; FVD-0051) requires the driver’s type to equal the sink’s accepted type and its domain to be the sink’s; it mentions realization nowhere. So a Source — a value the environment supplies — may be passed straight to a light, and production’s output pass accepts it; a Rule is refused because its type is an arrow (`output.type_mismatch`). Any statement that *only a Value can drive an output* is stale. **A provisioned Source is a Value.** After Phase 13’s provision (below) the target has a realization, so the same rule gives the same answer: no `ProvisionedSource` role exists or is needed (ADR-0032’s amendment; FVD-0127, `provision_source_role`).

Applied rules. A Rule has no value at a tick — it is a function — so the simulator shows it no column, and it does work only when a Value applies it. Production states, per analysis, references (the declarations a realization names by identity — the kernel’s `DependsOn`) and `applied_by`, its **direct** inverse, never a transitive closure; a rule nothing applies is reported as `reactive.rule_unapplied` (informational) and the value that would apply it is offered as the action `rule.apply`. These are IDE and product facts over the dependency graph of Part III, not kernel kinds, and the monograph treats them as such.

Commitments, exactly. The kernel gives every declaration a commitment list and makes evidence for it part of well-formedness (Part III); Phase 13’s `provision_wf` needs evidence for each target’s commitments on its new realization (FVD-0128). Production authors no commitments at the snapshot: `require` is a reserved word without a production, every declaration’s commitment list is empty, and no commitment solver exists. The formal hypothesis is therefore *vacuous* in production today, and this document does not present a production commitment mechanism as implemented.

Physical outputs without arbitration

The model

A declaration computes a value; it does not move hardware. Physical effect happens only through an explicit **drive edge** from a declaration to a **physical sink**:

- `OutputId` — the nominal identity of a sink, a logical actuator channel;
- $\Omega : \text{OutputId} \rightarrow \text{Option OutputSpec}$, a sink’s accepted type and clock, declared by the deployment;
- $\beta : \text{DeclId} \rightarrow \text{Option OutputId}$ — the drive edges, a write-once per-declaration projection of the same shape as `K`;
- `DriveWF  $\Omega$  K  $\Delta$   $\beta$` — each edge is well formed when the driver’s expected type *equals* the sink’s accepted type and the driver’s clock is the sink’s clock;
- `SingleDriver  $\beta$` — at most one driver per sink: if βd_1 and βd_2 are both some o then $d_1 = d_2$;
- `CompleteOutputs  $\beta$  req` — every required sink is driven.

Nothing was added to types, typing, the domain judgment, the evaluation relation, or the grant. The binding neither coerces nor converts nor synchronizes. A declaration typed `Tilt`, or bare `q Angle`, cannot drive a `MotorAngle` sink. A sink that accepts a representation type needs an explicit `rep`-typed declaration in front of it, so the distinction between semantic target and hardware representation stays visible. A slow driver reading a fast value must sync it upstream, since a fast driver cannot drive a slow sink and the edge never synchronizes.

Sink identity is nominal for the same reason concept identity is. Keying the binding by type collides when two servos accept the same type; keying by concept conflates a concept with a device, since one concept may feed several; keying by declaration makes two declarations that both mean the steering motor into two sinks, and the multiple-driver counterexample cannot even be stated. “Desired steering angle” is a value; “the steering motor” is a resource; the kernel keeps them in different sorts.

One final driver

The principle is *many contributors, one explicit final driver*. Take two declarations driving one sink, each globally well typed, well clocked, causal, and individually well formed. Only `SingleDriver` fails, and it fails globally rather than at either edge. What has gone wrong is semantic: with two drivers the physical output is not a function of

the tick, and there is a tick at which the sink receives two values. With one driver, the physical output is a partial function of the tick, and with SingleDriver it is unique wherever it exists:

$$\begin{aligned} \text{PhysicalOutput } S \Delta I \Omega \beta o t v &:= \exists d \text{ spec.} \\ \beta d &= \text{some } o \wedge \Omega o = \text{some spec} \\ \wedge \text{MEv } S \Delta I \text{ spec. clock } t \parallel (\text{declRef } d) v. \end{aligned}$$

Contributors are dependencies, not drivers. `base + corr -> final -> motor` passes every check; priority is an ordinary conditional in the single driver; blend, maximum, and clamp are ordinary declarations of the target type. Why arbitration must be explicit is best shown rather than argued: first-wins, last-wins, and maximum over the same value graph give three different physical outputs. A hidden policy is a design decision made on the designer's behalf.

Binding an unbound declaration to an undriven sink is a refinement and preserves SingleDriver. Binding to an already-driven sink is invalid. Retargeting a sink's accepted type, renaming a sink, or detaching an edge invalidates an unchanged design. Partial designs may leave sinks undriven; executable designs may not.

What was removed

The earlier draft of BDL had an effect row on the behavior judgment, action requests as values, and per-context policies that allowed, suppressed, transformed, and arbitrated requests, resolved in a dedicated phase of each tick. Each was tried in toy form against the single-driver model, and the outcome is narrow. Direct effect rows — the set of sinks a declaration drives — are exactly the drive edges, and single-driver is exactly their pairwise disjointness. Propagated rows, which also include the sinks of everything a declaration reads, flag a valid design in which a display reads the driver. Action values move the conflict into the collector that consumes them, which must then be a policy, which is the single driver by another name. None of this bears on richer effect systems; it says that these formulations add no rejection the single-driver rule lacks. Per-context action policies no longer exist as a mechanism, and the StateHandler cases that involved them — event-latched activation with exit-wins, state-local output choice, nested choice with an output — are ordinary declarations with one driver and were run as such.

Why $A \rightarrow ()$ is not a sink

Once zero-input relationships have the canonical type $() \rightarrow A$ (above), the dual form suggests itself: could a physical consumer be a relationship $A \rightarrow ()$, a function that takes a value and returns nothing? The kernel has no unit type, so the form cannot be written; Phase 12 records why it should not become writable (**formally proved**, denotationally). In a pure total language every function into the one-point type is the same function — `unit_codomain_collapse :  $\forall f g : A \rightarrow 1, f = g$` , by function extensionality alone — so two “consumers” $\text{sink}_1 \text{ sink}_2 : A \rightarrow ()$ are indistinguishable (`consumers_indistinguishable`): nothing in a value of that type says *which* physical output receives A, or that anything receives it at all. The kernel's evaluation relation has no effect component (`eval_independent_of_drives`); a derivation relates a tick, an environment, a term and a value, and the physical consequence of a value lives outside it. Naming a receiver needs an effect or output semantics, and BDL already has exactly one: the sink identity `OutputId`, the drive edge with `DriveWF` (the driver's type equals the accepted type, in the sink's domain), `SingleDriver` and `CompleteOutputs`. The receiver is named by the edge, the value delivered is the driver's, of type A, and the driver is a unit-domain declaration whenever the sink accepts a concept (`driver_is_unit_domain`). This is a design result, not a preference: physical consumption stays on the drive boundary, and $A \rightarrow ()$ is removed from consideration.

Provision: how a Source gets its value on a product

A Source $s : () \rightarrow C$ is environment provision at the concept's type: the kernel input gives a `Temperature`, and nothing in the design or in the kernel says how a value of `Temperature` comes to exist. On a product it does not. An ADC yields counts, a GPIO a level, an I²C sensor a register image. The gap between the abstract Source and the physical reading is the last boundary the formal development had to settle, and the shape of the answer is fixed by everything before it: no new kernel construct, no Source kind, no effect, and no way for the design to tell the difference.

The picture, before the theorems:

abstract Source	$s : () \rightarrow C$	unresolved; its value is the environment's, at type C
raw provider	$r : () \rightarrow R$	a fresh unresolved declaration at the device's raw type

device channel	$\text{tr} : R \rightarrow \text{Rep}(C)$	a pure term with its transfer function, from a device profile
provisioned Source	$s := \text{mk } C (\text{tr } r)$	an ordinary realization; s is now a Value

The design file keeps mapping $\text{TempSensor} : () \rightarrow \text{RoomTemp}$; the deployment supplies r , tr and the realization; the consumer tooHot computes the same truth values from counts as it did from the induced temperature. The key result is a containment, and the direction matters: every trace of the provisioned design under a raw input is a trace of the abstract design under the induced input — $\text{Trace}(\text{device}) \subseteq \text{Trace}(\text{abstract})$ — and this strict refinement is the *normal, intended* case. A saturating ADC never yields 451 K; the abstract design admits that observation and the provisioned one does not; nothing is wrong. Trace *equality* needs a **joint section** — one raw trace every channel transfers to what the abstract input gives its target — and the absence of a joint section is not a deployment error and is reported by no surface. Production’s proposal PRP-0001 asked for this construction in seven claims; Phase 13 built it, kept three of the claims, and corrected four. What follows is the construction and the corrections (**formally proved** unless marked; `Surface/Provision.lean`; FVD-0121 ... FVD-0130; PRP-0001 remains a draft, revised, and nothing in production implements it).

The construction. A *channel* is a representation type rep , a term tr , its transfer function transfer on values, and the coherence computes (the term computes the function on every raw-typed value); the term is **pure** — no `declRef`, `delay` or `sync`. A *device profile* is a raw type (sem-free data) and its channels; it mentions no concept. A *provision* is one fresh raw declaration r with its clock and an assignment of channels to target Sources — several targets may share one raw reading (an IMU image feeding pitch, roll and acceleration), and one target is the singleton case. Then

$$\text{provision}(\Delta, P) = \Delta [r \mapsto \langle \text{raw}, [] \rangle \text{ unresolved}] [s \mapsto \text{mk}_c (\text{tr} (\text{declRef } r)) \text{ for each target } s : \text{sem } c],$$

with $\text{tr} (\text{declRef } r)$ alone at a representation-typed Source; the raw declaration’s kernel type is `raw` and its canonical type $() \rightarrow \text{raw}$ (above). Fitting is decidable: at $\text{sem } c$ the concept’s representation is the channel’s; at a representation type the types coincide. Nothing enters the kernel: provision is a function on environments built from `DesignDecl`, `declRef`, `app` and `mk`.

Why purity, and why a transfer function. Typing the channel term in the *empty* design already forbids reading a declaration (`Channel.WF_refFree`), but not memory: $(\lambda k. \lambda n. k) (\text{delay } 0 \ 1)$ is typed at $q_0 \rightarrow q_0$ and maps the raw value 7 to 0 at tick 0 and to 1 at tick 1 (exD, **executable example**). A transducer must be a function of the raw reading, so the profile condition is purity — equivalently, typed in the empty design and delay-free (`pure_iff_delayFree_of_wf`). The channel carries transfer beside tr because the *induced* abstract input must be a function: extracting it from per-tick existence of a value would be a choice principle, which this development does not use. The grant argument the proposal relied on is a theorem of the existing rules: a declaration typed $\text{sem } c$ is realized under $\text{Grant.of } (\text{sem } c) = \{c\}$ (`realization_checked_under_own_grant`, `grant_of_sem`), and a channel term typed under Grant.none constructs nothing (`channel_constructs_nothing`); the same $\lambda x. \text{mk } \text{RoomTemp } x$ is refused under the empty grant and accepted under the Source’s own (exC).

Structure. provision is an environment refinement (`provision_envRefines`): each target keeps its identity and interface and goes from unresolved to realized, r is new, nothing else moves. The provisioned design is globally well formed (`provision_wf`) from the abstract design’s well-formedness, monotone evidence, the provision’s preconditions — and evidence for each target’s *commitments* on its new realization, a hypothesis the proposal did not state: a Source’s commitments are obligations on the profile. Causality is preserved with the rank shifted by one and r at the bottom; purity keeps the channel term edge-free (`provision_causal`). The domain judgment is preserved with $K \ r = K \ s$ (`provision_wellClocked`); no device clock is introduced, and a device with its own rate is a later `sync`.

Transparency. For every schedule, domain, tick, term that does not mention r , and local environment whose closures avoid r ,

$$\text{MEv } S \ \Delta \ I \ \text{ctpev} \Leftrightarrow \text{MEv } S \ (\text{provision } \Delta \ P) \ I' \ \text{ctpev}, \quad I = \text{induced}(\Delta, P, I'),$$

where the induced input gives each target the wrapped transfer of the raw reading and leaves every other identity as I' gives it (`provision_transparent`). The hypotheses the proposal lacked are stated: the raw input is typed at r and closure-free, and the abstract design mentions no r — true of every globally well-typed design (`NoMention_of_globalWF`). The observation boundary has two equivalent forms: syntactic ($r \notin e.\text{refs}$) and by typing (a term typed in the abstract design cannot name r , `provision_transparent_typed`). The proof is one simulation lemma over MEv

with the closure invariant “no closure body mentions r ”, instantiated in both directions and once more for input congruence; the provisioned target’s value comes from the pure term’s canonical evaluation transported to any design, input, domain and tick (`Transduces.mev, MEV.of_ev_pure`) — at top level, where a realization is evaluated, which is what avoids a closure-equivalence theorem. Physical outputs are unchanged (`provision_physicalOutput`).

Trace abstraction, exactness, strictness. Every behaviour of the provisioned design under a raw input is a behaviour of the abstract design under the induced input (`provision_abstracts`): deployment *restricts* the abstract environment; it does not give the abstract design its meaning. The proposal’s converse — “when the transducer is surjective the trace sets are equal” — is wrong as stated: pointwise surjectivity is an existence per tick, and with a shared raw reading it is insufficient even in principle (`id` and `succ` from one reading are each onto, and the abstract pair (5, 9) has no witness, `no_joint_witness`). Equality needs a *joint section*, a raw trace every channel transfers to what the abstract input gives its target (`provision_exact`); for one channel a pointwise right inverse on typed values is one (`JointSection.one`). The strict case is executed: a saturating ADC never yields 451 K, the abstract design observes `TempSensor = 451 K`, and no provisioned deployment does (`exE`).

Re-application and commutation. The proposal called provision “idempotent per Source”. After provision a target is realized, is no longer a Source, and the operation’s precondition fails because r is no longer fresh (`provision_not_reapplicable`; the Source role moves to r , `provision_source_role`). The totalized function does satisfy $P(P(\Delta)) = P(\Delta)$ (`provision_idem_total`) — only because a second pass overwrites every target with the same body; a second pass with a different term is not a refinement (`provision_reprovision_not_refinement`). Independent provisions commute *exactly*, as environment equality (`provision_comm`), and the channel assignment is a set (`provision_perm`).

Executed (`Experiments/ProvisionExamples.lean`): the identity GPIO channel on a `bool` Source; the thermistor $T = 2n + 250$ K on `TempSensor`, with the consumer `tooHot` computing the same truth values from counts as from the induced temperature and the provisioned design proved refining, causal and well clocked; rejected profiles; the impure typed term; the saturating ADC; one IMU image provisioning `pitch` and `roll` with `level` reading both.

What changed in the proposal (design recommendation): the purity condition; the transfer function beside the term; the commitment hypothesis; the transparency hypotheses; the joint-section exactness; “not re-applicable” for “idempotent”; shared raw as the primitive with the singleton as its case; the terminology — *abstract Source*, *provisioned Source*, *raw declaration*, never “monomorphised”, which Part V’s rank-1 polymorphism owns; and the removal of the dependency on designer-facing °C/°F (ISS-0004): the thermistor is a linear chart on counts and the language keeps kelvin. Output provision remains the duality note the proposal made; nothing here made it free. Verdict: provision is a deployment/surface construction over existing kernel terms, not a kernel construct (FVD-0121). The proposal stays a draft, revised, for human review; nothing in production implements it.

A shared raw reading. One inertial-measurement image feeds `pitch`, `roll` and acceleration; one thermistor feeds one temperature. Phase 13 made the shared reading the primitive — a provision is *one* fresh raw declaration with its clock and an assignment of channels to target Sources — and the singleton the special case (`Provision.one`; FVD-0125), because the joint-section finding is invisible in the singleton: with one raw reading and two channels each pointwise onto its target, `id` and `succ`, the abstract pair (5, 9) has no raw witness (`no_joint_witness`, **counterexample**). The shared case is the realistic one, and stating every theorem for it and instantiating the singleton (`WF.one`, `provisionOne_transparent`, `induced_one`) costs nothing.

Where the pieces will live. Production has decided, in advance of building any of it (ADR-0032’s amendment; `relationship-roles.md` § Phase 13), what owns what when PRP-0001 is implemented: the device profile and raw type in a **device catalogue** (`hardware/`, never the Standard Library, which is an authoring catalogue of fragments); the transducer as a checked BDL formula; *Fits* in deployment analysis; the provision transformation in the compiler’s deployment pass; the raw input in the platform adapter; the induced input as a testing oracle. None of it exists at the snapshot (ISS-0016; **proposal / not implemented**).

What stays open (FVI-0020): memory in a transducer (a debouncing or filtering channel is not a function of the raw reading, and the transparency and exactness theorems are stated over functions — a stream-level theorem would be needed); a device clock, $K \ r \neq K \ s$ with a `sync` at deployment (the transport exists; the construction would add one declaration per target); how a profile’s declared range discharges a Source’s commitments, once production authors

any; output provision, the dual; whether computes is checked by the compiler or trusted from the catalogue with differential tests; and out-of-type raw readings, a validation question.

Behavior semantics and realization, once more. The separation this Part makes explicit is the one Parts X–XI depend on. *Behavior semantics* is environment-provided inputs ($I(d, \tau)$), pure internal computation (Ev/MEv) and output obligations ($DriveWF$, $CompleteOutputs$, $PhysicalOutput$). *Realization* is sensor reads, ADCs and buses on the input side, and GPIO, PWM and device I/O on the output side — the platform adapter’s business, named nowhere in the kernel. A Source is not a read; a drive edge is not a write. Provision is the theorem that the input side of that separation can be crossed by a construction the design cannot observe; the output side has no such construction yet.

Two remarks across Parts III–IX

Why the signature is a design object

The most consequential decision in BDL is treating a declared relationship as visible product intent. In programming, a signature is often documentation and a static contract around code. In BDL it can precede any code-like definition and remain useful on its own: $?f : \text{Tilt} \rightarrow \text{Brightness}$ says that the designer has committed to a causal design relationship and to its semantic boundary, and does not say how the mapping is computed. Clients are typed against the type view and depend on the interface only through it and through monotone evidence, so the partial commitment is not a weaker form of a complete one. It is the form on which everything downstream already rests.

Nominal identity in three places

BDL makes the same choice three times. On the axis of quantity, `sem` makes semantic identity nominal: two concepts of equal representation are distinct, and moving between them is a declared relationship. On the axis of time, `clockId` makes temporal identity nominal: two domains of equal rate are distinct, and moving between them is a `sync` with an initial value. On the axis of effect, `outputId` makes sink identity nominal: two sinks of equal accepted type are distinct, and driving one is an explicit edge. In each case the identity is independent of representation, of rate, and of type respectively; in each case the crossing is a visible artifact rather than a compiler action; and in each case the alternative — identity by representation, by rate, or by type — collided or was ambiguous for a mechanized reason. The three are not all placed alike. Semantic identity is in the type, while clock and sink identity are projections beside the interface checked by separate global judgments, because putting the clock in the type forces polymorphism on every pure mapping. Symmetry was not a design goal; it is what remained.

Part X — Validation and Deployment

A design that is typed, causal, clock-consistent and output-complete has three more questions to answer before it runs on a product, and none of them is a question about the design alone. Does it fit the chosen board? Do its collections fit the memory and the schedule? And, once it is compiled, does the code that runs behave as the model says? The formal development answers the first two with decidable validation layers that never touch the kernel, and leaves the third as a stated gap held closed by testing. This Part is those three answers and the vocabulary that keeps them apart.

Three verdicts that must not be confused

verdict	what it says	depends on	re-established when	who states it
behavior-valid	every realization satisfies its interface; the instantaneous graph is acyclic; every reference respects domains	the design	a refinement preserves it; an edit reopens dependents	<code>analyze(snapshot)</code> ; the kernel judgments of Parts III–VII
executable	behavior-valid, and every required output has exactly one driver	the design and its required outputs	any drive or output change	<code>CompleteOutputs</code> , <code>SingleDriver</code> ; <code>bdl-output</code>
deployable	executable, and the target carries every derived requirement, and every collection has a sufficient bound under the schedule	the design <i>and</i> a target and a schedule	every design change, every board change	<code>analyze_deployment(snapshot, target)</code> ; <code>bdl-hardware</code> , <code>bdl-reactive::capacity</code> , <code>bdl-exec-ir::bounds</code>

The first two are monotone in the sense of Part III — a refinement preserves them — and the third is not: six actuators fit a Nano and a seventh, a monotone extension of the design, does not; a fast source added to a slow consumer can exceed a window bound that was sufficient. Deployability is evidence about a *pair*, re-solved from scratch, and never merged with the evidence that survives refinement. Production reports the two in different places — the status line and the inspector for the first two, the Deploy page for the third — and words them differently, so that a valid design is never mistaken for a deployable one or the reverse (ADR-0006, ADR-0015 (FVD-0057, FVD-0062)).

Target-specific hardware validation

Everything to this point is board-independent. A design that is typed, causal, clock-consistent, and output-complete may still not fit the microcontroller it is to run on, and that question is answered by a validation layer that never touches the kernel.

Resources, capabilities, requirements

A target is a finite table. A **resource** — a pin — has an identity, a list of **capabilities** from a shared vocabulary (digital in/out, PWM, analog in, interrupt, the I2C, SPI, and UART lines), and, per capability, the **unit** backing it, such as the timer behind a PWM pin or the peripheral behind a bus line. A **hardware description** is a list of resources with a per-capability sharing policy: bus lines are shareable, everything else is exclusive.

A design’s needs are **requirements**. Each has an identity, one capability, an optional fixed resource for a manual pin choice, and an optional membership in a group with a unit relation, *same* or *distinct*:

$$\text{Requirement} = \langle id, cap, \\ \text{fixed} : \text{Option ResourceId}, \\ \text{group} : \text{Option } (\mathbb{N} \times \text{UnitRel}) \rangle.$$

Requirements are generated from the physical sinks of the previous section by a **device kind**: an H-bridge channel needs a PWM line and a digital output; an I2C sensor needs SDA and SCL on the same unit; a quadrature encoder needs two interrupt lines. The pipeline is

OutputId $\rightarrow$ DeviceKind $\rightarrow$ Requirements $\rightarrow$ solve $\rightarrow$ Assignment,

and the sink never enumerates pins, so swapping the board is re-solving the same requirements with the design untouched. Requirement identity is independent of sink and declaration identity: one sink generates several requirements, and two identical PWM needs must be two distinct variables.

An **assignment** is a list of requirement–resource pairs. It is **partially valid** when every entry is supported — the resource has the capability and any fixed choice is respected — and every pair is **compatible**: two entries on the same resource must have the same capability and it must be shareable, and two entries in the same group must have equal units under *same* and different units under *distinct*. It is **valid for** a requirement list when partially valid and covering exactly that list; the target is **satisfiable** when such an assignment exists.

A sound and complete solver

Every constraint is unary or binary, so validity is prefix-closed, and an exhaustive depth-first search that prunes on unary support and pairwise compatibility with the current prefix is complete as well as sound. Both are proved: if the solver returns an assignment it is valid for the requirements, and if a valid assignment exists the solver returns one. Feasibility of a finite instance is therefore decidable, and the solver runs inside the proof checker by decide. The instance is small enough that a general constraint solver [7] is unnecessary; the claim is about this scope, not about embedded allocation in general.

Extension of a target — adding capabilities, units, or sharing to existing resources — preserves every valid assignment. Removing a resource, adding or strengthening a requirement, or fixing a pin may not.

Case study: an Arduino Nano

The Arduino Nano’s digital and analog pins are encoded as a table: PWM on D3, D5, D6, D9, D10, and D11 backed by timers 2, 0, 0, 1, 1, and 2; external interrupts on D2 and D3; I2C on A4 and A5. A design with four H-bridge motor channels and one I2C inertial sensor is satisfiable, and the solver’s output is the assignment a tool would present:

```
M1 -> D3 / D0      M2 -> D5 / D1
M3 -> D6 / D2      M4 -> D9 / D4
IMU -> A4 / A5
```

Two counterexamples fix the boundary between kernel and validation. Seven independent PWM actuators pass every kernel condition — globally well formed, well clocked, causal, every drive edge well formed, single-driver, output-complete — and are unsatisfiable on the Nano, which has six PWM pins. The same requirements are satisfiable on a larger mock board with six more PWM pins on three more timers. Two interrupt lines and six PWM lines meet every capability *count* exactly, two and six, and are unsatisfiable, because D3 is both the only second interrupt pin and one of the six PWM pins. Capability counting is not feasibility. Further examples establish that two PWM requirements pinned to the same pin are rejected, that two I2C sensors on A4/A5 are accepted since allocation is not all-different, that a manual pin choice can turn a satisfiable design unsatisfiable while a consistent one is honoured, that four PWM lines required on independent timers are unsatisfiable on the Nano’s three timers though six PWM pins exist, and that TX and RX pinned to one UART unit stay together.

The explanation facility reports the first dead end under greedy placement. For the seven-actuator design it names the seventh actuator and, for each PWM pin, the actuator blocking it. This is *a* conflict under one placement order, not a minimal unsatisfiable core, and it is meaningful only when the solver has already returned no assignment.

Two kinds of evidence

Hardware feasibility is evidence about the pair (design, target), and it is not monotone in the sense the preservation theorems require. Six actuators are satisfiable; adding a seventh — a monotone extension of the design by a declaration and its sink — is not. The distinction drawn earlier between evidence that survives refinement and evidence that is rechecked after every change is here concrete. Commitments discharged compositionally survive EnvRefines; deployability on a target is re-solved; and the two are never merged. This is why the workspace reports them as different states.

What the layer does not model should be stated as plainly. Voltage, current, thermal budgets, memory, processor load, deadlines, bus bandwidth, torque, travel, and PWM frequency values are outside it. Some of these need summation

constraints that are not binary, and while the architecture leaves room for a compatibility predicate over sets, nothing here establishes it. Units are one integer per capability per resource, which models which timer or which UART but not timer modes.

Deployment capacity

The third kind of validation evidence, added in Phase 9a, is deployment capacity: whether every bounded collection in the design has a bound sufficient for the target’s memory and the schedule. Like feasibility and unlike a commitment, it is not monotone — adding a fast source to a slow consumer can exceed a bound that was sufficient — and it is re-established after every change. Part VII gives the formal result (*sufficient_capacity_preserves*, *periodic_capacity_sufficient*; **formally proved**) and production’s realization (*bd1-exec-ir::bounds*, refusal at deploy; ADR-0027). The workspace state *hardware-feasible* of Part II is, in production, the conjunction of allocation and capacity.

Provision as deployment architecture

Part IX’s provision is a deployment construction: it takes the executable design and a device profile per Source and yields another design, provably an environment refinement of the first, in which every abstract Source is a Value computed from a raw declaration. In the deployment picture it sits between *executable* and *deployable*: the provisioned design is what the platform adapter’s input slots feed and what the requirement generator sees, and *Fits* — the concept’s representation is the channel’s — is a deployment check beside allocation and capacity. Nothing of this is built at the snapshot (PRP-0001, ISS-0016); the placement is recorded so that when it is built the transducer is a checked term in the deployment pass and not host code in the adapter.

The generated-code boundary

Below deployability is one more boundary, and it is the weakest link in the trust chain. The formal semantics is a tick-indexed relation; production’s reference evaluator is its executable transcription; the generated *no_std* core (Part XI) is an implementation of that evaluator which lowers representation — dense slots, static structs, *u64* fields, inlined lambdas — and may not change meaning. The core is held to the evaluator by a 22-case differential corpus, golden files, a determinism check, property-generated designs and an error-tick agreement check (**production implemented and tested**). No theorem relates the generated code to *Ev/MEv*, and none relates the static list-bound analysis to the capacity theorems: a generated-code refinement proof and a proved bound analysis are the open items that would close the gap (Part XV). Differential tests are evidence for the corpus; they are not a proof, and this document never infers formal correctness of generated code from them.

Open hardware questions

The validation layer covers discrete pin and peripheral allocation with unary and binary constraints, and the explanation facility reports a first dead end under one placement order. What it does not cover is stated once, in Part XV: minimal unsatisfiable cores; voltage, current, thermal and timing budgets, which need summation constraints that are not binary; timer modes and PWM frequency values; the device catalogue that provision presupposes; and any board file beyond the Nano and the larger mock board (an RP2040 file, runtime loading of *hardware/boards/*, and the first platform adapter are production’s roadmap priorities, not records of anything that exists).

Part XI — Production Compiler, Runtime and Daemon

KCN-judu/BDL is the engineering implementation of the language whose kernel was derived in BDL_FV. It builds what the formal development deliberately did not: the elaborator, the tooling and the execution path. It follows the formally developed semantics and is not itself formally verified. One property the whole system is built to keep obvious: *BDL semantics flows downward; implementation mechanisms never flow upward and redefine the language*. This Part describes the architecture at the production snapshot (de8154f, Appendix F) in enough detail to be a reference for it.

Four trust layers

layer	owns	never does
Flutter Studio	presentation, interaction, layout, ephemeral render state	compute type validity, semantic identity, dimensions, causality, clocks, output ownership, hardware feasibility, simulation
Rust compiler (bdl and crates)	the canonical project model, every semantic judgment, diagnostics, simulation, allocation, code generation, the placement of entities that have no position yet	render, decide where a placed node goes
generated Rust core	deterministic executable behavior: domain step functions, state, output values	touch hardware, know about tasks or executors
platform adapter	physical I/O, clock activation sources, telemetry transport	interpret BDL semantics

Studio may render an edit optimistically, but the truth comes back from the compiler as a *projection*; Studio never holds a second copy of the language (ADR-0001). bdl is a process boundary (ADR-0002), speaking protobuf over framed stdio (ADR-0007), with one canonical revision stream per opened project.

Crates and semantic ownership

The dependency direction is strict and acyclic: `model` $\rightarrow$ `ir` $\rightarrow$ `{syntax` $\rightarrow$ `elab`, `check` $\rightarrow$ `equations` $\rightarrow$ `elab`, `check` $\rightarrow$ `reactive` $\rightarrow$ `output`} $\rightarrow$ `compiler` $\rightarrow$ `ide-db` $\rightarrow$ `ide` $\rightarrow$ `{lsp`, `daemon`}; `protocol` sits between `compiler` and `daemon`; `text` and `layout` hang off `system` and are joined by `daemon`, `lsp` and the CLI; `hardware` depends on `model` only — it never sees Δ — and `compiler` joins the two; `lower` $\rightarrow$ `codegen` hang off `exec-ir`; `runtime-core` depends on nothing and is what generated code links against. A crate exists only where a real boundary exists.

crate	owns
bdl-model	stable ids · surface model · revisioned edits (<code>apply_edit</code> , <code>pure</code>) · persistence · quantity vocabulary · the derived relationship role (<code>RelationshipRole</code> , <code>MappingBlock::role</code>)
bdl-ir	the Design IR and the Reactive Core IR: the kernel's <code>Ty</code> , <code>Expr</code> , <code>Prim</code> , <code>environments</code> , <code>transcribed</code>
bdl-diagnostics	<code>Diagnostic</code> , <code>Span</code> , stable codes, deterministic order
bdl-syntax	Logos lexer · event parser (recursive descent + Pratt) · Rowan lossless CST · typed AST · lowering
bdl-equations	the equation library: rank-1 schemes with type and dimension variables and <code>{Data, Eq, Ord}</code> , first-order matching, one closed Core builder per equation
bdl-elab	<code>concepts</code> $\rightarrow$ Θ · <code>signatures</code> $\rightarrow$ <code>interfaces</code> · <code>formulas</code> $\rightarrow$ Core with equations inlined at their instance · the unit registry and charts
bdl-check	Core typing · <code>Grant</code> · <code>realization</code> vs <code>interface</code> · pretty-printing; the authority for every type judgment
bdl-reactive	dependency graph · <code>causality</code> · <code>Clocked</code> · the reference evaluator · simulation · window capacity
bdl-output	<code>DriveWF</code> · <code>SingleDriver</code> · <code>CompleteOutputs</code> · <code>output_values</code>

crate	owns
bdl-hardware	capabilities · resources · hardware · device → requirements · boards · solve/diagnose
bdl-exec-ir	the executable IR: slots, first-order expressions, evaluation plan; interpreter; static list bounds
bdl-lower	reactive lowering: Design IR → Exec IR (clock, state, input and output slots; inlining; order)
bdl-codegen-rust	Exec IR → owned Rust AST → printed crate + host bridge + bdl-manifest.json
bdl-compiler	analyze(snapshot), analyze_deployment(snapshot, target), compile(snapshot, options), the collections report
bdl-system	components · instances · bindings · freshening · flatten → ProjectSnapshot + origins · packaging
bdl-text	persistence: source discovery · identity sidecar and reconciliation · load_workspace · item-level write-back · legacy migration
bdl-layout	deterministic, incremental placement of entities without a position; never semantics
bdl-library	the Standard Library: items whose fragments instantiate ordinary concepts and Sources through the ordinary edits; search; a multi-library set
bdl-ide-db	IDE ground state: IdeHost · overlays · EntityRef/EntityRole · projections · immutable stamped AnalysisSnapshot · cancellation
bdl-ide	semantic IDE queries over a snapshot: diagnostics, hover/explain, completion, references, rename, actions (incl. rule.apply), invalidation preview, symbols, the one token classifier (tokens), the Formula Composer queries
bdl-lsp	an LSP adapter only (ADR-0017)
bdl-protocol	protobuf schema · framing · conversions
bdl-daemon	bdld: session, coordinator, transport, analysis push, layout on open and commit; bdld check compile simulate
bdl-runtime-core	no_std vocabulary of every generated core: ActiveDomains, ClockSlot, RuntimeError, checked numerics; feature collections: list operators and the recursor over alloc::Vec
bdl-runtime-host	std harness: DynValue, JSON run request/trace over stdio, cargo driver

Planned and designed but not implemented: bdl-component (supplied Rust component contracts; “supplied Rust cannot drive outputs”, ADR-0005) and bdl-runtime-embassy (the first platform adapter).

The compiler as a pipeline of explicit passes

load/parse → identity resolution → signature resolution → surface elaboration
→ type checking → semantic-construction (grant) checking → dimension checking
→ dependency analysis → causality → clock domains → physical outputs
→ hardware requirement generation → hardware allocation → reactive lowering
→ Rust code generation

Each pass has an explicit input and output type and is pure where practical (docs/architecture/compiler-pipeline.md). Diagnostics are first-class outputs: an incomplete project is the normal case, never a fail-fast, which is the engineering face of the signature-first position. apply_edit classifies every operation as a *refinement* (dependents’ established facts remain valid) or an *edit* (dependents must be re-validated) and reports an explicit Invalidation set — Interface | Realization | Semantic | Reactive | Clock | Output | Deployment — with the originating declarations (ADR-0009); incremental analysis subscribes to these categories, so the refinement-versus-edit distinction of Part III is an engineering asset, not folklore.

Numerics are IEEE f64 (ADR-0011), a recorded deviation from the kernel’s Nat: division by an exact zero and any non-finite result fail the tick with a structured error, equality is exact, and the production numerics are not what the formal theorems are about (Part XIII).

Formula elaboration (ADR-0013, extended by ADR-0025): a designer-facing formula language `— + - * / < <= > >= == != && || !, if, match` over `Bool/Option/numbers/counts`, `let`, `calls`, `list` and `pair literals`, rules `x => ...` as equation arguments, membership `x in [...]`, the slot `?` — parsed by a hand-written Pratt parser into a surface AST that is never reused as `Core`. Input names are the concepts' display names, re-resolved on every analysis; inputs appear as their representations (`rep (var i)`) and the whole formula is wrapped in `mk B` under the declaration's own grant — the elaborator never emits `mk` of any other concept, and the checker refuses one anyway. Units elaborate through the registry; equations are matched and inlined at the use; `bd1-check` re-derives the `Core` term's type and is the authority.

Executable IR and lowering

`bd1-exec-ir` is a first-order IR with dense slots: clock slots, state cells addressed by `StateCellId { decl, path }` (the declaration and the expression path of the delay/sync inside its realization, so identity survives unrelated edits), input slots for unresolved declarations, output plans with their driver. Lambdas and applications are inlined as `let` bindings; the recursor becomes `Fold { elem, acc, step, init, list }` with one closure per recursor applied by the runtime — never a closure *value*; `rep/mk` become `Unwrap/Wrap` on a per-concept newtype. Lowering fixes an evaluation order from the plan; the reference evaluator's own tests show its result is independent of the host's processing order and the generated program's traces equal the reference's.

The generated core

```
pub const DESIGN: &str; pub const HAS_DOMAINS: bool; pub const CLOCK_COUNT: u16;
pub const CLOCK_k: ClockSlot;
pub struct SemN(pub Repr); // one per concept carried
pub struct Cells { pub cell_k: Option<T>, ... } // temporal state, None until first written
pub struct State { pub cells: Cells }
pub struct Inputs { pub decl_n: Option<T>, ... } // unresolved declarations
pub struct Values { pub decl_n: Option<T>, ... } // every declaration, None when not due
pub struct Outputs { pub output_n: Option<T>, ... }
pub struct Tick { pub values: Values, pub outputs: Outputs }
pub fn init() -> State;
pub fn step(state: &mut State, active: ActiveDomains, inputs: &Inputs) -> Result<Tick, RuntimeError>;
```

A program without lists is `Copy`, statically sized, laid out by the compiler: no graph, no map, no allocation, no traversal at runtime. Types: `q d → f64` (the dimension is static and in the manifest), `bool, nat → u64`, `sem s → SemN`, `opt τ → Option<T>`, `list τ → Vec<T>` (last element first), `τ × σ → (T, S)`; a function type has no runtime representation. Symbols derive from stable ids (`decl_17`, `Sem3`, `cell_0`), never from display names. The core is `#![no_std] !#[forbid(unsafe_code)]` and mentions no `HAL`, `pin`, `peripheral` or `board`; `cargo check --lib` of every corpus crate is a test.

Embedded execution model

The formal semantics is a tick-indexed relation; production gives it a deterministic step function per domain and states what the generated core must preserve (`docs/spec/runtime-semantics.md`):

- **Declarations are not tasks** (ADR-0004). Never one `async` task per declaration. Per clock domain, one deterministic `step(prev_state, inputs) → (next_state, outputs)`: hardware timer or interrupt → activate domain `c → step → publish c's snapshot → commit the physical outputs owned by c`. Meaning never depends on executor task order.
- **Two phases**. *Read*: every declaration due this tick is evaluated (lazily, memoized) with temporal forms yielding their cell's committed value, or the initial value if the cell was never written. *Write*: every temporal site whose writing domain is active — the owner's domain for `delay`, `src` for `sync` — evaluates its operand in the same read mode into the *next* state. Nothing is updated in place; reads never see writes of the same tick.
- **Previous/next state**. `step` reads only `prev`, writes only `next`, and assigns at the end — not at all on error. Every `delay` carries its explicit initial value; a cell is `Option<T>`, `None` until first written; there is no implicit zero.
- **The sync snapshot rule**. `sync src init e` reads the last committed snapshot of `src` from an activation strictly before the current tick, `init` if there was none, and never invokes `step_src` recursively. When two domains are ready at the same instant, each observes only the other's previously committed activation — the scheduler's order is unobservable, exactly as in the formal model (`scheduling_order_observable` is the counterexample for the alternative). In generated code a `sync` cell's writer is the *source* domain's slot.
- **Output commit timing**. Outputs are built after the write phase from the driving declarations' values (`output_values`), and committed by the domain that owns the sink.

- **Allocator requirements.** A core that carries a list needs a global allocator (ADR-0024); the manifest records requires_allocator, per-cell bounds, state_bytes_max and tick_bytes_max; the first platform adapter is to declare an arena sized from the manifest and state the most it supplies for each list input.
- **Collection bounds.** Static, sound per declaration and cell (bdl-exec-ir::bounds); the window capacity model per crossing; refusal of an unbounded state on a bounded-memory target (ADR-0027, Part VII).
- **Cost discipline.** Moves, not clones: a use analysis per expression tree moves a local referenced once in its scope, clones any other reference, and always clones a local captured by a fold's closure, so a fold step that conses onto its accumulator moves it — map, filter, append, sum, any, all, contains clone nothing per element (clone-counting tests; measured $\times 15$ for 2 000 $\rightarrow$ 32 000 elements). A total conditional — neither branch can fail — is emitted as a Rust if, observationally the strict ite; a branch that can fail keeps prim::ite and its strictness. Borrowed reads for length, head, take, ==, <. Commit-only writes: a tick clones no cell it does not read. What remains: zip clones its accumulator pair per element (ISS-0013); no fusion of map $\rightarrow$ filter chains, because each is linear and measured and a fused emission was not justified by the numbers.
- **The realization boundary.** A source (an unresolved unit-domain declaration) is an *input slot* of step, filled by the adapter from a sensor, a bus or a simulation trace; a driven output is an *output field* of the step's result, committed by the adapter to GPIO or PWM. Neither is a function call inside the core: a zero-input relationship compiles to a zero-argument accessor of the committed value (its unit argument erased, ADR-0029), and a sink is a field, not an A $\rightarrow$ () callback (Part IX). The core therefore has no device vocabulary at all.
- **Targets.** The core is target-independent. macOS and Windows are first-class hosts for the toolchain; no platform adapter exists at the snapshot — Embassy is roadmap priority 1; an RP2040 board file is not yet present.

Current cost figures are recorded, not optimized: a lamp core is ~95 lines and ~7 KB of source with a zero-byte State; the collections and buffer corpus cases allocate and their debug-build timings are two orders of magnitude above the allocation-free cases — a baseline for later evaluation, nothing more.

Differential testing

For the same design, schedule and input trace, the reference evaluator and the generated program must agree value by value, tick by tick (crates/bdl-compiler/tests/backend_*.rs). The corpus: lamp and lamp-with-output, pure arithmetic with dimensions and a count, collections (any, sum, map, filter, zip, contains, clamp, getOrElse/head, structural equality, a list in a state cell), buffer (the Phase-9a window as five declarations), bounded_buffer and overflowing_buffer, semantic rep/mk with a Boolean concept, booleans/comparisons/strict if/options, delay, a cycle broken by delay, sync across two domains in both directions with the slow domain on period 2, a domain-agnostic declaration shared by two domains, division by zero, non-finite result, missing input, a design with no domain — 22 cases, each generated, cargo checked as a no_std library, built with its bridge, run and compared, with golden files pinning the generated bytes and a determinism check that generating twice gives identical output; a 3 000-tick run; property-based generation of designs (250 per run in process, a seeded batch of 8 through the full toolchain); and the storage-order audit of the reversed Vec. Comparison is exact (f64 bit-for-bit in effect; the generated code performs the same IEEE operations in the same order — no reassociation, fusion or CSE — and the JSON transport is exact); errors must fail at the same tick with a failure the reference could report, computed by starting the reference's traversal at every due declaration.

Project persistence and the daemon

A project is one directory (ADR-0023, superseding ADR-0020's third project *kind*): bdl.toml (name, schema, compiler version — no kind), src/**/*.*.bdl (the authored design and system — the semantic source), .bdl/identities.json (source key $\rightarrow$ stable id, allocators, the flat-id freshening table — tool-owned), .bdl/authoring.json (behavior groups by identity), ui/layout.json (positions, viewports, group boxes — presentation). “Source of truth” is a per-fact statement: the semantic model never depends on geometry, the graph never stores a semantic fact, the text never stores a position; erasing both sidecars and the layout changes no semantic fact; erasing the sources loses the design. A legacy JSON project is migrated in place the first time it is opened, identities and layout kept.

bdld's coordinator is the single imperative loop: it owns the Session, applies edits serially through the pure apply_edit, and hands immutable snapshots to analyses. Every semantic edit is sent against the revision Studio holds and refused if the project has moved on; every response and event carries its revision; revisions are strictly monotone (undo produces a new revision), so staleness is a < comparison. Overlays — a definition draft per mapping, a text

document per file — never create a revision; `AnalysisSnapshot` composes committed + overlays, runs `analyze`, and is stamped (revision, generation); a query holding an old snapshot keeps seeing the world it started in.

The IDE service, the daemon and the protocol

`Studio` (visual) and text editors (textual) are two projections of one semantic model and consume one service. `bdl-ide-db` holds the ground state — the committed snapshot plus overlays for unsaved drafts and text buffers — and produces immutable, stamped `AnalysisSnapshots`; `bdl-ide` answers diagnostics, hover and `Explain`, scope-aware completion, definition/references/rename across files, semantic actions and edit plans, invalidation previews, symbols, semantic tokens and the `Formula Composer`’s three queries, all in BDL-owned types keyed by `EntityRef` and role — never by text position or node id; `bdl-lsp` and `bdld` are thin adapters at the edge (ADR-0017). The daemon speaks a typed, versioned protobuf protocol over framed `stdio` (ADR-0002, ADR-0007). Every semantic edit is sent against the revision the client holds and refused if the project has moved on; every response carries its revision; revisions are strictly monotone, so staleness is a `<` comparison (ADR-0009). Protocol versions are additive within a minor series and volatile: the version at the snapshot and what each recent minor added are in Appendix F, not here.

Incremental invalidation. `apply_edit` classifies every operation as a refinement or an edit and reports an `Invalidation set` — `Interface` | `Realization` | `Semantic` | `Reactive` | `Clock` | `Output` | `Deployment` — with the originating declarations. That is the refinement-versus-edit distinction of Part III as an engineering asset: what is re-analysed after an edit is decided by a model of which established facts an edit can disturb, not by folklore. A proved per-category preservation theorem does not exist (Part XV).

One classifier for text. Highlighting looks lexical and is not: whether `tilt` is a `Source` or a `Value`, whether `deg` is a unit or a parameter, whether `all` is a binder word here, whether `clamp` is the library’s — each is a fact of the snapshot, the same fact that drives hover, `Explain`, completion and the canvas. So there is one classifier, `bdl-ide::tokens`: a lexical layer from the lossless syntax tree (total on any text, so a file mid-edit keeps its reading) merged with a semantic layer from the snapshot’s projections and the derived role, into one sorted, disjoint stream of byte-range tokens. Its vocabulary is the LSP semantic-token vocabulary — the standard types and modifiers, themed by every LSP editor without configuration — plus two BDL additions, `unit` and `slot`, with BDL’s distinctions expressed as *modifiers* on standard types (a `Source` is `variable` + `source`), under a versioned legend. The language server encodes the stream as `textDocument/semanticTokens` in the client’s position encoding through a pure encoder for UTF-8, UTF-16 and UTF-32; the daemon sends the same stream structured to `Studio`; conversion between the two is lossless and lives in `bdl-ide`. No Dart, TypeScript or grammar file decides a class; no colour crosses a boundary (ADR-0035). The rule is the one ADR-0001 sets for semantics generally — Rust classifies, clients render — applied to the one place a client would most naturally have kept its own copy of the language.

The elaboration passes the kernel implies

The surface editor and kernel are connected by an elaboration function from surface designs to kernel environments with diagnostics. These passes were stated in the formal development before any elaborator existed; the production pipeline above realizes them — resolution and formula elaboration in `bdl-elab`, temporal lowering and domain assignment in `bdl-lower` and `bdl-reactive`, output binding in `bdl-output`, hardware validation in `bdl-hardware`, normalization and erasure in `bdl-exec-ir` and `bdl-codegen-rust`. The list is kept because it is the specification those crates were written to, and because it says what each pass may and may not consult.

Name and signature resolution. Resolve concepts to identities, Mapping signatures to declaration interfaces, contexts, device kinds, units, and imported components. At this stage an unresolved `Tilt` -> `Brightness` mapping already has a stable identity and an expected type, and every reference to it is typed.

Formula elaboration. For each Mapping with a definition, check the definition against the declared codomain under the grant of the declaration’s own signature. A scalar formula attached to a `Brightness` output elaborates to `mkBrightness(...)` because the surrounding signature announces the concept; the constructor is inserted by the elaborator and is legal only there. Units elaborate to scaled dimensioned literals. `Curve`, `example`, and component definitions elaborate to the same realization form.

Temporal lowering. Surface temporal phrases lower to the declaration shapes of the derived-operator table. A context lowers to an activation declaration, an entry declaration, gated and reset local state, and a conditional in each

output’s driver. A reusable stateful component is instantiated into fresh declarations at each use, because temporal state cannot occur under a binder.

Domain assignment. Domains are declared, so this pass records the clock environment and checks the domain judgment. A reference across domains without a transport is reported at the reference, in the designer’s names for the two domains, with the two legitimate resolutions — transport the value with a stated initial value, or move the reader to the source’s domain — stated in terms of their behavioral consequence.

Output binding. Record the drive edges and check type and clock equality, single-driver, and, for executable designs, completeness. Where several contexts would drive one output, the elaborator emits one driver whose body selects among them; the selection is visible in the elaborated design.

Hardware validation. Generate requirements from device kinds, run the solver against the selected target, and attach the assignment or the explanation to the bindings.

Normalization and erasure. After checking is complete, semantic identities, dimensions, and domains carry no computational content and may be erased; erasure is proved sound for identities and for dimensions. Nominal wrappers introduced by elaboration cancel, $\text{rep}(\text{mk}_s e) \rightsquigarrow e$, and the flattened program is checked under the universal grant because every construction was authorized at its declaration. Unfolding all realizations into a closed term preserves typing on the delay-free fragment and agrees with tick-by-tick evaluation on first-order designs; the higher-order case holds up to closure equivalence and was not formalized. Erasure is applied selectively at boundaries the checker cannot see through — supplied blocks, device bindings, the public interface of generated code — where wrappers are retained so that the host compiler continues to check what BDL cannot.

The kernel never depends on normalization to decide type equality; it is not dependently typed, and normalization is an analysis and code-generation instrument. Floating-point arithmetic is not associative, so any symbolic normalization over the reals must record the resulting numerical deviation as an obligation rather than silently altering the property being checked.

Part XII — Studio and the IDE: The User Interface as a Semantic Projection

Studio is not a drawing tool with a compiler attached; it is a set of projections of the compiler’s verdicts, arranged so that semantic distinctions become interaction distinctions and nothing else does. This Part describes the interaction model as designed and, at the snapshot, largely built (docs/architecture/studio-ui.md; what remains is in Appendix F). Whether the model is *usable* is an empirical question no study has yet answered (Part XV).

Three information levels

Every semantic fact is designed for exactly one primary level and may echo at the next only as detail behind the first (ADR-0018):

level	answers	may use	may not use
1 Canvas	what does the product do; what depends on what; what is still open; where does behavior become physical	object silhouette, socket shape, socket hue, links, grouping, containment, line style, the object’s own state; one state word where unavoidable	type labels, ids, compiler words, badges, counts
2 Inspector	what does the selected object mean; what can I change; what will the change affect	designer vocabulary: <i>Meaning</i> , <i>Value</i> , <i>Unit</i> , <i>Reads</i> , <i>Produces</i> , <i>Relationship</i> , <i>Used by</i> , <i>affects</i> , <i>checked again</i> ; diagnostics in product language attached to the field they concern	SemanticId, DeclId, Ty, Grant, realization, Clocked, SingleDriver, solver, protocol, revision, enum names
3 Explain	why was this accepted or refused; what did the surface form elaborate into; which rule applies	all of the above, kernel notation, Core IR, diagnostic codes, revision	—

Explain is one collapsed disclosure at the end of the inspector, never open by default; nothing in levels 1–2 depends on it. This is the progressive-disclosure claim of Part I made operational: the kernel is reachable, and it is never in the way.

The canvas

Socket hue = semantic identity. In a node editor a socket’s colour is its data type; in BDL the type that matters is the nominal concept, so each concept gets a stable hue derived from its `SemanticId` — deterministic, never from the name, with lightness chosen per hue so every identity clears 3:1 contrast in both appearances. A link is accepted only between sockets of the same concept: while a link is dragged every compatible socket gains a faint halo, the one under the pointer a strong halo, and an incompatible socket shows the forbidden cursor. The canvas shows the nominal typing rule without a diagnostic. Tilt and `MotorAngle`, both angles, have two hues and never connect.

Socket shape = value form. Quantity ○, on-off ◇, count □, collection  (a stack), grouped value  (a split square), optional value ◯ (a ring with a hole); a concept whose value form is not yet chosen is a hollow ring. The same glyph, drawn by the same code, appears in the library, in chips, toggles and pop-ups. No type words are written beside a socket anywhere.

Unresolved state. A declared rule — realization *none* with inputs — is drawn with a dashed outline, an empty definition region and the one word *declared*; dashed survives selection; never red. A Source — realization *none* on the unit domain — is *not* dashed: it is complete, and it is drawn as the next section says. A definition the compiler cannot accept gets a red mark at the definition line, where the problem lives, and no word in the header. A relationship waiting on a concept whose value form is not chosen is a solid node whose read socket is hollow, with the inspector saying *checked once Temperature’s value is decided*. These are the states of Part II’s table, drawn.

What the canvas never means. Edges are dependency, not execution order; drawing order does not set output priority; node position is layout only (ADR-0003) — an entity without a position is placed by the daemon’s layout service on open and on commit, deterministically, without moving anything placed, and Studio arranges nothing at

render time. No node type exists per arithmetic operator: formulas live in the inspector. Dragging a node over a link does not auto-insert it, because links are typed by concept and silent insertion would be a semantic edit.

Header colour = category: concept, mapping, context, output, transport — muted; identity hues are the only saturated marks on the canvas. Interaction follows Blender’s node editor conventions; a concept in use cannot be deleted without the banner naming its dependents.

Source, Rule and Value on the canvas

The three roles of Part IX are the canvas’s first distinction, drawn without colour alone. A **Source** has no input socket and one output socket, never a `()` port; its header carries the word *Source*, an entry glyph — an arrow crossing a boundary tick — and a solid bar on the node’s **left** edge, the environment side, the mirror of a sink’s bar on the right, with a green header strip as the redundant cue. It is not dashed: nothing is missing. Assistive technology hears *name, Source: a value entering the behavior model from the environment, provides C*. A **Rule** wears the word *rule*; when nothing applies it, its output socket is hollow and the header says *not applied*. A **Value** is the plain node. The inspector says what an object *is*, not what it lacks: a Source’s Meaning section has *Role: Source* with one sentence, its output section is *Provides* (a relationship’s is *Produces*), and its Relationship section opens with *Realization: provided by the environment; no device is bound yet* — never *calls, reads hardware* or *sensor*. Hover and Explain speak the formal vocabulary: *role: Source, type: () -> C, definition none — provided by the environment, observed once per activation; a resolved () -> A cites resolved_not_source*.

Edges. A **signature edge** runs socket to socket and is the interface; a **reference edge** is a thin neutral line into a relationship’s formula line for every relationship the formula names, drawn from `MappingAnalysis.references` — the kernel’s `DependsOn` by identity — never from a reading of the formula text, not editable, not a drop target (ADR-0034 (FVD-0005)). *Produces* is the signature; *carried by*, in the Simulate probe, is a value per tick. Studio consumes references and `applied_by` and never inverts or scans anything.

Semantic actions

A finding may carry a **fix**, and a fix is a semantic action the IDE service plans and the daemon applies as an ordinary edit: *Make empty domain explicit* on the legacy shorthand, *Insert explicit sync* at a cross-domain reference, and *Add a value that applies X* (`rule.apply`) on a rule nothing applies — ready when each read concept has exactly one producing value, a choice when one has several, blocked with the reason when one has none or the arguments update in two timing domains; blocked, with the instance named, for a rule inside an instance, because the value belongs in the component’s source. Actions and entity hover are requested in the system context only, on the flat entities. A fix chosen on the Simulate page selects the object, asks for its actions and applies the one of that kind when it arrives ready. The tool never guesses.

Colour in the Code view and the formula field

The Code view and the definition editor’s text field colour BDL text by the IDE service’s semantic tokens (Part XI): one request per text shown and per pause in typing, the spans of the unchanged prefix and suffix shifted while a keystroke’s answer is on its way, and never a stale span over changed text; only the latest generation’s answer lands. Studio’s `SyntaxTheme` maps the token *names* to styles in the canvas’s own category encoding; Studio classifies no BDL syntax itself (ADR-0035). This is the broader architectural rule made visible at the character level: semantic authority stays in Rust, and clients render projections.

The Library

The Library tab lists the Standard Library’s items in two sections, *Concepts* and *Sources*, with names and search localized by id through Studio’s ordinary localization pipeline. Inserting an item is one request (`InstantiateLibraryItem`) the daemon plans as a fragment of ordinary objects and applies in one transaction — every step or none, one revision, one Undo — and the canvas draws what was created exactly as it draws what was made by hand: a Source item’s relationship is a Source by the derived rule, not by a flag. The Library is an authoring catalogue; it says nothing about devices.

Groups and components on one canvas

A behavior system has three zoom levels on one canvas: the system of instances, an instance’s body, and the flat design. A group is a box that can be collapsed to its aggregate sockets (`crossIn ++ openMembers in, crossOut out` — Part

VIII), moved, merged and split with no revision and no re-check; *Package as Component* opens a sheet that shows the inferred required, provided and private members and lets the designer widen but not narrow them. An instance node shows its required and provided ports with the same socket glyphs; a component's body is edited in its own scope with its own drafts and completion. The packaging sheet teaches by showing: the boundary the designer sees on the collapsed group is exactly the interface the component receives.

Design, Code and Split: views of one project

The same open project is shown as a graphical behavior model, as its source files, or both side by side (ADR-0023). Switching is a view change — no conversion, import, export or project type. Graph → text: a canvas operation is a semantic edit on the model, and the textual projection is the minimal item-level splice of the sources, keeping comments and formatting outside the touched item; computed for the Code view on request and written on save. Text → graph: a Code-view edit is the whole text of one file against a revision; if it builds, declarations are bound to their identities by reconciliation (retained, allocated, dropped, renamed, ambiguous, duplicate) and the project moves to a new revision; if not, the committed project stays and the draft is held with the loader's faults — malformed text never erases the graph. An entity authored first in Code and one authored first in Design are indistinguishable once synchronised. The Code pane shows an out-of-step banner with the daemon's reasons and syncs selection in Split.

Ownership boundaries, stated once and relied on everywhere: source semantics in `src/**/*.bdl`; identity in `.bdl/identities.json`; authoring metadata (groups) in `.bdl/authoring.json`; layout in `ui/layout.json`; viewport and session state in the running Studio only. Studio state is three things kept apart — semantic projection (owned by the compiler, only rendered), editor state (selection, open inspector, pending requests), rendering state (drag position, hover, zoom, never persisted or sent).

The definition editor and the Formula Composer

The definition editor has two projections of one draft, chosen with a **Formula | Text** control (ADR-0028). The Text view is a field over the compiler's verdict, with completion, hover and fixes from the IDE service. The Formula view draws the compiler's `FormulaProjection` as the expression it is — `clamp( [Tilt] ÷ [90][deg ▼], [0], [1] )` — never as a graph inside the node.

Architecture. The draft text of a mapping's definition is an overlay in the IDE host (`MappingDefinitionDraft`), never a revision; `bdl-ide::formula` builds the projection from the *effective* definition text on every request and never stores it:

```
FormulaProjection  formula_projection(&snapshot, mapping)
                  = trace_formula_in (bdl-elab: the surface tree and the type of every
                    sub-expression, on success and on failure)
                  → FormulaNode tree: tree-path ids, byte ranges, kinds, actual types,
                    expected types by local dimension inference (solve),
                    diagnostics placed on the innermost node, the slots in source order

SlotInfo           formula_slot(&snapshot, mapping, node)
                  = the projection's expectation at `node` + units_for(dim)
                    + reference candidates by type + equations by scheme match and capability

ComposeResult      compose(&snapshot, mapping, source, ComposeOp)
                  = the text a structured action makes: byte-range edits of the draft,
                    parenthesised by precedence, a unit switched with the value kept
```

`FormulaTrace` is the elaborator's typing trace, kept on success and on failure, so a formula that does not check still has a projection for the parts that do. A `FormulaNode` carries its *actual* type (what the sub-expression is) and its *expected* type (what its position demands, by `solve`), and node identity is the tree path — ephemeral, stable within one draft generation, not an entity, never in `.bdl/identities.json`. `bdld` carries the projection with every `DefinitionDraftAnalysis` (one round trip per keystroke, the same generation) and answers `GetFormulaProjection` (the committed definition, no overlay), `GetFormulaSlot` and `ComposeFormula` (protocol 0.12).

Every structured action is a text edit. `compose` answers a `ComposeOp` — fill a slot, wrap in an operator, call an equation with the component as first argument, set a literal's unit, set a literal's coordinate, remove — with byte-range edits of the draft and the whole new source; Studio puts it into the draft through the ordinary `DefinitionDraftChanged`, and the ordinary analysis follows. One commit path, one conflict model, one set of diagnostics. Studio stores the mode,

the selection and the open pop-up — never a parsed tree, an inferred dimension or a unit rule; building the tree in Dart was rejected because precedence, parenthesization and unit conversion would move into Studio (ADR-0001).

Encodings. A *slot* is a dashed hollow chip (dashed = not decided, as the canvas’s *declared* node); a *reference* is a chip with its concept’s socket glyph; a *literal* is two fields, the coordinate and a unit pop-up listing the compiler’s units for the literal’s own dimension, where the pop-up switches the unit and keeps the quantity and the coordinate field makes a new quantity in the same unit — only a literal has a unit pop-up (Part VI); *operators* are their glyphs, a *call* its name and parentheses, and / or / not their words, a *choice* if with its condition over indented then / else branches (with true / false buttons and truth-valued references offered on a slot that expects true or false), a *binder* a head over an indented body with italic local chips, a *range* two ends around . . . , and an *opaque form* (let, match, a block, a rule as an argument, a collection or grouped literal, delay/sync) its text in monospace, selectable and edited as text. A *finding* is a red underline on the component *and* its row under the field — one diagnostic, two projections. Beneath the field, for the selected component: the compiler’s sentence — *Expected: an angle, because an angle ÷ an angle = a dimensionless quantity* — with the kernel’s notation behind Explain; for a slot, a number entry whose unit pop-up holds the units of the expected dimension (none for a dimensionless slot; none, with a sentence, when the position is not determined), then references by type and equations folded; for a component, + − × ÷, Compare, Function (the equations whose result fits, wrapping the component as first argument) and Remove.

Stale projection policy. A projection is current only when it is of exactly the text on screen and that text parsed; otherwise the field is dimmed with a notice — *Waiting for the compiler to read the formula...* while the verdict for this text is on its way, *The text cannot be read as a formula.* when it never will be, and then no tree is shown because none is invented — no component answers a click, no slot panel opens, no key acts, the reducer refuses a structured action (or a second one in flight) and discards an answer for text that has moved on; **Edit as text** is the way out. Generation safety: every answer carries the draft generation it was computed for and is discarded if the draft has moved. A formula with a slot may be saved and is *invalid* until filled; commit, revert, conflict and detach are the Text view’s, unchanged.

A walkthrough. The designer selects `dimByTilt : Tilt → Brightness` and chooses `Function → clamp`; the draft becomes `clamp(?, ?, ?)`, three dashed slots. Selecting the first, the slot panel says *Expected: a dimensionless quantity* — because `clamp`’s scheme unifies all three arguments with the result and the result is `Brightness`’s representation — and lists references of that kind; the designer chooses `tilt`, and the compiler answers *This is an angle; a dimensionless quantity is expected*, red on the chip. Wrapping it in `÷` yields `clamp(tilt ÷ ?, ?, ?)`; the new slot’s panel now says *Expected: an angle, because an angle ÷ an angle = a dimensionless quantity* and offers `rad`, `deg`, `turn`; the designer types 90 and picks `deg`. Filling 0 and 1 completes `clamp(tilt / 90 deg, 0, 1)`, which checks, and the node’s red mark goes. Switching the `deg` pop-up to `rad` rewrites the literal to `1.5707963267948966 rad` and nothing else. Every step was a byte-range edit of one text; the Text view shows the same characters throughout.

Partial expressions: ?

The slot deserves its own record because it is the clearest example in BDL of authoring semantics distinct from runtime semantics. `?` is a `SlotExpr` of the textual grammar: it parses wherever a value may stand and formats as itself; it is never a value, an operator or a unit (`1 ? 2` is a syntax error). It exists in the *surface* because an incomplete structured formula must be ordinary draft text — the same state as any unfinished draft, savable, visible in the Text view, subject to the same conflict model — rather than a second representation held in an overlay of partial expressions (the rejected alternative, which would have needed its own persistence and its own protocol of operations). Elaboration refuses it with `formula.slot.empty` — *This slot is empty; it expects an angle* when the context says what it expects — and it never reaches Core, the checker, the evaluator or generated code; a kernel hole term was rejected because the kernel has no notion of an unfinished expression and needs none. Expected information flows *into* the slot from the formal Composer’s solve: the operator above it and the known sibling determine its dimension; a product of two slots is *insufficient information* — the formal model reports it unsolved rather than searching, and Studio shows the position as not determined with a sentence. That two-hole case is not a limitation to be engineered away: local, deterministic inference is what makes the expectation explainable in one sentence, and a global solver would trade that for guesses.

Simulate, Deploy, Library, Monitor

Simulate runs the reference evaluator over structured inputs per tick (including list and pair values, and concept values in the design's terms — `Brightness(0.5)`), showing every declaration's value and every output; the simulator does not get a separate interpretation. *Deploy* is the target-relative read model (ADR-0015): board selection reruns deployment analysis only, and the workspace reports validity and deployability in different places; the readiness matrix (bounded-memory refusal, unbounded state, window capacity) is shown here. *Simulate* lists the Sources as its inputs and refuses to step until each has a value; a probe on a concept names what carries it and, on a rule, where it is applied. *Library* is the previous section's. *Monitor* — telemetry from a deployed core — is a placeholder page, as is the platform adapter it needs.

What Studio does not decide

Studio computes no type validity, semantic identity, dimension, causality, clock, output ownership, hardware feasibility or simulation; it holds no parsed formula tree and no unit rule; it arranges nothing at render time. Every one of those is a projection from the daemon, keyed by `EntityRef` and role, never by text position or node id. That is the discipline that lets the same facts appear as socket hues, inspector sentences and Explain notation without three implementations of the language.

Part XIII — Formal ↔ Production Correspondence

The formal development is a specification (ADR-0010): production transcribes it and discharges it by tests; a theorem never proves the Rust or Dart code. This Part states, construct by construct, the formal object and its result, the production object and its status at the snapshot, how strong the correspondence is, every known deviation with its reason, and the proof gap that remains. The classification of each row is one of: **exact** (the production object is the transcription and the tests discharge the stated property); **faithful implementation** (production implements the semantics and is tested against it, without a theorem about the production code); **approximation** (production’s domain differs, with a stated contract); **deliberate product projection** (a production reading of a kernel fact, with no separate formal object); **production-only** (an engineering concern the model does not speak to); **formally proved, not implemented**; **intentionally deferred**; **unresolved divergence** (none at the snapshot). The snapshot is de8154f (Appendix F); the correspondence page of the formal repository (docs/project/production-correspondence.md) and production’s formal-correspondence.md carry the same rows with the FVD and ADR identifiers.

Construct by construct

construct	formal object · result	production object · status	correspondence	deviation and its reason	remaining gap
declarations, interfaces, refinement	DeclEnv, DeclInterface, DeclLeq, EnvRefines; local_refinement_preserves_global_typing, _wf (FVD-0005 ... FVD-0016)	bdl-model snapshot, MappingBlock; apply_ edit with refinement/edit classification and Invalidation sets (ADR-0009); implemented, tested	faithful implementation	production’s seven invalidation categories are an engineering refinement of the two-way split	a per-category preservation theorem
commitments and evidence	commitments : List PropertyId, Evidence, Evidence. Monotone, Satisfies (FVD-0001, FVD-0009, FVD-0010, FVD-0017)	Interface. commitments, always empty; require reserved	intentionally deferred	production authors no commitments; the evidence relation is abstract	a concrete compositional evidence model; interface-level references (ISS-0003)
typing through the type view	HasType, infer_sound/_complete/_unique (FVD-0006)	bdl-check over bdl-ir (Ty, Expr, Prim transcribed); implemented, tested	exact	—	—
nominal concepts, representation, grant	sem s, θ , rep/mk, Grant.of, constructs_granted (FVD-0019 ... FVD-0030)	elaborator inserts rep on inputs and one mk under the declaration’s grant; checker refuses others (ADR-0013); semantic corpus	exact	—	—
dimensions	q d, algebra in Prim.ty (FVD-0031)	bdl-model::Dim (eight bases), primitive typing; product-language diagnostics	exact	—	—
numerics	Nat magnitudes in the executable kernel (FVD-0102)	IEEE f64; division by an exact zero and non-finite results fail the tick (ADR-0011)	approximation	proofs need decidable saturating naturals; products measure reals; NaN/ ∞ have no kernel counterpart	not planned to close; f32 on device a further deviation (ISS-0006)

construct	formal object · result	production object · status	correspondence	deviation and its reason	remaining gap
units and charts	Units.lean, Charts.lean, Rational.lean: exact symbolic scales, exact Q charts, the groupoid laws (FVD-0101 ... FVD-0110)	UnitDef { id, symbol, dim, chart }, Chart:: {Linear, Affine}, one convert; laws property-tested on f64 within ulps; affine charts as infrastructure, not offered (ISS-0004)	approximation	$\pi/180$ is not rational; exactness is what the formal laws are about	the point/difference validation, or a decision not to need it
Composer inference	Composer.lean solve, candidates; solve_sound, _complete (FVD-0104)	bdl-ide::formula projection, slot, compose (ADR-0028); formula_composer.rs	faithful implementation	two-hole operands unsolved by design in both	—
equation library, schemes	Stdlib, Poly, Generic; matchTy_sound/_complete, lib_expansion, *_spec (FVD-0088 ... FVD-0097)	bdl-equations: rank-1 schemes with {Data, Eq, Ord}, matching, one Core builder per instance inlined at the use (ADR-0025)	exact	—	—
equality and order	eq on data with the proof field; lt on quantities; Ty.ordB, Ordered (FVD-0098 ... FVD-0100)	Concept::ordered, ordered concept; ADR-0026's mixed-comparison rule	exact, plus a deliberate product projection	the concept-beside-plain-value case is a production decision the model does not make	recorded as intended
natural binder syntax	Natural.lean: desugar, desugar_rename, binder_local_type, binder*_eval (FVD-0111 ... FVD-0114)	bdl-elab::formula:: binder; natural_forms_lower_to_the_same_core_as_the_call_forms	exact	no parser is modelled; the elaborator's local inference is the output of binder_local_type, described not proved	—
reactive semantics	Ev, Ev.det, reactive_total, Causal (FVD-0034 ... FVD-0042)	bdl-reactive reference evaluator, two-phase tick, StateCellId (ADR-0004)	exact (the evaluator is the executable definition)	Causal is conservative for lambda-guarded cycles in both	causality precision
clock domains, sync	MEv, Clocked, strictly-before, delay_is_sync_own, single_domain_embedding (FVD-0043 ... FVD-0048)	the Clocked pass, the sync snapshot rule, step_in_order	exact	—	—
outputs, drive edge	OutputId, DriveWF, SingleDriver, CompleteOutputs; single_driver_output_deterministic (FVD-0050 ... FVD-0056)	bdl-output; a Source or a Value may drive, a Rule may not	exact	—	—
relationship roles	only Source Δ d (FVD-0118) has a formal object	RelationshipRole { Source, Rule, Value }, one derived predicate	deliberate product projection	Rule and Value are production's names for the arrow type	—

construct	formal object · result	production object · status	correspondence	deviation and its reason	remaining gap
		stated on every projection (ADR-0032 amended, protocol 0.20)		and the realized unit domain	
applied_by, unapplied rules	DependsOn (FVD-0005, FVD-0012)	MappingAnalysis. references and its direct inverse applied_by; reactive.rule_unapplied; rule.apply	production-only over an exact base	—	—
unit-domain canonical type	UnitDomain.lean: elim_canonical, decode_encode, zero_input_obligation, refForms_agree, source_value (FVD-0115 ... FVD-0120)	bdl_ir::Ty::Unit, Ty::of_signature, kernel_of_signature, Signature::is_unit_domain (ADR-0029); ...the_encodings_are_inverse, ...argument_is_erased	exact at the interface; the kernel encoding transcribed	Ty::Unit is an IR type that never types a Core term, representation or runtime value — exactly elim unit = none	—
simulation inputs	the kernel input provides for every unresolved declaration	inputs are the Sources (Source ∧ UnitDomain, SimulationInput.value)	deliberate product projection	an environment provides values, not functions	—
hardware validation	solve_sound, solve_complete, satisfiable_iff_solve, diagnose (FVD-0057 ... FVD-0063)	bdl-hardware, board files (Nano, a larger mock board), target-relative deployment analysis (ADR-0006, ADR-0015)	exact for the finite fragment	numeric constraints out of scope in both	MUS; numeric constraints; RP2040 board file
list data, buffer, capacity	Ty.list, Theorem M buffer_window_correspondence, Capacity.lean (FVD-0083 ... FVD-0087)	Vec last-element-first behind the collections feature, take cap bounds, bdl-reactive::capacity, bdl-exec-ir::bounds, bounded-memory refusal (ADR-0024, ADR-0027); corpus buffer, bounded_buffer, overflowing_buffer	faithful implementation	the static bound analysis has no theorem behind it	a proved bound analysis, or a surface window form (ISS-0001)
products, fold, toList, drop	Phase 9b kernel (FVD-0088 ... FVD-0091)	bdl-ir gains exactly these; Fold in exec IR with one runtime closure	exact	—	—
behavior systems	Theorems A–I; Theorem J restricted (FVD-0064 ... FVD-0073)	bdl-system: freshening from the project allocator, bindings, flatten with origins, stored contracts (ADR-0021, ADR-0022)	faithful implementation	production supports transported bindings under MEV and higher-order bodies beyond the theorem fragment	Theorem J under MEV with a domain-indexed modular input

construct	formal object · result	production object · status	correspondence	deviation and its reason	remaining gap
behavior groups	Theorems A–G (rfl), H–L, M–R (FVD-0074 ... FVD-0082)	.bdl/authoring.json, no invalidation on group edits, the packaging sheet (ADR-0019)	faithful implementation	Theorem R is proved for single-domain wiring designs without transported bindings; production packages beyond it	a port-level causality graph
Source provision	Provision.lean: provision_ envRefines, _wf, _causal, _wellClocked, _transparent, _abstracts, _exact, _not_reapplicable, _comm (FVD-0121 ... FVD-0130)	not implemented (PRP-0001 draft; ISS-0016); ownership decided in relationship-roles.md § Phase 13	formally proved, not implemented	—	stateful transducers, device clock, commitment discharge, output dual
A -> () as a consumer	unit_codomain_ collapse, consumers_ indistinguishable (FVD-0119)	() refused in output position	exact	—	—
generated Rust	— (no theorem)	bdl-lower + bdl-codegen-rust, the no_std core, the host bridge; 22-case differential corpus, golden files, property tests (ADR-0016)	faithful implementation, tested only	the core lowers representation; a lowering bug would be a semantic bug	a generated-code refinement proof
project persistence, identities	—	src/**/*.bdl canonical, .bdl/identities.json, .bdl/authoring.json, ui/layout.json; migration in place; complete authoring state saved (ADR-0023, ADR-0030)	production-only	the model has no files	—
the protocol, the IDE service, highlighting	—	protobuf over framed stdio; bdl-ide-db/bdl-ide/bdl-lsp; one token classifier on the LSP vocabulary (ADR-0007, ADR-0017, ADR-0035)	production-only	—	—
Studio's semantic projection	—	canvas, inspector, Explain, Composer, Simulate, Deploy, Library, Code/Split; Studio decides no semantic fact (ADR-0001, ADR-0018)	production-only	—	usability: open empirical questions

construct	formal object · result	production object · status	correspondence	deviation and its reason	remaining gap
Standard Library	—	LibraryItem → Fragment → ordinary objects, one-transaction instantiation	production-only	an authoring catalogue; not the device catalogue provision needs	the device catalogue
temporal modifiers, contexts	the elaboration cases of Phase 4, executed (FVI-0009)	not offered (ISS-0010)	design recommendation, not implemented	—	handler-scoped clocks; a preservation theorem for the elaboration
supplied computation blocks	—	designed (ADR-0005, component-boundary.md); nothing built	proposal / not implemented	—	—

Deviations, stated once with their reasons

deviation	why	risk	evidence that bounds it	closing the gap
kernel Nat magnitudes vs production f64	proofs need decidable, saturating naturals; products measure reals	truncation in the kernel's executed examples is not production's rounding; NaN/∞ and division by zero have no kernel counterpart	production fails the tick on exact-zero division and non-finite results; exact comparison; the formal theorems are about structure, not rounding	not planned to close; f32 on device is a further recorded deviation (ISS-0006)
exact symbolic unit scales (Sym, π as a generator; exact 0 charts) vs approximate floats	π/180 is not rational; exactness is what the formal laws are about	round trips and compositions hold only within ulps in production	toleranced property tests of identity, composition, inverse, display switch and difference law; exact-rational oracles for registered charts	not planned; the contract is stated and tested
kernel Nat registry has no radian; production is radian-canonical	no integer scale for radians against degrees	none semantic; the registries differ in which units are exact	documented per registry	none needed
theorem-level semantics for direct bindings on wiring designs vs production's transported bindings and higher-order bodies in systems	Theorem J's obstacle: a domain-indexed input for a transported port	production behavior beyond the fragment is tested, not proved	flattening plus the reference evaluator; corpus systems	Theorem J under MEv with a domain-indexed modular input
formal buffer model (unbounded, capacity as a validation predicate) vs production static list bounds and bounded-memory refusal	the kernel has no capacity; production must decide deployability	the static bound analysis has no theorem behind it	differential tests of bounded_buffer vs buffer at 7 and 3 000 ticks; overflowing_buffer	a proved bound analysis, or a surface window form (ISS-0001)
formal Ev/MEv vs generated Rust	the core lowers representation (slots, structs, f64, inlined lambdas)	a lowering bug would be a semantic bug	22-case differential corpus, golden files, determinism, property-based designs, error-tick agreement	a generated-code refinement proof (Part XV)
abstract evidence conditions (Monotone, Equivariant, PortSound, InterfaceLocal) vs	the formal model is parametric in evidence; production authors none	a future production evidence model could violate a condition unnoticed	nothing to violate today; require is reserved	a concrete compositional evidence model, before commitments are authored

deviation	why	risk	evidence that bounds it	closing the gap
production's empty commitments				
mixed comparison rule (ADR-0026)	the formal order policy is between concept values only; production had to decide the concept-beside-plain-value case	$\min(o1, 0.5)$ accepted while $\min(o1, o2)$ refused	a tested matrix; Explain shows the observation	recorded as intended
affine charts hidden from formulas	conversion is safe; point/difference validation does not exist	a designer cannot write $20\text{ }^{\circ}\text{C}$	ISS-0004 open	the point/difference validation, or a decision not to need it
<code>bdl_ir::Ty::Unit</code> exists; the formal kernel <code>Ty</code> has no unit	production wants one type language for the interface and the kernel encoding; the formal model keeps the canonical types in <code>CTy</code> above the kernel	none semantically: <code>Ty::Unit</code> never types a Core term, a representation or a runtime value, which is exactly <code>elim unit = none</code>	<code>elim_canonical</code> ; the production tests that the encodings are inverse and the argument is erased	not planned: the formal <code>CTy</code> is the record that <code>Unit</code> is an interface type
simulation inputs are unresolved <i>unit-domain</i> declarations; the kernel Input provides for every unresolved declaration	an environment provides values, not functions	none: on a unit-domain source the two agree (<code>SimulationInput.value</code>)	<code>source_value</code> , <code>resolved_not_source</code>	not planned; recorded as surface policy
three roles in production; one formal role	production needs a name for the arrow type and the realized unit domain in every surface; the kernel does not	none: the role is a function of two kernel facts	<code>MappingBlock::role</code> tests; <code>the_role_is_one_answer_across_the_component_boundary</code>	none needed
provision proved; nothing provisions	the construction was audited before production builds a device catalogue	none today; the risk is building the transducer as adapter host code	PRP-0001 revised; ownership decided in advance	implement PRP-0001 (ISS-0016)

The snapshot

What is implemented, partial and planned at `de8154f`, with the protocol history and the milestones of the last week, is Appendix F, so that this Part's classification survives the next milestone and the snapshot is updated in one place.

Part XIV — Minimality and the Rejected Alternatives

The kernel was obtained by a method, and the method is part of the record. Each phase of the formal development took a family of candidate constructs — from the earlier draft of the language, from the production roadmap, or from the standard repertoire of typed functional languages — formalized the smallest plausible version and its alternatives in Lean 4 without external libraries, and asked the same questions of each: What does it reject that the others accept? What does it accept that it should not? Is it a special case of another? Which operations on a design are refinements under it? A construct entered the kernel only when every tested alternative failed for a stated reason, recorded as a theorem or an executable counterexample. Verdicts use one vocabulary throughout this document and the formal records: KEEP IN KERNEL, KEEP IN SURFACE-DESUGAR (or DEPLOYMENT CONSTRUCTION), KEEP IN STANDARD LIBRARY, MOVE TO VALIDATION, MOVE TO AUTHORING/UI (presentation), DEFER, REMOVE. Parts III–X give the verdict for each construct where the construct is discussed; this Part is the ledger that gathers them, and the construct-by-construct table with every row’s evidence is docs/kernel/minimality.md in the formal repository (the decisions are docs/decisions/, FVD-0001 ... FVD-0130).

Two kinds of minimality

Two questions are asked of every construct, and they have different answers more often than not.

Kernel minimality asks whether the construct is needed for some design to be representable or unambiguous, with a theorem or counterexample as the argument. By this test `Signal`, `Event`, `effect` rows, `action` values, a `StateHandler`, a `group`, a `unit`, a `binder`, a `polymorphic` type, a `Source` kind and a `provision` primitive are all *unnecessary*: each is either derived from what exists or rejected outright, and the kernel band of Part III’s architecture figure is what remains.

Designer-facing vocabulary minimality asks whether the construct captures a distinct design concept that cannot be expressed cleanly as a property of an existing one — the cognitive-budget test of Part I. By this test a **Source**, a **behavior** (`group`), a **natural binder**, a **unit** on a literal, a **choice** in the Formula Composer and a **context** are all *valuable*: each names something a designer means, and each is kept at the surface, in presentation, or as a design recommendation for a surface not yet built. That a construct desugars away is a fact about the kernel; it is never a reason to remove it from the product. The `Source` is the clearest case — nothing in the kernel distinguishes it, the product draws it first — and the provision result of Part IX is what makes the two facts consistent: the surface concept is exactly a realization state, and the state changes at deployment by a construction the kernel already admits.

The verdicts below say which kind of minimality each row is about. REMOVE is a kernel verdict and a surface verdict at once only where the rejected table says so.

Kernel constructs retained

construct	why it could not be derived	evidence
<code>DeclId</code> and frozen expected types	references resolve by identity; retyping invalidates clients	probes 1–6, <code>local_refinement_preserves_global_wf</code>
nominal <code>sem s</code> with <code>θ</code> , <code>rep</code> , <code>mk</code> under a grant	the numeric baseline accepts the invalid wire; a metadata checker is η -evaded; concept-as-declaration has category errors	Phase 2/3 Models A–C, <code>constructs_granted</code> , <code>temporal_state_preserves_semantic_identity</code>
<code>q d</code> with the algebra in <code>Prim.ty</code>	erasure accepts <code>length + time</code>	<code>dimension_mismatch_rejected</code> , <code>counterexampleB_baseline_accepts_length_plus_time</code>
<code>delay</code> , <code>sync</code> (one temporal read)	totality forces data-typed, top-level state; <code>delay</code> is <code>sync</code> at the own domain	<code>reactive_total</code> , <code>delay_is_sync_own</code> , <code>scheduling_order_observable</code>
<code>ClockId</code> , <code>Clocked</code>	a direct wire is ambiguous without it; equal rate is not the same domain	<code>equal_rate_not_same_domain</code>
<code>OutputId</code> , drive edge, <code>SingleDriver</code> , <code>CompleteOutputs</code>	two drivers make an output non-functional; hidden policy is observable	<code>multiple_direct_drivers_rejected</code> , <code>hidden_arbitration_observable</code>
<code>list τ</code> and its operators	a lossless window is unbounded sequence data	<code>bounded_summary_not_lossless</code> , Theorem M

construct	why it could not be derived	evidence
prod, pair, fst, snd	function encodings are not data and cannot be state	arrow_not_delayable, church_fst_rank
fold	one eliminator derives every collection operation; operators never apply closures	fold_total, the *_spec family
eq at every data type (proof field)	boolean/concept/pair/list equality was unwritable	Value.beq_iff
lt on quantities	ordering is a property of magnitudes	lt_rejected, lt_only_on_quantities

Rejected constructs

candidate	reason it was considered	counterexample or theorem	verdict
Signal τ in Ty	the earlier draft's reactive types	inhabited by exactly the terms of τ ; rejects nothing	REMOVE
Event τ (single domain)	occurrences as a type	opt τ streams are the multiplicity- ≤ 1 streams	REMOVE
Event τ (across domains), a buffer primitive	multiplicity across rates	the window is five declarations over delay/sync/lists; Theorem M	REMOVE (surface desugar)
effect rows	output effects in types	duplicate the drive edge or false-positive	REMOVE
action requests / action values	outputs as values	relocate the conflict without resolving it	REMOVE
runtime output arbitration	several drivers	hidden_arbitration_observable: the hidden policy is observable	REMOVE (single driver)
clock-indexed types Signal[c, τ]	domain in the type	clocked_type_forces_polymorphism on every pure mapping	REMOVE
same-tick cross-domain visibility	"synchronous sub-domains"	scheduling_order_observable: the scheduler order becomes observable	REMOVE (strictly-before)
scheduler / staging rule	ordering simultaneous domains	unnecessary under strictly-before	REMOVE
explicit machine state / Step	a state machine object	Phase 4/8: state is gated self-delayed declarations; components hold state by instantiation	REMOVE
StateHandler kernel	contexts as a construct	elaborates to activation, entry, gated and reset state, conditional drivers (tested behavior only)	KEEP IN SURFACE-DESUGAR; nesting/handler-scoped clocks untested
state identity as a kernel notion	naming state	StateCellId by declaration and expression path is an engineering device; the kernel has none	REMOVE
user-defined typeclasses / dictionaries	constrained generics	closed {Data, Eq, Ord} covers the library; comparators recover the rest (minBy_recovers_min)	REMOVE
structural ordering on all data	Phase 9b's first form	mode < mode, None < Some, lexicographic pairs have no design meaning	REMOVE (9c)
Set type	finite sets	oneOf_mem, oneOf_dup_irrelevant: membership is a fold	REMOVE
Interval type	ranges	a pair with a convention; inRange_spec; no case stores or compares a range	REMOVE

candidate	reason it was considered	counterexample or theorem	verdict
record type / row polymorphism	records	nested pairs with positional projection; no case needs shape-generic functions	REMOVE
kernel polymorphism (type variables, $\forall$, Λ , $[\tau]$)	generics	prenex fragment = family instantiation; <code>matchTy_sound/_complete</code>	REMOVE (surface families)
higher-rank types	polymorphic function arguments	every candidate rank ≥ 2 with a rank-1 replacement	REMOVE
existential types	hiding	Phase-8a instantiation hides; encoding is rank 2	REMOVE
general recursion	loops	total language; <code>fold</code> is the one eliminator	REMOVE
general quantification in expressions	$\forall x : \text{Real}$	finite $\forall/\exists$ are folds (<code>forall_in_list</code>); unbounded forms belong to a commitment layer	REMOVE (surface finite forms)
general comprehension (generators, <code>yield</code> , <code>where</code>)	list syntax	nested binders cover every required case	REMOVE
runtime unit values	units as data	no case delays, syncs, stores or compares a unit	REMOVE
units in quantity types	unit-indexed q	<code>1 m</code> and <code>100 cm</code> would differ in type	REMOVE
a kernel conversion primitive	<code>convert</code>	it is the composition of <code>inUnit</code> and <code>withUnit</code>	KEEP IN STANDARD LIBRARY
affine quantity sort in $Ty.q$ ($Ty.q\ d\ \text{sort}$)	$^{\circ}\text{C}$ arithmetic	no conversion theorem needs it; <code>sort_orthogonal_to_conversion</code>	REMOVE from types; optional validation
kernel hole term	the Composer's slot	the surface $?$ is refused by elaboration; holes belong to the editor	REMOVE
semantic <code>BehaviorGroup</code>	groups as objects	every group operation is the identity on the design	MOVE TO AUTHORING/UI
component runtime object	components at runtime	flattening produces an ordinary design; no system evaluator exists	REMOVE
aggregate socket as a declaration / tuple boundary	collapsed groups	Counterexamples 5 and 6: spurious dependencies	REMOVE
kernel <code>sum</code> type	enums	encoded as tag $\times$ optional payload (<code>exM</code>)	DEFER
kernel unit type / unit value / unit term	the literal reading of $() \rightarrow B$	<code>elim unit = none</code> ; a unit binder around memory is untypable (<code>delay_not_under_binder</code>); the kernel encoding permits memory (<code>zero_input_memory</code>)	REMOVE — KEEP $()$ IN THE INTERFACE LAYER
a source semantic kind	sensors as a kind of declaration	the source role is <code>realizationOf d = none</code> ; a resolved $() \rightarrow A$ never reads the input (<code>resolved_not_source</code>)	REMOVE / DERIVE
$A \rightarrow ()$ as a physical consumer	the dual of $() \rightarrow A$	every pure total $A \rightarrow 1$ is one function (<code>consumers_indistinguishable</code>); the drive edge names the receiver	REMOVE
numeric rates in the kernel	periods	a rate induces a schedule; validation data	MOVE TO VALIDATION
buffer capacity, overflow policy	bounded memory	<code>sufficient_capacity_preserves</code> , <code>negE</code> : only refusal preserves semantics	MOVE TO VALIDATION; explicit take in the design
preferred display unit	UI preference	<code>presentation_irrelevant_*</code>	MOVE TO AUTHORING/UI

candidate	reason it was considered	counterexample or theorem	verdict
interface <code>semanticRole</code> field with a direct-wire checker	concept identity as metadata beside the type	evaded by η -expansion: $(\lambda x. x) \text{ tilt}$ has the same flow and no direct wire; the compositional form duplicates nominal typing	REMOVE (kernel); the concept stays the surface object
a source kind / <code>MappingKind::Source</code> / a <code>Source</code> keyword	sensors as a kind of declaration	the source role is <code>realizationOf d = none</code> ; a <code>resolved ()</code> -> A never reads the input (<code>resolved_not_source</code>); a persisted kind could disagree with the shape after a definition is added	REMOVE (kernel, text, protocol); KEEP the <code>Source</code> as a derived surface role
a <code>ProvisionedSource</code> role or kind	a <code>Source</code> realized by a device	after provision the target has a realization and is a <code>Value</code> by the same rule (<code>provision_source_role</code>)	REMOVE
singleton-only provision (one raw declaration per <code>Source</code>)	the simple case first	the IMU case is the realistic one; the joint-section finding is invisible in the singleton (<code>no_joint_witness</code>)	REMOVE; shared raw is primitive, the singleton its case
a stateful transducer as the profile condition	debouncing and filtering in the profile	a typed impure term maps one raw value to two results at two ticks (<code>exD</code>); the induced input must be a function	DEFER (FVI-0020); purity is the condition today
“idempotent” provision	re-applying a deployment	the precondition fails after provision (<code>provision_not_reapplicable</code>); a second pass with another term is not a refinement	REMOVE the word; not re-applicable
Sensor as the library category	the everyday word	not every <code>Source</code> is a sensor; the template must carry no semantics the text cannot	MOVE TO AUTHORING/UI: the category is <i>Sources</i>
the Standard Library as the device catalogue	one catalogue	authoring fragments and deployment profiles are different objects with different owners	REMOVE; a device catalogue is separate (not built)
a Dart tokenizer or <code>TextMate</code> grammar as Studio’s highlighter	instant colour	a second definition of the language in the presentation layer, blind to role, unit-by-position and contextual keywords	REMOVE; one classifier in <code>bd1-ide</code> (ADR-0035)

Decisions that changed

Not rewritten to look inevitable:

original position	evidence that broke it	replacement
Signal and Event as kernel primitives (draft)	a signal type rejects nothing; occurrences are optional streams; cross-domain multiplicity is a window over lists	ordinary typed declarations; the window as five declarations
runtime output arbitration (draft)	the hidden policy is observable	one explicit driver per sink; combination is ordinary computation
<code>eq/lt</code> at every data type through a structural order (Phase 9b)	<code>mode < mode</code> , <code>None < Some</code> , lexicographic pairs have no design meaning (Phase 9c audit)	<code>eq</code> on data; <code>lt</code> on quantities; order by declaration at the surface
affine units need a point/difference sort — “missing information” (Phase 10)	chart laws, groupoid laws and the difference law hold without a sort; conversion never takes one (Phase 10b)	conversion complete as coordinate change; point/delta is optional validation

original position	evidence that broke it	replacement
a third project kind, text, beside JSON kinds with conversion (ADR-0020)	two persistence forms for one meaning are two authorities that drift	one project; Design, Code and Split are views (ADR-0023)
an authoring overlay of partial expressions (considered for the Composer)	a second representation of the formula to keep in step with the text	the slot ? in the text; every action a byte-range edit (ADR-0028)
a relationship without inputs as “a value declaration of type B”, tested by <code>inputs.is_empty()</code> at every layer (production before ADR-0029)	no record said what the type of such a relationship <i>is</i> ; six layers each carried their own special case	one canonical type <code>domain(inputs) -> B</code> with <code>()</code> for no inputs, one predicate <code>is_unit_domain</code> , the kernel encoding by unit elimination proved exact (Phase 12)
a fixed-capacity list type with an overflow error (considered)	a runtime failure the formal model does not have; a type depending on a deployment fact	<code>Vec</code> under a validated bound; refusal on bounded targets (ADR-0024/0027)
Studio deriving the Source role from the shape of the projection (ADR-0032, first form)	two derivations of one fact drift the day either changes; the IDE service and Studio had already answered differently for port-backed declarations	one derived role in the model, stated by the compiler and the daemon on every projection; Studio maps the enum (ADR-0032 amended, 2026-09-20)
a BDL-specific semantic-token vocabulary in the language server (concept, mapping, output, ...)	every LSP client needed a mapping per name	the LSP standard names with BDL distinctions as modifiers; <code>unit</code> and <code>slot</code> the only additions (ADR-0035)

Part XV — The Open Agenda

The agenda has two halves that must not be confused: questions of semantics and formal correctness, which a theorem or a counterexample would settle, and questions about designers, which only a study would. Items production or the formal development has already answered are gone from this list; each remaining item names what exists and what would resolve it, and none is hidden in a “future work” sentence.

Formal limits, stated once

Causality is conservative for lambda-guarded cycles: `Causal` rejects them and the negative theorem does not cover them. The agreement between unfolding and tick-by-tick evaluation is proved for first-order wiring designs; higher-order closure equivalence is not. Theorem J (modular semantics) is proved for direct bindings on wiring designs with closure-free inputs; transported bindings under `MEv` and higher-order bodies are not covered, in either direction, for the same reason — a domain-indexed input for a transported port. Theorem R (extraction preserves behavior) is likewise proved for single-domain wiring designs without transported bindings. The `StateHandler` reduction covers the tested behavior and not contexts with their own clocks or independently clocked nesting. `InstAcyclic` is too coarse for extraction; a port-level graph would subsume `flatten_causal` and `flat_causal`. Four conditions rest on the abstract evidence relation (`Monotone`, `Equivariant`, `PortSound`, `InterfaceLocal`), and production discharges none of them because it authors no commitments. Hardware validation covers discrete pin and peripheral allocation with unary and binary constraints, and no numeric electrical, thermal or timing property; the explanation facility reports a first dead end, not a minimal core. Interface-level references — commitments that mention other declarations — are not modelled, so the dependency graph is over realizations only. Whether a realization may delegate its grant to a higher-order argument is untested. The buffer correspondence assumes an input source. The equation library’s evaluation lemmas assume the predicate value implements a Boolean function; the executed cases discharge it. The exact unit model is related to production floating point by a stated contract, not a theorem. Sums are encoded, not added. Two-hole operands are outside `solve` by design. No parser is modelled for the natural surface; its type annotations are the output of local inference, described, not proved. Provision is proved for pure transducers at the target’s clock, and for shared raw readings with a joint section where equality is claimed. No theorem covers the Rust or Dart code.

Open formal and engineering problems

1. **A general edit/invalidation relation** (FVI-0015). Refinement is a preorder with proved client stability; an arbitrary edit is outside it and forces a recheck of dependents. Production classifies edits into seven invalidation categories (ADR-0009). Missing: a formal relation that says, per edit kind, exactly which established facts survive — the theory behind incremental re-analysis. Would resolve: a proved per-category preservation theorem.
2. **Commitments that depend on interface references** (FVI-0001; ISS-0003). A commitment is a property id discharged by evidence over the realization; a commitment mentioning *another declaration* has no model, so interface-level cycles are invisible. Would resolve: an interface-level dependency relation and its interaction with refinement — and, before it, any production authoring of commitments at all.
3. **Lambda-guarded causality precision** (FVI-0003). `Causal` counts a reference under a lambda as instantaneous. Would resolve: a causality judgment with application sites and its totality theorem.
4. **Higher-order closure equivalence** (FVI-0004). `unfolds_preserves_eval` holds for wiring designs. Would resolve: an equivalence of closures across unfolding, or a first-order normalization of higher-order realizations before unfolding.
5. **Full multi-clock component semantics** (FVI-0006). Theorem J under `MEv` with transported bindings needs a domain-indexed consistent modular input and a proof of its existence. Production supports the case; the theorem does not.
6. **Nested and stateful component elaboration limits** (FVI-0007, FVI-0009; ISS-0007, ISS-0010). `toComponent`’s side condition is decidable and unproved; packaging inside a component body is not modelled; contexts with handler-scoped clocks and independently clocked nesting were not examined, and production offers neither temporal modifiers nor contexts. Would resolve: an elaboration with a preservation theorem, or a counterexample forcing a kernel construct.

7. **Affine physical arithmetic validation** (FVI-0017; ISS-0004). The point/difference sort as an operation-level validation, its rules (`affAdd`, `affSub`) at the surface, and the Composer offering only difference units for a delta slot. Would resolve: the validation and a decision to offer °C in formulas, or a decision not to need it.
8. **Hardware: minimal unsatisfiable cores** (FVI-0011). `diagnose` reports a first dead end. Would resolve: a minimal core with a proof of minimality, or a decision that the first dead end is the better explanation.
9. **Numeric electrical, thermal and timing constraints** (FVI-0011). Out of the solver’s scope; a design can be allocated and still exceed a current budget. Would resolve: a numeric constraint layer beside the finite solver, with its own soundness.
10. **A generated-code refinement proof**. The core is held to the reference evaluator by differential tests. Would resolve: a proof that lowering plus code generation refines `Ev/MEv` — or a verified evaluator — closing the largest deviation of Part XIII. A proved static bound analysis is the same gap on the capacity side.
11. **Source provision, next steps** (FVI-0020; PRP-0001, ISS-0016). Stateful transducers and a stream-level transparency theorem; a device clock with a deployment sync; how a profile’s declared range discharges a Source’s commitments; output provision, the dual; whether `computes` is checked or trusted at the catalogue; out-of-type raw readings as validation; and, on the production side, the device catalogue and the implementation itself.
12. **Enums and sums** (ISS-0005). Encoded as `tag × optional payload`; production keeps user enums open. Would resolve: a case that needs `match` exhaustiveness beyond the encoding, and then one eliminator term former like `fold`; or a decision that the encoding is the language.
13. **A surface form for occurrence windows** (FVI-0012 in part; ISS-0001), so a bounded refinement can be recognized and a ring representation offered without a hidden transformation.
14. **Several candidate definitions with one active** (FVI-0014; ISS-0002): whether this is a surface convenience over a write-once kernel realization.
15. **Grant delegation** (FVI-0018): whether a realization may delegate its grant to a higher-order argument; the `display-name` table and the preferred display unit as presentation objects (FVI-0019).
16. **A mathematical specification backend and a verification backend**. The kernel is a specification; a backend rendering a design as a mathematical document for engineering hand-off does not exist; commitments beyond typing have an evidence slot and no discharge mechanism; the exact chart model and the choice-free rational field are the first pieces of a verification story about quantities.
17. **Production engineering items with no formal content**: projection deltas and a persisted edit history (ISS-0009); a structural diagnostic entity for outputs (ISS-0008); a linear `zip` in the core (ISS-0013); the compiler’s diagnostic sentences in every locale (ISS-0015); the first platform adapter, build orchestration, flash and telemetry (production’s roadmap priorities 1–4).

Empirical questions, explicitly

The interaction model of Part II and Part XII is fully described, largely built, and no part of it has been evaluated with users. The following are hypotheses, to be tested, and nothing in this document — no theorem, no test, no differential corpus — is evidence for any of them:

- that an unresolved typed relationship is a natural stopping point for designers, and that its cost is close to zero;
- that Source, Rule and Value are learnable as one distinction, and that *not applied* with an offered value is understood;
- that socket hue for identity and socket shape for value form are read correctly without type labels;
- that the three information levels place each fact where a designer looks for it, and that Explain is opened rarely and usefully;
- that the Formula Composer’s slot expectations and candidate lists reduce the semantic-translation cost the workshop exposed, and that designers read `all reading in readings: better than all(readings, reading => ...)`;
- that product-language diagnostics (“This adds an angle and a time”; “Mode values have no default order”) are understood and acted on;
- that Design, Code and Split are experienced as views of one thing, and that semantic colour in the Code view helps rather than distracts;
- that groups, packaging and instances match how designers organize behavior;

- learnability across the vocabulary; productivity against a Node-RED- or Arduino-style baseline; cognitive load in the sense of the cognitive-dimensions framework.

The studies that would answer them are restated below with their measures. Neither has been run.

Expected cognitive advantages, as hypotheses

Lower viscosity. Changing a transfer function edits one Mapping definition rather than a procedural chain of read/compute/store/write nodes.

Reduced hidden dependencies. Semantic signatures expose what a relationship consumes and produces, while the kernel rejects incompatible connections before device code exists.

Reduced premature commitment. An unresolved declaration allows a designer to commit to a relationship without committing to its implementation, sensor, or exact parameters.

This last item requires a qualification that the rest do not. Signature-first authoring is not claimed to be the spontaneous habit of every designer. The author, working on a hardware project, adopted it when the complexity of a subsystem exceeded what could be held in view at once, and did not adopt it on simpler tasks, where a signature and its definition were written in a single motion. The hypothesis is therefore narrower than a claim about how designers think. It is that conventional tools provide no legal position for an undefined relationship, so the strategy cannot be adopted even when it would help, and that BDL supplies that position at no cost. Whether inexperienced designers take up the strategy once it is available, and whether doing so improves their designs, is an empirical question and is included in the comparative study.

Better role expressiveness. Contexts, Mappings, and temporal modifiers correspond to product-design concepts rather than generic program-control constructs.

Improved error locality. A mismatch is attached to a product relationship or binding rather than surfacing later as an embedded runtime fault, and a hardware infeasibility is attached to the binding that causes it.

Comparative study

A first controlled study should compare BDL against at least two baselines: a statechart-based prototyping environment and a node-based or Arduino-style implementation workflow. Participants should be industrial-design students and practitioners with limited professional software-engineering experience.

Tasks should include specifying a sensor-to-actuator mapping; adding temporal qualification such as “for 300 ms”; adding an orthogonal safety override that competes with an interaction context for one output; replacing a sensor with a different sample rate; relating a slowly updated quantity to a fast interaction, which forces a cross-domain decision; choosing a board that cannot accommodate the design; and modifying a mapping late in the task.

Primary outcomes should not be limited to task time or a usability scale. More important measures are semantic errors in the final behavior; the number of implementation-only concepts participants must manipulate; time to detect an impossible or conflicting behavior; fidelity between verbal design intent and the elaborated model; the number and diversity of behavior alternatives explored; the quality of handoff to an engineer who did not observe the authoring session; subjective confidence calibrated against actual correctness; and whether, and at what level of task complexity, participants declare a relationship before defining it when the tool permits both.

The last measure tests the hypothesis stated above rather than assuming it. Because signature-first authoring may be a strategy that appears only above a complexity threshold, the tasks should vary in scale, and the default state of a newly created relationship is itself a manipulable factor: a block that opens onto an empty formula editor and one that opens onto a signature with an explicitly legal undefined body invite different first actions. A small comparison of these two defaults is considerably cheaper than the full study.

The interaction model of Part II adds hypotheses of its own. Whether the single-driver diagnostic leads participants to an explicit combination block they can later read, whether the cross-domain question is answered correctly for a safety condition, and whether participants distinguish a valid design from a deployable one when the workspace reports them separately, are each measurable in the tasks above.

Field study

A controlled study cannot establish whether the representation fits real design practice. A second phase should embed the tool in a semester-long product-design studio or an industry project. The study should observe where unresolved declarations persist, which semantic types designers invent, where they request escape hatches, how often the single-driver condition is met by a combination rule the designer finds natural, and how often engineers reinterpret or replace BDL artifacts during implementation. This field evidence is necessary before claiming that the language is native to industrial design rather than merely pleasant to its authors.

Risks of the design

Semantic-type proliferation. If every semantic distinction creates a visible type, the editor may become bureaucratic. The system needs reusable type libraries and sensible defaults, and the study should record which types designers invent.

Formula anxiety. Not every designer wants to write equations. Formula authoring must coexist with curves, examples, and direct manipulation.

Hidden elaboration. A context that elaborates to an activation declaration, an entry declaration, gated state, and a conditional driver is, in the elaborated design, several objects the designer did not draw. Hiding this can make runtime behavior mysterious. The explanation view of the interaction model is the proposed answer; whether designers use it, and whether it explains what they came to ask, is untested.

False confidence. A formally typed diagram can look verified even when no physical property has been checked, and a typed, causal, clock-consistent, output-complete design can be unplaceable on the chosen board. The workspace states of the interaction model exist to keep these apart, and the risk is sharpest for obligations discharged by declaration rather than by analysis.

Complexity migration into tooling. Much of what was removed from the kernel — event policies, output selection, context semantics — reappears as elaboration. The kernel is smaller and better understood; the elaborator is larger, and it exists (Part IX) and is held to the kernel by differential testing rather than by proof (Part XIII). The claim that the surface is “only syntax” over the kernel is a claim about tested cases, and the untested cases are the ones most likely to demand a kernel extension.

Usability hypotheses unestablished. Every statement in this document about what designers find natural is a hypothesis, including every sentence of the interaction model. The one anecdote reported is a single author’s practice on a single project.

Intellectual Context and Related Work

This chapter places BDL against the traditions it intersects: what it borrows from each, and where its abstraction boundary differs. Each subsection ends with that difference, stated as a difference in *artifact* or *author*, not as a claim of superiority; the chapter is context for the design, not a case for its novelty.

Industrial design tools and physical prototyping

Phidgets reduced the implementation cost of physical interaction by presenting hardware components through a uniform software abstraction [1]. d.tools integrated physical prototyping, statechart-based behavior, testing, and analysis for designers [2]. Exemplar addressed the same authoring cost from the opposite direction, letting designers demonstrate sensor behavior and having the system infer the recognizer [8]; in BDL that technique is one of several ways to supply a definition, attached to a signature that already fixes the relationship’s semantic boundary. The Arduino ecosystem [9] is the de facto medium of the workshop reported in Part I, and its `loop()/digitalRead/analogWrite` vocabulary is the operational structure whose missing “place for a fact” motivated the language. The difference from these systems is not that they could not represent behavior. It is that their dominant representation still asks designers to formulate substantial portions of behavior in an operational structure; BDL investigates whether the typed semantic relationship can be the primary artifact, with operational machinery elaborated underneath.

Visual programming and end-user programming

Node-RED [10] is the closest deployed representative of the flow-graph medium that Part II analyses: nodes are computations wired by message passing, and the graph is a dependency view onto an event loop. Scratch and its block-language descendants [11] showed that syntax can be removed as an obstacle without removing the program-counter model, and Part II’s argument is precisely that the program counter, not the syntax, is the cost for designers. End-user software engineering [12] documents the tension between low-threshold authoring and the errors that follow from the absence of static structure; BDL’s answer is to make the static structure — semantic identity, dimension, domain, single driver — the medium, and to make it legible through the three information levels rather than through diagnostics after the fact. Cognitive dimensions [4] supplies the vocabulary — viscosity, hidden dependencies, premature commitment — in which Part XV’s empirical questions are posed.

Synchronous languages and functional reactive programming

The kernel’s temporal basis is that of the synchronous dataflow tradition. Lustre’s `pre` with an initial value, in a declaration-per-stream setting, is the delay primitive here [5]; the rule that a value computed at an instant is visible at the next instant, applied across domains, is the strictly-before rule; and causality as a static property follows the same line [13]. Esterel established the synchronous hypothesis under which these semantics are deterministic [14], and SCADE industrialized the tradition with a qualified code generator [15] — the destination that Part XV’s “generated-code refinement proof” would move BDL toward. Where these languages recover clocks by a clock calculus [6], BDL requires domain identity to be declared, for the reason given in Part VII. Zélus extends the lineage to hybrid systems [16], which is the direction in which the continuous-dynamics limitation would have to be addressed.

Functional reactive programming established behaviors and events as compositional abstractions for time-varying computation [17], and FrTime gave a dynamic dataflow embedding with formal semantics [18]. BDL’s reactive core is much more restricted, and the restriction is a result rather than a starting point: under a tick semantics with one domain, a signal type rejects nothing and an event type is an optional-valued stream, so neither appears in the kernel; across domains, the event buffer is five declarations over `sync` and lists (Part VII).

Model-based design, statecharts and systems engineering

LabVIEW established the graphical dataflow instrument-control paradigm; Simulink with Stateflow combines dataflow blocks with hierarchical state machines; Modelica models physical systems through acausal equations with units and dimensions [19], [20], [21]. These systems are considerably more capable than BDL. The distinction is in the primary artifact and the intended author: in each of them the artifact is an executable model whose blocks denote computation and the author holds an engineering model; BDL’s artifact is a set of typed relationships that need not yet compute anything, and the author holds a product model. Statecharts [22] are the representation that BDL’s *contexts* replace for the designer: a context is a named situation with an activation condition, elaborated to gated and reset state and a conditional driver, rather than a transition table the designer maintains. SysML and model-based

systems engineering address precise system structure, requirements, and verification at a broader level [23]; BDL is intended as a front end for early design that could later export into such representations, not as a replacement.

The model-driven-architecture ladder is a useful comparison because BDL’s boundary sits at a different rung. Classic MDA moves a computation-independent model through a platform-independent and a platform-specific model to an implementation, and the platform-independent artifact is already a software model — classes, components, messages. BDL’s artifact is a behavior design plus a realization and deployment step, and the behavior design is itself executable, formal and simulatable *before* any software ontology appears: its objects are concepts, relationships, domains and outputs, and the elaborator, the generated core and the platform adapter are the rungs below it. BDL places the platform-independent boundary above the conventional software-implementation ontology; it does not replace MDA, and a BDL design could be the input to an MDA pipeline that starts where BDL’s deployment step ends.

Typed holes, live and structured editing, projectional editing

Hazelnut demonstrated that incomplete structured terms can remain statically meaningful under bidirectional typing [24], and Hazel extended this to live evaluation around holes [25]. BDL takes the idea that incompleteness is a first-class static state and relocates it: the kernel object is a named declaration whose realization is optional; holes are positional in the Hazelnut tradition and named here at the declaration level, while the Formula Composer’s slot ? (Part VI, Part XII) is a positional hole *inside* a definition, elaborated as an expression with an unresolved type and never shipped. The stability result concerns clients of a declaration under refinement rather than the typing of the incomplete term itself. Projectional editors [26] edit an AST directly and render it; the Composer is deliberately not one — the text remains the source of truth and the Composer is a projection over it (ADR-0028), which is why byte-range edits, ephemeral node identity and a stale-projection policy exist at all. Cognitive dimensions [4] frames the trade-off these editors make and the one BDL makes.

Dimensional typing and units

Dimensional typing follows the units-of-measure line begun by Kennedy [27]. What is specific here is the placement — the algebra in primitive operator types with no dimension-specific rule, and the separation of dimension from nominal identity — together with the mechanized observation that the numeric baseline is the erasure. Part VI goes further than the units-of-measure literature usually does in two respects: units are *coordinates on a dimension* with a symbolic exact scale group, and affine units are charts whose conversions form a groupoid of affine maps with a proved point/difference decomposition; Modelica’s `displayUnit` [21] is the closest deployed analogue of Part VI’s presentation layer.

Effects

An earlier draft of BDL borrowed the separation between operation and interpretation from algebraic effects [28] and anticipated scoped effects for context-sensitive interpretation [29]. The kernel has no effect system. The negative result is stated narrowly in Part IX and does not bear on effect systems in general; it bears on the formulations tried for this design problem, where a single explicit driver per output expressed everything the request-and-policy model expressed and made visible what it hid.

Resource allocation

The hardware validation layer is a finite constraint satisfaction problem with unary and binary constraints [7], and its solver is a plain backtracking search whose soundness and completeness are proved. No claim is made relative to the embedded co-design literature; what is specific to the layer is architectural — the design is never an input to the solver, and feasibility is kept as a separate, non-monotone kind of evidence — rather than algorithmic.

Formal verification, verified compilation and differential testing

The kernel is mechanized in Lean 4 [3] without external libraries, and every result in this document rests on propositional extensionality and quotient soundness alone. The relationship between the kernel and the production compiler is *specification*, not *extraction*: production reimplements the reference evaluator in Rust and holds itself to it by differential testing [30] over a corpus (Part XI). This is the weakest link in the trust chain (Part XIII) and the point where a verified-compiler approach in the CompCert tradition [31] would apply; Part XV lists a refinement proof from lowering to Ev/MEv as an open problem. The choice-free rational field of Part VI was built because importing a

general-purpose library would have brought classical choice into the axiom base; that discipline is a methodological choice of this project and not a claim about the libraries.

Embedded DSLs and toolchains

Production BDL's shape — a daemon with a project model, a compiler producing a `no_std` core, and a thin platform adapter — is a conventional embedded-DSL toolchain, and no novelty is claimed for it. What is specific is the placement of the semantics: the generated core is held to a reference evaluator that is itself held to a mechanized kernel, and the capacity, bounds and allocation questions are answered at validation rather than by a runtime allocator (Part VII, Part XI).

Appendix A — Formal notation

The notation is the Lean development’s, kept uniform across Parts III–X; where an earlier phase used a different spelling the current one is used throughout and the old one is mentioned only in Appendix G.

symbol	Lean object	meaning
Δ	<code>DeclEnv : DeclId → Option DesignDecl</code>	the design: an environment of declarations
d, DeclId	<code>DeclId</code>	a declaration’s stable identity; display names are not modelled
<code>DesignDecl</code>	<code>(id, interface, realization)</code>	a declaration: identity, interface, optional realization
<code>DeclInterface</code>	<code>(expectedType, commitments)</code>	the frozen expected type and the monotone public commitment list
Δ^{ty}	<code>DeclEnv.tyView</code>	the type view: the expected type of each declared identity
Θ	<code>ConceptEnv : SemanticId → Option Ty</code>	the write-once representation binding of concepts
$s, \text{SemanticId}$	<code>SemanticId</code>	a concept’s identity; <code>sem s</code> its nominal type
$q \ d$	<code>Ty.q Dim</code>	a physical quantity of dimension d (an exponent vector)
<code>Ty</code>	<code>bool nat arr sem q opt list prod</code>	the kernel types
G, Grant	<code>Grant, Grant.of τ</code>	the construction grant: the concepts in result position of a signature
<code>rep e, mk s e</code>	<code>Expr.rep, Expr.mk</code>	observe a concept’s representation; construct a concept value under a grant
<code>declRef d</code>	<code>Expr.declRef</code>	a reference to a declaration by identity
<code>delay init e</code>	<code>Expr.delay</code>	last activation’s value of e in the own domain, <code>init</code> before
<code>sync c init e</code>	<code>Expr.sync</code>	the value of e in domain c at c ’s last activation strictly before now, <code>init</code> if none
<code>fold f z l</code>	<code>Expr.fold</code>	the list recursor, the one term former that applies a function value
$\Theta; \Delta; G; \Gamma \vdash e : \tau$	<code>HasType $\Theta \ \Delta \ G \ \Gamma \ e \ \tau$</code>	typing; reads Δ only through the type view
<code>Satisfies ev $\Theta \ \Delta \ \Gamma \ e \ S$</code>	<code>Satisfies</code>	e has S ’s type under S ’s grant and evidence discharges every commitment
<code>GlobalWF ev $\Theta \ \Delta$</code>	<code>GlobalWF</code>	every stored declaration sits under its identity and its realization satisfies its interface
$\sqsubseteq, \text{InterfaceRefines}$	<code>InterfaceRefines</code>	same type, commitments grow
<code>DeclLeq, EnvRefines</code>	<code>DeclLeq, EnvRefines</code>	the structural refinement order on declarations and environments
<code>Ev $\Delta \ I \ t \ p \ e \ v$</code>	<code>Ev</code>	single-domain tick-indexed evaluation under input I and local environment ρ
<code>MEv $S \ \Delta \ I \ c \ t \ p \ e \ v$</code>	<code>MEv</code>	multi-domain evaluation in domain c under schedule S
I	<code>Input : DeclId → Nat → Value</code>	the input stream: the environment’s value for every unresolved declaration
S	<code>Sched : ClockId → Nat → Bool</code>	which domains activate at which global ticks
<code>prevAct $S \ c \ t$</code>	<code>prevAct</code>	c ’s last activation strictly before t
K	<code>ClockEnv : DeclId → Option ClockId</code>	each declaration’s timing domain; none is domain-agnostic
<code>Clocked K c e</code>	<code>Clocked</code>	the domain judgment

symbol	Lean object	meaning
Causal Δ	Causal	the instantaneous dependency graph is acyclic, witnessed by a rank
Ω	OutputEnv : OutputId $\rightarrow$ Option OutputSpec	each physical sink's accepted type and clock
β	DriveEnv : DeclId $\rightarrow$ Option OutputId	the drive edges, write-once per declaration
DriveWF, SingleDriver, CompleteOutputs	—	the output well-formedness conditions
Source Δ d	Source	realizationOf d = none: the environment provides d
CTy, elim, encode, decode	Surface/UnitDomain.lean	canonical interface types above the kernel and the normalization to kernel types
Channel, DeviceProfile, Provision	Surface/Provision.lean	a transducer with its transfer function; a raw type with channels; a raw declaration with its channel assignment
provision Δ P, induced Δ P I'	Surface/Provision.lean	the provisioned design; the abstract input induced by a raw input
Hardware, Requirement, Assignment, ValidFor, solve	Validation/Hardware.lean	the target table, the design's needs, an allocation, its validity, the solver
CapacitySufficient, requiredCapacity	Validation/Capacity.lean	window capacity under a schedule

On categorical language. Where the text says that the canonical types form an interface layer whose normalization is inverse to encoding, or that compatible charts form a *groupoid* of affine isomorphisms, the claims are theorem-level statements in the development (decode_encode, convert_compose, convert_inverse) and the categorical words are explanatory. No categorical framework is formalized; “realization as a functor preserving composition” and “natural transformations between elaborations” are intuitions for future work, not results.

Appendix B — Theorem and result index, by concept

Every name is a Lean declaration in `KCN-judu/BDL_FV` (the file is given per group; theorem names are unqualified where the namespace is `BDL`). *Kind* is **T** theorem, **C** counterexample (a theorem whose content is a rejection), **X** executed example (`decide/#eval`), **D** definition-level fact (`rfl/Iff.rfl`, reported as such). Hypotheses that restrict a result are in the *scope* column; a theorem with no scope note holds as stated in Parts III–X.

Refinement and client stability (Core/Interface, Core/Decl, Core/Satisfaction, Core/Env)

name	kind	states	scope
<code>InterfaceRefines_iff_semantic</code>	T	the syntactic refinement order is complete for abstract evidence	—
<code>DeclRefinesStar_iff</code>	T	the lifecycle closure is the structural order plus well-formedness of the target	—
<code>naive_breaks_wellformedness</code>	C	strengthening a realized interface without re-verification breaks well-formedness	—
<code>local_refinement_preserves_global_typing</code>	T	refining one declaration preserves every typing judgment	the structural order only
<code>local_refinement_preserves_global_wf</code>	T	refining one declaration preserves global well-formedness	<code>Evidence.Monotone</code>
<code>badEv_not_mono</code>	C	non-monotone evidence is destroyed by a valid realization step	—
<code>EnvRefines_update</code>	T	an update at an identity is an environment refinement	—
<code>HasType.mono_env</code> , <code>HasType.mono_concept</code> , <code>HasType.mono_grant</code>	T	typing is monotone in the three environments	—

Typing, unfolding and dependency (Core/Typing, Core/Dependency)

name	kind	states	scope
<code>infer_sound</code> , <code>infer_complete</code> , <code>HasType.unique</code>	T	inference is sound, complete and unique	—
<code>constructs_granted</code>	T	a well-typed term constructs a concept only where granted	—
<code>Unfolds.exists_of_acyclic</code> , <code>Unfolds.not_of_cyclic</code>	T	unfolding exists iff the reference graph is acyclic	delay-free fragment
<code>Acyclic.not_cyclic</code>	T	the rank witness excludes cycles	—
<code>unfolds_preserves_eval</code>	T	unfolding agrees with tick evaluation	first-order wiring designs

Semantic identity, representation and the grant (Core/Base, Experiments/SemanticTypeAlternatives, Experiments/RepresentationBindingAlternatives)

name	kind	states	scope
<code>no_semantic_value_without_declaration</code>	T	a semantic value originates only in a declaration of semantic type	—
<code>temporal_state_preserves_semantic_identity</code>	T	delay carries a concept’s tag and never creates one	—

name	kind	states	scope
HasType.erase, erase_not_injective, baseline_is_erased_modelA	T	a well-typed semantic term is well typed at its representation; the numeric baseline is the erasure	representations are data
semantic_rename_preserves_identity, rename_under_name_identity_breaks_client	T/C	display names are not identity; name-as-identity makes renaming destructive	—
bweak_evaded_by_eta	C	the <i>semanticRole</i> metadata checker is evaded by η -expansion	—
unrestricted_representation_binding_bypasses_semantic_identity, hidden_crossing_inside_unrelated_body	C	unrestricted mk/rep admits a hidden concept crossing	—
hidden_crossing_rejected_under_grant, representation_binding_does_not_enable_hidden_semantic_mapping	T	the hidden crossing is refused under the grant of an unrelated declaration	—
representation_change_is_edit_not_refinement	C	rebinding a concept's representation breaks existing realizations	—

Quantities and dimensions (Core/Base, Experiments/DimensionAlternatives)

name	kind	states	scope
dimension_mismatch_rejected	T	length + time is ill-typed	—
counterexampleB_baseline_accepts_length_plus_time	C	erasing dimensions accepts it	—
same_dimension_does_not_imply_same_semantic_identity, explicit_semantic_mapping_uses_dimensioned_formula	T	Tilt and MotorAngle at q Angle stay distinct	—

Reactive semantics (Core/Reactive, Experiments/ReactiveAlternatives)

name	kind	states	scope
Ev.det	T	evaluation is a partial function	—
reactive_total, fundamental	T	every declaration has a value at every tick	causal, globally well formed, well-typed inputs
evalF_sound	T	the interpreter is sound for Ev	—
Ev.not_of_strictCyclic	C	a strict cycle has no value	strict cycles only
Ev.tag_provenance	T	no signature announces a concept and no input carries it $\Rightarrow$ no value carries it	—
delay_not_under_binder, sync_not_under_binder	T	memory and transport are typed only in the empty context	—
arrow_not_delayable	T	nothing of function type can be delayed or transported	—
first_tick_undefined_without_init, first_tick_nondeterministic_without_init	C	delay without an initial value	toy relations

Clock domains and transport (Core/Clock, Experiments/ClockAlternatives)

name	kind	states	scope
MEV.det, multi_domain_total, mfundamental	T	multi-domain evaluation is deterministic and total	causal, globally well formed
delay_is_sync_own	T	delay is sync at the own domain	—

name	kind	states	scope
single_domain_embedding	T	under the always-active schedule MEv is Ev	—
scheduling_order_observable	C	same-tick cross-domain visibility makes the scheduler order observable	—
equal_rate_not_same_domain	C	a clone of a domain with the same schedule is a different domain	—
clocked_type_forces_polymorphism	C	a clock-indexed type forces polymorphism on every pure mapping	—
opt_loses_multiplicity_under_sync	C	sync as an event transport loses multiplicity	—
buffer_from_log_and_cursor	T	the window equals log-now minus log-at-previous-activation	—
policies_lose_information	T	latest, count, count+latest identify distinct windows	—

Lists, the buffer and capacity (Core/ListData, Surface/Buffer, Validation/Capacity, Experiments/BufferAlternatives)

name	kind	states	scope
buffer_window_correspondence (Theorem M)	T	the five declarations compute the window at every tick	src an input
buffer_elaboration_well_typed, buffer_elaboration_well_clocked	T	the elaboration types and clocks	—
buffer_lossless, window_to_list_preserves_order, window_to_list_preserves_multiplicity	T	the list summary is injective, ordered, multiplicity-preserving	—
bounded_summary_not_lossless, lossless_iff_injective	T	any summary of the newest k entries is lossy; lossless $\Leftrightarrow$ injective	—
latest_not_lossless, count_not_lossless, sum_not_lossless, modelD_not_lossless	C	models A–D lose windows	—
requiredCapacity_sufficient, periodic_window_bound, periodic_capacity_sufficient	T	the least sufficient capacity; one period suffices for periodic schedules	—
sufficient_capacity_preserves, bounded_buffer_agrees	T	under sufficient capacity every overflow policy is the identity	—
negE, negF, buffer_trace, transport_trace, latest_transport_loses	X	executed windows, overflow, clone-domain refusal, the component transport	—

Products, the recursor, equality and order (Core/Base, Surface/Poly, Experiments/PolyAlternatives)

name	kind	states	scope
Ty.prod_data	T	a pair is data iff both parts are	—
church_fst_rank, church_pair_prenex_one_projection	C	Church pairs need rank 2	toy System F
fold_total, mfold_total	T	the recursor is total on finite lists	—
Value.beq_iff	T	structural equality is equality on first-order values	—

name	kind	states	scope
Cap.eq_iff_data, Cap.ord_data, Cap.ord_not_data_converse	T	Data $\Rightarrow$ Eq; Ord $\Rightarrow$ Data; not conversely	—
lt_rejected, min_mode_rejected, lt_only_on_quantities	C	order on modes, pairs, lists, options, booleans is refused	—
minBy_recovers_min	T	the comparator escape hatch loses nothing	—

The equation library and rank-1 polymorphism (Surface/Poly, Surface/Stdlib, Surface/Generic, Experiments/EquationExamples)

name	kind	states	scope
matchTy_sound, matchTy_ complete	T	one-way matching is sound and complete	—
Scheme.instantiate_sound	T	an instantiated scheme checks its capabilities	—
instances_are_monomorphic	T	three uses of min are three kernel terms	—
applyBoth_rank, existential_ rank, applyBoth_replacement	C	higher-rank uses have rank-1 replacements	toy System F
HasType.comb_irrelevant, lib_ eval_context_free, lib_clocked, Comb.noConstruct, lib_ expansion	T	a combinator's typing, value and clock are context-free; it constructs nothing; inlining is sound	—
fold_spec, any_spec, all_spec, contains_spec, map_spec, filter_spec, min_spec, max_spec, clamp_spec, inRange_spec	T	each entry computes the mathematical function	the predicate value implements a Boolean function
forall_in_list, exists_in_list, oneOf_mem, oneOf_dup_ irrelevant	T	finite quantification and set membership are folds	—
generic_preserves_identity, generic_preserves_dimension, pair_projections_keep_ concepts, map_keeps_concepts, eq_across_concepts_rejected	T	nominality and dimensions survive generics	—
exA ... exM	X	the thirteen required equation cases	—

Natural surface (Surface/Natural, Experiments/NaturalExamples)

name	kind	states	scope
elab_local_nearest, elab_ shadow, elab_unbound, elab_core	T	scoping of binder locals	—
desugar_rename, alpha	T	alpha-equivalence of the elaboration	—
desugar_constructs	T	no mk is introduced	—
binder_local_type, range_ bounds_forced	T	the local's type is the element type; range bounds at the nominal type	—
binder_all_eval, binder_any_ eval, binder_map_eval, binder_ filter_eval, natural_forall, natural_exists, range_eval	T	evaluation is the library's	—
binder_clock, range_clock	T	clocks are the operands'	—

Units, coordinates and charts (Surface/Units, Surface/Composer, Surface/Affine, Surface/Rational, Surface/Charts)

name	kind	states	scope
inUnitE_typed, inUnitE_safe, withUnitE_typed, withUnitE_is_quantity	T	coordinate extraction and quantity construction type as stated	—
convert_eq, convert_trans, convert_self, inUnit_mismatch	T	conversion is the composition; transitive; identity; never coerces	—
inUnit_withUnit, withUnit_inUnit, inUnit_withUnit_nat, withUnit_inUnit_nat	T	round trips, exact over the symbolic group; the Nat laws	the second Nat law needs the scale to divide the magnitude
unitOps_no_construction, nominal_distinct	T	unit operations construct nothing; concepts stay distinct under round trips	—
unitsFor_sound, unitsFor_complete	T	the registry's units for a dimension	—
presentation_irrelevant_typing, presentation_irrelevant_eval, presentation_changes_display, display_may_identify_distinct	D/T	presentation is semantically irrelevant	—
normalizations_agree	T	tilt / 90 deg and inUnit(tilt, deg) / 90 agree	—
solve_sound, solve_complete, candidates_sound, candidates_complete, slot_sound, refCandidates_sound	T	local dimension inference and candidate units	the + - × ÷ fragment with holes
celsius_not_linear, sum_of_points_is_not_a_point, sum_well_typed, delta_is_linear, delta_candidates_need_sort	C/T	the linear model cannot represent °C; absolute-value arithmetic is the problem	—
affLitE_typed, affInUnitE_typed, affine_roundtrip_ev, scales_agree	T	affine literals and coordinates elaborate exactly	K/180 basis
chart_left_inverse, chart_right_inverse, convert_is_affine, convert_identity, convert_compose, convert_inverse	T	the chart and groupoid laws	any field
difference_map, difference_offset_cancels, difference_converts_linearly, linear_part_identity, linear_part_compose, not_additive_of_offset	T	differences carry the linear part; offsets are not additive	any field
unit_eraser_preserves_conversion_structure, display_switch_preserves_quantity, coordinate_edit_changes_quantity, coordinate_needs_chart	T	erasure keeps conversion; the coordinate is chartless	—
sort_orthogonal_to_conversion, conversion_orthogonal_to_sort	D	the point/difference sort and conversion are independent	—
CtoF_closed, FtoC_closed, exH, exI	X	Celsius/Fahrenheit, the ADC calibration, the encoder offset, exactly	—

Physical outputs (Core/Output, Experiments/OutputAlternatives)

name	kind	states	scope
multiple_direct_drivers_rejected	C	two drivers of one sink violate <code>SingleDriver</code>	—
single_driver_output_deterministic	T	with one driver the physical output is unique where it exists	—
first_output_binding_is_monotone	T	binding an unbound declaration to an undriven sink is a refinement	—
hidden_arbitration_observable	C	first-wins, last-wins and maximum give three outputs	—
driver_is_unit_domain	T	a driver of a concept-accepting sink is unit-domain	—

Unit domain and the source role (Surface/UnitDomain, Experiments/UnitDomainExamples)

name	kind	states	scope
elim_canonical, encode_decode, decode_encode, canonicalOfKernel_encode, encode_injective, canonical_injective	T	the normalization is inverse to encoding	signatures whose output is not an arrow
zero_input_obligation	D	the realization obligation of $() \rightarrow B$ is $\vdash e : B$	—
zero_input_memory, lams_typed	T	memory is legal in a zero-input declaration; a formula body is a realization under n binders	—
refForms_agree, HasType.refForms, Clocked.refForms	D	$f, f(), f(())$ are one term	—
Ev.declRef_env_irrelevant, MEv.declRef_env_irrelevant, same_tick_same_value	T	a reference's value is independent of the local environment; two readings in one tick agree	—
source_value, source_reads, resolved_not_source, SimulationInput.value	T	a Source's value is the input's; a resolved $() \rightarrow A$ never reads it	—
transport_needs_unit_domain, delay_needs_unit_domain	T	only a unit-domain declaration can be transported or remembered	—
unit_codomain_collapse, consumers_indistinguishable, eval_independent_of_drives	T	$A \rightarrow ()$ cannot name a receiver	denotational
homUnit	D	$(1 \rightarrow \beta) \equiv \beta$	—

Source provision (Surface/Provision, Experiments/ProvisionExamples)

name	kind	states	scope
realization_checked_under_own_grant, grant_of_sem, channel_constructs_nothing	T	the grant boundary of the profile	—
Channel.WF_refFree, pure_iff_delayFree_of_wf	T	typing in the empty design forbids references; purity is delay-freedom	—
exD	X	a typed impure transducer is tick-dependent	—
provision_envRefines, provision_tyView_eq, realizeAt_typed	T	provision is an environment refinement	WF

name	kind	states	scope
provision_wf	T	the provisioned design is globally well formed	monotone evidence; evidence for the targets' commitments
provision_causal, provision_wellClocked	T	causality and clocks are preserved, $\kappa \ r = \kappa \ s$	NoMention $\Delta \ r$
simulate	T	the design-simulation lemma behind transparency	closures avoid r
provision_transparent, provision_transparent_typed, provision_decl_transparent, provision_physicalOutput	T	the design cannot tell the provisioned environment from the abstract one	RawInput, NoMention, $r \notin e.\text{refs}$
provision_abstracts	T	every provisioned trace is an abstract trace	as above
provision_exact, JointSection.one	T	trace equality under a joint section	JointSection
exE, sat_never_451	X	the strict case: a saturating ADC	—
no_joint_witness, exF	C/X	shared raw readings: pointwise surjectivity is not enough	—
provision_not_reapplicable, provision_idem_total, provision_reprovision_not_refinement, provision_source_role	T	not re-applicable; the totalized function is idempotent only trivially	—
provision_comm, provision_perm	T	independent provisions commute; the assignment is a set	independence; nodup keys
Transduces.mev, MEV.of_ev_pure, Ev.pure	T	a pure term's canonical evaluation transfers to any design, input, domain and tick	—

Hardware validation (Validation/Hardware, Experiments/HardwareAlternatives)

name	kind	states	scope
solve_sound, solve_complete, satisfiable_iff_solve	T	the solver is sound and complete; feasibility is decidable	finite instances, unary and binary constraints
Hardware.Extends preservation	T	extending a target preserves valid assignments	—
motor_control_sat_on_nano, seven_pwm_unsat_on_nano, seven_pwm_sat_on_big, multifunction_overlap_unsat, seven_pwm_design_semantically_valid	X	the Nano case study and its two counterexamples	—
diagnose	D	a first dead end under greedy placement	not a minimal core

Behavior systems and groups (Behavior/*, Experiments/BehaviorAlternatives, Experiments/GroupAlternatives)

name	kind	states	scope
HasType.rename, Satisfies.rename, Clocked.rename	T	the judgments are equivariant under renaming	—
inst_decl_disjoint (Theorem A)	T	instances never share identities	—
binding_satisfies (Theorem C)	T	a binding is an ordinary realization the checker verifies	—

name	kind	states	scope
flatten_WF, flatten_causal, flatten_wellClocked, flatten_ singleDriver, open_port_stays_ open (Theorems D–I)	T	the flattening is a well-formed design	ComposeWF; InstAcyclic for causality; Monotone, Equivariant, PortSound evidence
eval_flat_to_inst, eval_inst_ to_flat, modular_iff_flat (Theorem J)	T	modular and flat evaluation agree	restricted: single domain, wiring designs, closure-free inputs, direct/constant bindings
substitute_composeWF	T	replacing an instance by a refining component preserves composition	—
group_is_identity_on_design and Theorems A–G	D	every group operation is the identity on the design	—
mem_crossIn, mem_crossOut, internal_not_crossIn, internal_only_not_crossOut, socket_no_fanout (Theorems H–L)	T	boundaries are projections; sockets add no dependency	finite enumeration
restrict_realizes, system_ composeWF, flat_WF, flat_causal, private_unobservable, provided_iff (Theorems M–Q)	T	extraction yields a well- formed system	Evidence.InterfaceLocal
orig_iff_flat (Theorem R)	T	extraction preserves behavior	restricted: single-domain wiring designs, no transported bindings

Appendix C — Decision index

Every formal design decision by its stable identifier, with the Part of this document that discusses it, the Lean files that carry its evidence, and the production record it supports, audits or bears on. The records themselves — choice, alternatives rejected, formal reason — are docs/decisions/ in the formal repository; the map from the retired ledger numbers is Appendix E. Status is *accepted* for every record; FVD-0106 carries a Phase-10b amendment and FVD-0098 records the partial reversal of FVD-0090’s lt generalisation.

id	decision	status	main section	formal evidence	production correspondence
FVD-0001	Commitments are atomic labels; evidence is abstract	accepted	Part III	Phase 0: Core/Interface, Core/Satisfaction	ADR-0010 (supports)
FVD-0002	InterfaceRefines freezes the type and grows commitments; not logical implication	accepted	Part III	Phase 0: Core/Interface, Core/Satisfaction	ADR-0010 (supports)
FVD-0003	strengthen on a realized declaration carries a re-verification premise	accepted	Part III	Phase 0: Core/Interface, Core/Satisfaction	ADR-0010 (supports)
FVD-0004	DeclInterface. commitments is a List, not a Finset	accepted	Part III	Phase 0: Core/Interface, Core/Satisfaction	ADR-0010 (supports)
FVD-0005	References are by DeclId inside Expr (declRef), not a separate DesignExpr	accepted	Part III	Phase 1: Core/Base, Core/Decl, Core/Typing, Core/Env, Core/Dependency	ADR-0010 (supports)
FVD-0006	The declRef typing rule reads $\Delta. \text{tyView}$ only	accepted	Part III	Phase 1: Core/Base, Core/Decl, Core/Typing, Core/Env, Core/Dependency	ADR-0010 (supports)
FVD-0007	Realization is write-once; “detach” is not a refinement step	accepted	Part III	Phase 1: Core/Base, Core/Decl, Core/Typing, Core/Env, Core/Dependency	ADR-0009 (supports)
FVD-0008	The structural order (DeclLeq, EnvRefines) is separated from the invariant (GlobalWF)	accepted	Part III	Phase 1: Core/Base, Core/Decl, Core/Typing, Core/Env, Core/Dependency	ADR-0010 (supports)
FVD-0009	Evidence takes the environment as an argument	accepted	Part III	Phase 1: Core/Base, Core/Decl, Core/Typing, Core/Env, Core/Dependency	ADR-0010 (supports)
FVD-0010	Evidence.Monotone is a kernel-imposed constraint on the validation layer	accepted	Part III	Phase 1: Core/Base, Core/Decl, Core/Typing, Core/Env, Core/Dependency	ADR-0010 (supports)
FVD-0011	Semantics at Phase 1 is unfolding to a reference-free term	accepted	Part III	Phase 1: Core/Base, Core/Decl, Core/Typing, Core/Env, Core/Dependency	ADR-0010 (supports)
FVD-0012	Acyclicity is witnessed by a rank function	accepted	Part III	Phase 1: Core/Base, Core/Decl, Core/Typing, Core/Env, Core/Dependency	ADR-0010 (supports)

id	decision	status	main section	formal evidence	production correspondence
FVD-0013	Phase 0's Artifact (bare list of ids) removed	accepted	Part III	Phase 1: Core/Base, Core/Decl, Core/Typing, Core/Env, Core/Dependency	FV-only
FVD-0014	Display names are not in the kernel	accepted	Part III	Phase 1: Core/Base, Core/Decl, Core/Typing, Core/Env, Core/Dependency	ADR-0008 (supports)
FVD-0015	Kernel ontology migrated from holes to declarations	accepted	Part III	Phase M: Core/* (the rename)	ADR-0010 (supports)
FVD-0016	Monotone refinement is distinct from arbitrary editing	accepted	Part III	Phase M: Core/* (the rename)	ADR-0009 (supports)
FVD-0017	Commitments and validation obligations share PropertyId for now	accepted	Part III	Phase M: Core/* (the rename)	ISS-0003 (bears-on)
FVD-0018	Evidence.Monotone is a stability condition, not a definition of validity	accepted	Part III	Phase M: Core/* (the rename)	ISS-0003 (bears-on)
FVD-0019	Semantic identity lives in the type: $Ty.sem : SemanticId \rightarrow Ty$	accepted	Part IV	Phase 2: Core/Base, Experiments/SemanticTypeAlternatives	ADR-0013 (supports)
FVD-0020	SemanticId is independent of DeclId and of display names	accepted	Part IV	Phase 2: Core/Base, Experiments/SemanticTypeAlternatives	ADR-0013 (supports)
FVD-0021	No introduction/elimination forms for semantic types in Phase 2	accepted	Part IV	Phase 2: Core/Base, Experiments/SemanticTypeAlternatives	ADR-0013 (supports)
FVD-0022	Explicit semantic mappings are ordinary declarations	accepted	Part IV	Phase 2: Core/Base, Experiments/SemanticTypeAlternatives	ADR-0013 (supports)
FVD-0023	Semantic identity change is an edit	accepted	Part IV	Phase 2: Core/Base, Experiments/SemanticTypeAlternatives	ADR-0013 (supports)
FVD-0025	Future constraint for Phase 3: representation binding must not defeat semantic identity	accepted	Part IV	Phase 2: Core/Base, Experiments/SemanticTypeAlternatives	ADR-0013 (supports)
FVD-0024	Canonical closed inhabitants replaced by unresolved declarations	accepted	Part IV	Phase 2: Core/Base, Experiments/SemanticTypeAlternatives	ADR-0013 (supports)
FVD-0026	Unrestricted symmetric mk/rep rejected	accepted	Part IV	Phase 3: Core/Decl, Core/Typing, Experiments/	ADR-0013 (supports)

id	decision	status	main section	formal evidence	production correspondence
				RepresentationBindingAlternatives, Experiments/DimensionAlternatives	
FVD-0027	Representation types are sem-free (ConceptEnv.WF)	accepted	Part IV	Phase 3: Core/Decl, Core/Typing, Experiments/RepresentationBindingAlternatives, Experiments/DimensionAlternatives	ADR-0013 (supports)
FVD-0028	Observation is unrestricted; construction is licensed by the realized declaration's signature	accepted	Part IV	Phase 3: Core/Decl, Core/Typing, Experiments/RepresentationBindingAlternatives, Experiments/DimensionAlternatives	ADR-0013 (supports)
FVD-0029	Representation binding is a separate, write-once concept environment θ	accepted	Part IV	Phase 3: Core/Decl, Core/Typing, Experiments/RepresentationBindingAlternatives, Experiments/DimensionAlternatives	ADR-0013 (supports)
FVD-0030	Unfolding preserves typing under Grant.all , not under the client grant	accepted	Part IV	Phase 3: Core/Decl, Core/Typing, Experiments/RepresentationBindingAlternatives, Experiments/DimensionAlternatives	ADR-0013 (supports)
FVD-0031	Dimensions in $\text{Ty as } q \text{ d; algebra in } \text{Prim.ty}$; no dimension rule	accepted	Part IV	Phase 3: Core/Decl, Core/Typing, Experiments/RepresentationBindingAlternatives, Experiments/DimensionAlternatives	ADR-0011 (supports), ADR-0013 (supports)
FVD-0032	Units are surface: elaborated to scaled dimensioned literals	accepted	Part IV	Phase 3: Core/Decl, Core/Typing, Experiments/RepresentationBindingAlternatives, Experiments/DimensionAlternatives	ADR-0028 (supports)
FVD-0033	Semantic identity is not indexed by dimension	accepted	Part IV	Phase 3: Core/Decl, Core/Typing, Experiments/RepresentationBindingAlternatives, Experiments/DimensionAlternatives	ADR-0013 (supports)
FVD-0034	One temporal primitive: delay init e	accepted	Part VII	Phase 4: Core/Reactive, Experiments/ReactiveAlternatives	ADR-0004 (supports), ADR-0016 (supports)
FVD-0035	delay is data-typed and top-level (empty context)	accepted	Part VII	Phase 4: Core/Reactive, Experiments/ReactiveAlternatives	ADR-0004 (supports), ADR-0016 (supports)
FVD-0036	Signal is not a type; Event is opt	accepted	Part VII	Phase 4: Core/Reactive, Experiments/ReactiveAlternatives	ADR-0004 (supports), ADR-0016 (supports)
FVD-0037	Causality replaces blanket acyclicity:	accepted	Part VII	Phase 4: Core/Reactive,	ADR-0004 (supports),

id	decision	status	main section	formal evidence	production correspondence
	Causal on InstDependsOn			Experiments/ ReactiveAlternatives	ADR-0016 (supports)
FVD-0038	Explicit initial value on every delay	accepted	Part VII	Phase 4: Core/ Reactive, Experiments/ ReactiveAlternatives	ADR-0004 (supports), ADR-0016 (supports)
FVD-0039	State has no identity; state is structural	accepted	Part VII	Phase 4: Core/ Reactive, Experiments/ ReactiveAlternatives	ADR-0004 (supports), ADR-0016 (supports)
FVD-0040	Representation types are data (ConceptEnv.WF strengthened)	accepted	Part VII	Phase 4: Core/ Reactive, Experiments/ ReactiveAlternatives	ADR-0004 (supports), ADR-0016 (supports)
FVD-0041	Temporal changes are realization edits	accepted	Part VII	Phase 4: Core/ Reactive, Experiments/ ReactiveAlternatives	ADR-0004 (supports), ADR-0016 (supports)
FVD-0042	The reactive semantics is a relation, not yet a machine	accepted	Part VII	Phase 4: Core/ Reactive, Experiments/ ReactiveAlternatives	ADR-0004 (supports), ADR-0016 (supports)
FVD-0043	Time is one global tick with a schedule; no rates, no timestamps in the kernel	accepted	Part VII	Phase 5: Core/Clock, Experiments/ ClockAlternatives	ADR-0004 (supports)
FVD-0044	Nominal ClockId, stored per declaration in ClockEnv K; none = domain-agnostic pure mapping	accepted	Part VII	Phase 5: Core/Clock, Experiments/ ClockAlternatives	ADR-0004 (supports)
FVD-0045	One transport primitive sync src init e, reading strictly before	accepted	Part VII	Phase 5: Core/Clock, Experiments/ ClockAlternatives	ADR-0004 (supports)
FVD-0046	The clock is interface data held in a projection, not a record field	accepted	Part VII	Phase 5: Core/Clock, Experiments/ ClockAlternatives	ADR-0004 (supports)
FVD-0047	Rates, drift, jitter, latency, buffer capacity, value age are validation	accepted	Part VII	Phase 5: Core/Clock, Experiments/ ClockAlternatives	ADR-0004 (supports)
FVD-0048	Event transport = window read; buffering derived, Event still not a primitive	accepted	Part VII	Phase 5: Core/Clock, Experiments/ ClockAlternatives	ISS-0001 (bears-on)
FVD-0049	The logical relation is generic in the application relation	accepted	Part VII	Phase 5: Core/Clock, Experiments/ ClockAlternatives	FV-only
FVD-0050	Physical sinks have nominal identity (OutputId), separate from SemanticId and DeclId	accepted	Part IX	Phase 6: Core/ Output, Experiments/ OutputAlternatives	ADR-0005 (supports)

id	decision	status	main section	formal evidence	production correspondence
FVD-0051	A drive edge is a per-declaration write-once projection β , checked by type and clock equality	accepted	Part IX	Phase 6: Core/ Output, Experiments/ OutputAlternatives	ADR-0005 (supports)
FVD-0052	Single-driver is a global invariant; completeness is the executable condition	accepted	Part IX	Phase 6: Core/ Output, Experiments/ OutputAlternatives	ADR-0005 (supports)
FVD-0053	No runtime arbitration, no implicit priority, no merge policy	accepted	Part IX	Phase 6: Core/ Output, Experiments/ OutputAlternatives	ADR-0005 (supports)
FVD-0054	Effect rows and action values rejected for this kernel	accepted	Part IX	Phase 6: Core/ Output, Experiments/ OutputAlternatives	ADR-0005 (supports)
FVD-0055	First binding is a refinement; rebinding is an edit; a second driver is invalid	accepted	Part IX	Phase 6: Core/ Output, Experiments/ OutputAlternatives	ADR-0005 (supports)
FVD-0056	Sinks are terminal	accepted	Part IX	Phase 6: Core/ Output, Experiments/ OutputAlternatives	ADR-0005 (supports)
FVD-0057	Hardware feasibility is a validation layer over Design $\times$ Target, not typing	accepted	Part X	Phase 7: Validation/ Hardware, Experiments/ HardwareAlternatives	ADR-0006 (supports), ADR-0015 (supports)
FVD-0058	Resources carry capabilities and per-capability units; sharing is a per-capability policy	accepted	Part X	Phase 7: Validation/ Hardware, Experiments/ HardwareAlternatives	ADR-0006 (supports), ADR-0015 (supports)
FVD-0059	Requirements are independent variables with nominal RequirementId, optional fixed resource, optional unit relation	accepted	Part X	Phase 7: Validation/ Hardware, Experiments/ HardwareAlternatives	ADR-0006 (supports), ADR-0015 (supports)
FVD-0060	Validity is unary support plus pairwise compatibility; the solver is exhaustive DFS, proved sound and complete	accepted	Part X	Phase 7: Validation/ Hardware, Experiments/ HardwareAlternatives	ADR-0006 (supports), ADR-0015 (supports)
FVD-0061	Hardware extension is monotone; requirement extension,	accepted	Part X	Phase 7: Validation/ Hardware, Experiments/ HardwareAlternatives	ADR-0006 (supports), ADR-0015 (supports)

id	decision	status	main section	formal evidence	production correspondence
	strengthening, fixing, and resource removal are revalidation triggers				
FVD-0062	Deployment feasibility is environment-sensitive evidence, not Evidence. Monotone	accepted	Part X	Phase 7: Validation/ Hardware, Experiments/ HardwareAlternatives	ADR-0006 (supports), ADR-0015 (supports)
FVD-0063	Numeric electrical/timing constraints deferred	accepted	Part X	Phase 7: Validation/ Hardware, Experiments/ HardwareAlternatives	FV-only
FVD-0064	Behaviour components are surface objects; the kernel is unchanged	accepted	Part VIII	Phase 8a: Behavior/ *, Experiments/ BehaviorAlternatives	ADR-0021 (supports), ADR-0022 (supports)
FVD-0065	A port is a template declaration by identity, with its public interface and clock	accepted	Part VIII	Phase 8a: Behavior/ *, Experiments/ BehaviorAlternatives	ADR-0021 (supports), ADR-0022 (supports)
FVD-0066	Instantiation renames every identity the template owns; concepts and sinks are partitioned into internal (fresh) and global (shared)	accepted	Part VIII	Phase 8a: Behavior/ *, Experiments/ BehaviorAlternatives	ADR-0021 (supports), ADR-0022 (supports)
FVD-0067	Binding is a Phase-1 realization step	accepted	Part VIII	Phase 8a: Behavior/ *, Experiments/ BehaviorAlternatives	ADR-0021 (supports), ADR-0022 (supports)
FVD-0068	Composition well-formedness is stated on interfaces, never on bodies	accepted	Part VIII	Phase 8a: Behavior/ *, Experiments/ BehaviorAlternatives	ADR-0021 (supports), ADR-0022 (supports)
FVD-0069	Causality across instances is a validation condition on the inter-instance direct-binding graph	accepted	Part VIII	Phase 8a: Behavior/ *, Experiments/ BehaviorAlternatives	ADR-0021 (supports), ADR-0022 (supports)
FVD-0070	Clock parameters are nominal variables substituted by κ at instantiation; rates never enter	accepted	Part VIII	Phase 8a: Behavior/ *, Experiments/ BehaviorAlternatives	ADR-0021 (supports), ADR-0022 (supports)

id	decision	status	main section	formal evidence	production correspondence
FVD-0071	Evidence must be equivariant and port-sound	accepted	Part VIII	Phase 8a: Behavior/ *, Experiments/ BehaviorAlternatives	ADR-0021 (supports), ADR-0022 (supports)
FVD-0072	Hierarchy is packaging, not a tree constructor	accepted	Part VIII	Phase 8a: Behavior/ *, Experiments/ BehaviorAlternatives	ADR-0021 (supports), ADR-0022 (supports)
FVD-0073	Theorem J is proved on the single-domain wiring fragment with direct/constant bindings	accepted	Part VIII	Phase 8a: Behavior/ *, Experiments/ BehaviorAlternatives	ADR-0021 (supports)
FVD-0074	A behaviour group is authoring metadata; eraseGroups is a projection	accepted	Part VIII	Phase 8b: Behavior/ Group, Boundary, Extract, ExtractPreservation	ADR-0019 (supports)
FVD-0075	Group operations are semantic no-ops, not refinements or edits	accepted	Part VIII	Phase 8b: Behavior/ Group, Boundary, Extract, ExtractPreservation	ADR-0019 (supports)
FVD-0076	Aggregate sockets are projections of DependsOn	accepted	Part VIII	Phase 8b: Behavior/ Group, Boundary, Extract, ExtractPreservation	ADR-0019 (supports)
FVD-0077	Boundary inference: required = crossing-in, provided = crossing-out, private = the rest without a sink, clocks = all clocks	accepted	Part VIII	Phase 8b: Behavior/ Group, Boundary, Extract, ExtractPreservation	ADR-0019 (supports)
FVD-0078	Extraction = two restrictions of the design reconnected by Phase-8a bindings	accepted	Part VIII	Phase 8b: Behavior/ Group, Boundary, Extract, ExtractPreservation	ADR-0019 (supports)
FVD-0079	Extraction causality is proved by subdividing the original graph, not by InstAcyclic	accepted	Part VIII	Phase 8b: Behavior/ Group, Boundary, Extract, ExtractPreservation	ADR-0019 (supports)
FVD-0080	Template realization needs interface-local evidence	accepted	Part VIII	Phase 8b: Behavior/ Group, Boundary, Extract, ExtractPreservation	ADR-0019 (supports)
FVD-0081	Identity: templates keep original identities; instances are fresh; the group id is never a component id	accepted	Part VIII	Phase 8b: Behavior/ Group, Boundary, Extract, ExtractPreservation	ADR-0019 (supports)

id	decision	status	main section	formal evidence	production correspondence
FVD-0082	Nested groups are a relation on the flat group list	accepted	Part VIII	Phase 8b: Behavior/ Group, Boundary, Extract, ExtractPreservation	ADR-0019 (bears-on), ISS-0007 (bears-on)
FVD-0083	τ_{list} is a kernel data type; the object-language buffer needs it and nothing else	accepted	Part VII	Phase 9a: Core/ ListData, Surface/ Buffer, Validation/ Capacity	ADR-0024 (supports)
FVD-0084	Six list operators, registered through $\text{Prim.ty: nil, cons, length, take, reverse, head}$	accepted	Part VII	Phase 9a: Core/ ListData, Surface/ Buffer, Validation/ Capacity	ADR-0024 (supports)
FVD-0085	The buffer is a surface elaboration into five declarations over delay/sync	accepted	Part VII	Phase 9a: Core/ ListData, Surface/ Buffer, Validation/ Capacity	ISS-0001 (bears-on)
FVD-0086	Capacity is validation; overflow policies are explicit; only rejecting the deployment preserves semantics	accepted	Part VII	Phase 9a: Core/ ListData, Surface/ Buffer, Validation/ Capacity	ADR-0027 (supports)
FVD-0087	Buffered transport is a Phase-8a binding choice, not a transport kind	accepted	Part VII	Phase 9a: Core/ ListData, Surface/ Buffer, Validation/ Capacity	ADR-0021 (supports)
FVD-0088	Products enter the kernel as value composition: $\text{prod, pair, fst, snd}$	accepted	Part V	Phase 9b: Surface/ Poly, Stdlib, Generic, Experiments/ EquationExamples	ADR-0025 (supports)
FVD-0089	The list recursor fold is a term former, not a registered operator	accepted	Part V	Phase 9b: Surface/ Poly, Stdlib, Generic, Experiments/ EquationExamples	ADR-0025 (supports)
FVD-0090	eq at every data type, with the data proof in the syntax	accepted	Part V	Phase 9b: Surface/ Poly, Stdlib, Generic, Experiments/ EquationExamples	ADR-0025 (supports)
FVD-0091	$\text{toList : opt } \tau \rightarrow \text{list } \tau$ and drop are registered operators	accepted	Part V	Phase 9b: Surface/ Poly, Stdlib, Generic, Experiments/ EquationExamples	ADR-0025 (supports)
FVD-0092	Rank-1 polymorphism is definitional: families instantiated by matching; no type variable in the kernel	accepted	Part V	Phase 9b: Surface/ Poly, Stdlib, Generic, Experiments/ EquationExamples	ADR-0025 (supports)

id	decision	status	main section	formal evidence	production correspondence
FVD-0093	Constraints: the closed vocabulary {Data}; no user-defined classes	accepted	Part V	Phase 9b: Surface/ Poly, Stdlib, Generic, Experiments/ EquationExamples	ADR-0025 (supports)
FVD-0094	The equation library is a set of combinators, inlined at use sites	accepted	Part V	Phase 9b: Surface/ Poly, Stdlib, Generic, Experiments/ EquationExamples	ADR-0025 (supports)
FVD-0095	Sets, intervals, records, predicates, finite quantifiers are surface	accepted	Part V	Phase 9b: Surface/ Poly, Stdlib, Generic, Experiments/ EquationExamples	ADR-0025 (supports)
FVD-0096	Sums are encoded; a kernel sum is deferred	accepted	Part V	Phase 9b: Surface/ Poly, Stdlib, Generic, Experiments/ EquationExamples	ISS-0005 (bears-on)
FVD-0097	Existentials are not needed: hiding is Phase-8a instantiation	accepted	Part V	Phase 9b: Surface/ Poly, Stdlib, Generic, Experiments/ EquationExamples	ADR-0025 (supports)
FVD-0098	Ordering is a quantity comparison; the kernel has no structural order	accepted	Part V	Phase 9c: Surface/ Poly (Cap)	ADR-0025 (supports), ADR-0026 (supports)
FVD-0099	The surface capability vocabulary is {Data, Eq, Ord}; Eq = Data today; Ord is by declaration	accepted	Part V	Phase 9c: Surface/ Poly (Cap)	ADR-0025 (supports), ADR-0026 (supports)
FVD-0100	Ordered library entries take Ordered evidence; comparators recover them	accepted	Part V	Phase 9c: Surface/ Poly (Cap)	ADR-0025 (supports), ADR-0026 (supports)
FVD-0101	Units remain entirely surface; coordinate extraction and quantity construction are elaborated quantity arithmetic	accepted	Part VI	Phase 10: Surface/ Units, Composer, Affine	ADR-0028 (supports)
FVD-0102	Unit semantics are stated exactly over an abstract scalar domain; the kernel's Nat and production's floats are models	accepted	Part VI	Phase 10: Surface/ Units, Composer, Affine	ADR-0028 (supports)
FVD-0103	A unit is an identity with a dimension and a	accepted	Part VI	Phase 10: Surface/ Units, Composer, Affine	ADR-0028 (supports)

id	decision	status	main section	formal evidence	production correspondence
	scale; spelling is presentation				
FVD-0104	The Formula Composer's formal basis is typed holes with local bidirectional dimension inference — no unification	accepted	Part VI	Phase 10: Surface/ Units, Composer, Affine	ADR-0028 (supports)
FVD-0105	Preferred display units are presentation, not design	accepted	Part VI	Phase 10: Surface/ Units, Composer, Affine	FV-only
FVD-0106	Affine conversion is complete as coordinate-change semantics; point/delta is optional physical-arithmetic validation	accepted	Part VI	Phase 10: Surface/ Units, Composer, Affine	ISS-0004 (bears-on)
FVD-0107	Unit coordinates are an erasure that preserves the affine coordinate change	accepted	Part VI	Phase 10b: Surface/ Rational, Charts	ISS-0004 (bears-on)
FVD-0108	Conversions form a groupoid of affine isomorphisms; differences carry the linear part	accepted	Part VI	Phase 10b: Surface/ Rational, Charts	ISS-0004 (bears-on)
FVD-0109	The exact scalar domain is a choice-free rational field built in the development	accepted	Part VI	Phase 10b: Surface/ Rational, Charts	ISS-0004 (bears-on)
FVD-0110	Unit conversion and sensor calibration are one affine-map abstraction	accepted	Part VI	Phase 10b: Surface/ Rational, Charts	ISS-0004 (bears-on)
FVD-0111	Binder syntax desugars to the Phase-9 library applied to a lambda; a binder local is the lambda parameter	accepted	Part V	Phase 11: Surface/ Natural	ADR-0028 (supports)
FVD-0112	Ranges are surface nodes desugared to <code>inRange</code> ; no interval type or value	accepted	Part V	Phase 11: Surface/ Natural	ADR-0028 (supports)
FVD-0113	No general comprehension, no general quantifier	accepted	Part V	Phase 11: Surface/ Natural	ADR-0028 (supports)
FVD-0114	<code>x ?? d</code> desugars to <code>getD</code>	accepted	Part V	Phase 11: Surface/ Natural	ADR-0028 (supports)
FVD-0115	The canonical type <code>domain(inputs) -> B</code>	accepted	Part IX	Phase 12: Surface/ UnitDomain	ADR-0029 (supports)

id	decision	status	main section	formal evidence	production correspondence
	with <code>domain([]) = ()</code> lives above the kernel; the kernel interface type is its normalization				
FVD-0116	<code>() -> B</code> is realized at B in the empty context; no unit binder	accepted	Part IX	Phase 12: Surface/UnitDomain	ADR-0029 (supports)
FVD-0117	<code>f</code> , <code>f()</code> , <code>f(())</code> are one reference; a reading is not a call	accepted	Part IX	Phase 12: Surface/UnitDomain	ADR-0029 (supports)
FVD-0118	The source role is a realization state, not a type shape and not a kind	accepted	Part IX	Phase 12: Surface/UnitDomain	ADR-0032 (supports)
FVD-0119	<code>A -> ()</code> is not a physical sink; the drive edge is	accepted	Part IX	Phase 12: Surface/UnitDomain	ADR-0029 (supports), ADR-0032 (supports)
FVD-0120	Transport and memory of a relationship require the unit domain — as corollaries	accepted	Part IX	Phase 12: Surface/UnitDomain	ADR-0029 (supports)
FVD-0121	Provision is a construction over designs, not a kernel construct	accepted	Part IX	Phase 13: Surface/Provision, Experiments/ProvisionExamples	PRP-0001 (audits)
FVD-0122	The profile condition is purity: <code>tr.Pure</code> , i.e. typed in the empty design and delay-free	accepted	Part IX	Phase 13: Surface/Provision, Experiments/ProvisionExamples	PRP-0001 (audits)
FVD-0123	A channel carries the transfer function and the term that computes it	accepted	Part IX	Phase 13: Surface/Provision, Experiments/ProvisionExamples	PRP-0001 (audits)
FVD-0124	The profile is generic in the concept; the Source's signature grants construction	accepted	Part IX	Phase 13: Surface/Provision, Experiments/ProvisionExamples	PRP-0001 (audits)
FVD-0125	Shared raw reading is primitive; the singleton is its special case	accepted	Part IX	Phase 13: Surface/Provision, Experiments/ProvisionExamples	PRP-0001 (audits)
FVD-0126	Trace equality needs a joint section; deployment is in general a strict refinement	accepted	Part IX	Phase 13: Surface/Provision, Experiments/ProvisionExamples	PRP-0001 (audits)
FVD-0127	Provision is not re-applicable; "idempotent" is the wrong word	accepted	Part IX	Phase 13: Surface/Provision, Experiments/ProvisionExamples	PRP-0001 (audits)

id	decision	status	main section	formal evidence	production correspondence
FVD-0128	A Source's commitments are obligations on the profile	accepted	Part IX	Phase 13: Surface/ Provision, Experiments/ ProvisionExamples	PRP-0001 (audits)
FVD-0129	Independent provisions commute exactly; the assignment is a set	accepted	Part IX	Phase 13: Surface/ Provision, Experiments/ ProvisionExamples	PRP-0001 (audits)
FVD-0130	Terminology: abstract Source, provisioned Source, raw declaration; not "monomorphised"	accepted	Part IX	Phase 13: Surface/ Provision, Experiments/ ProvisionExamples	PRP-0001 (audits)

Appendix D — Evidence-strength ledger

One row per claim of the conceptual chapters, with the formal evidence, the production implementation and its tests, the decisions and production records, and the known limitation. Every entry in the *formal* column is **formally proved** unless it is an executed example or a counterexample by its name; every entry in the *production* columns is **production implemented and tested** at the snapshot (de8154f); *not implemented* means **formally proved, not implemented**. Theorem names are in KCN-judu/BDL_FV.

claim	formal theorem / counterexample	production implementation	production tests	decisions · production records	known limitation
refining one declaration preserves every client	local_refinement_preserves_global_typing, _wf	bdl-model::apply_edit refinement classification	edit classification tests	FVD-0011–FVD-0016; ADR-0009	arbitrary edits are outside the order
typing consults only the type view	HasType.mono_env, refFree_env_irrelevant	bdl-check	pipeline tests	FVD-0004	—
semantic identity is nominal; construction needs a grant	constructs_granted, no_semantic_value_without_declaration, temporal_state_preserves_semantic_identity	elaborator rep/mk insertion; checker	semantic corpus case	FVD-0019–FVD-0028	—
dimensions reject length + time	dimension_mismatch_rejected; erasure counterexample	Prim.ty typing	diagnostics tests	FVD-0031	—
evaluation is deterministic and total on causal designs	Ev.det, reactive_total, Ev.not_of_strictCyclic	reference evaluator	delayed_cycle, property tests	FVD-0036–FVD-0042	lambda-guarded cycles conservative
one temporal primitive; delay is sync own	delay_is_sync_own, single_domain_embedding	StateCellId cells; sync snapshot rule	sync, agnostic corpus	FVD-0044–FVD-0048	—
scheduler order is unobservable	MEv.det, scheduling_order_observable (alternative)	step_in_order	order-independence tests	FVD-0045	—
outputs have one explicit driver	multiple_direct_drivers_rejected, single_driver_output_deterministic	bdl-output	lamp_output	FVD-0050–FVD-0054	—
hardware feasibility is decidable, sound and complete for the finite fragment	solve_sound, solve_complete, satisfiable_iff_solve	bdl-hardware::solve	Nano cases A–H	FVD-0055–FVD-0063	numeric constraints out of scope
behavior systems flatten into a well-formed design	flatten_wf, flatten_causal, flatten_wellClocked, flatten_singleDriver	bdl-system::flatten	system tests, behavior-systems-correspondence.md	FVD-0064–FVD-0073; ADR-0021/0022	Theorem J restricted
groups are transparent; extraction preserves behavior	Theorems A–G (rfl), orig_iff_flat	.bdl/authoring.json, packaging	group tests	FVD-0074–FVD-0082; ADR-0019	single-domain wiring fragment
the cross-domain window is expressible with lists	Theorem M buffer_window_correspondence;	five-declaration idiom; Vec core	buffer, bounded_buffer, overflowing_buffer	FVD-0083–FVD-0087; ADR-0024/0027	input source assumed

claim	formal theorem / counterexample	production implementation	production tests	decisions · production records	known limitation
	bounded_summary_ not_lossless				
capacity is validation; only refusal preserves semantics	sufficient_ capacity_preserves, periodic_capacity_ sufficient, negE	bdl-reactive:: capacity, bdl-exec- ir::bounds, refusal	3 000-tick differential	FVD-0086; ADR-0027	static bound unproved
products and one recursor suffice for the tested equations	fold_total, arrow_ not_delayable, church_fst_rank	bdl-ir prod, Fold	collections corpus	FVD-0088– FVD-0091; ADR-0025	no minimality theorem
rank-1 polymorphism by matching, no kernel change	matchTy_sound, matchTy_complete, instances_are_ monomorphic	bdl-equations	equation tests	FVD-0092; ADR-0025	—
the library adds no privilege	lib_expansion, Comb. noConstruct, lib_ clocked, lib_eval_ context_free	inlining at elaboration	—	FVD-0094	—
nominality survives generics	generic_preserves_ identity, generic_ preserves_ dimension, map_ keeps_concepts	checker on inlined terms	exI, exJ executed; mixed-comparison matrix	FVD-0092; ADR-0026	—
finite $\forall/\exists$ are folds	forall_in_list, exists_in_list	all/any equations	corpus	FVD-0095	—
equality is data; order is by declaration	Cap.eq_iff_data, Cap.ord_data, lt_ rejected, min_mode_ rejected, minBy_ recovers_min	Concept::ordered; semantic.no_order	mixed-comparison matrix	FVD-0098– FVD-0100; ADR-0025/0026	mixed case is a production rule
unit operations need no kernel construct	inUnitE_typed/_ safe, withUnitE_ typed, convert_eq, unitOps_no_ construction	bdl-elab::units coord/reconstruct/ convert	unit tests	FVD-0101– FVD-0103	float approximation
presentation is semantically irrelevant	presentation_ irrelevant_*	preferred units in authoring metadata	—	FVD-0105	—
Composer inference is sound and complete	solve_sound, solve_ complete, candidates_sound/_ complete	bdl-ide::formula	formula_composer.rs	FVD-0104; ADR-0028	two holes unsolved by design
affine conversion is a groupoid of affine maps; point/delta orthogonal	convert_compose, convert_inverse, difference_map, linear_part_ compose, sort_ orthogonal_to_ conversion, CtoF_ closed	Chart::Affine, convert	charts::tests (ulps)	FVD-0106 rev., FVD-0107– FVD-0110	affine charts hidden (ISS-0004)
binder syntax is conservative desugaring	desugar_rename, binder_local_type, binder_*_eval, range_eval, binder_ clock	bdl-elab::formula:: binder (P11)	natural_forms_ lower_to_the_same_ core_as_the_call_ forms, parser tests	FVD-0111– FVD-0114; ADR-0028 second amendment	no parser in the model
a Source is provisioned by a raw reading and a	provision_ transparent, provision_	not implemented	—	FVD-0121– FVD-0130; PRP-0001 (draft)	stateful transducers, device

claim	formal theorem / counterexample	production implementation	production tests	decisions · production records	known limitation
pure transducer without the design noticing	abstracts, provision_exact (joint section), provision_wf, provision_causal, provision_ wellClocked, provision_not_ reapplicable, provision_comm; exD, exE, no_joint_ witness				clocks, output dual open
() -> B is a conservative interface normalization whose kernel value is B	elim_canonical, decode_encode, canonicalOfKernel_ encode, zero_input_ obligation, lams_ typed, refForms_ agree, same_tick_ same_value, Clocked. refForms	bdI_ir::Ty::{Unit, of_signature, kernel_of_ signature, canonical_mapping_ ty}, Signature::is_ unit_domain	...the_encodings_ are_inverse, ... argument_is_erased, the_shorthand_and_ the_explicit_unit_ domain_are_one_ declaration	FVD-0115- FVD-0117; ADR-0029	elim proved on canonical types; () in output position refused by production
a unit binder in the kernel would forbid memory	delay_not_under_ binder, sync_not_ under_binder, zero_ input_memory	delay/sync typed outside binders (transcribed)	exD	FVD-0116; ADR-0029 alternatives	—
the source role is a realization state; A -> () cannot name a consumer	source_value, resolved_not_ source, SimulationInput. value, unit_ codomain_collapse, consumers_ indistinguishable, driver_is_unit_ domain, transport_ needs_unit_domain	RelationshipRole:: Source derived in the model and stated by the daemon (ADR-0032); the drive edge	the_role_is_one_ answer_across_the_ component_boundary; Studio simulation tests; exC, exE	FVD-0118- FVD-0120; ADR-0032	Rule and Value have no separate formal object
axiom discipline	every theorem on propext/Quot.sound; no sorry; no Classical.choice (audited per phase)	—	—	—	—

Appendix E — Migration map from the old decision numbers

Until 2026-09-20 the formal decisions were D-01 ... D-130, numbered in the order they were written in one ledger; the open items were unnumbered bullets, and for a few hours on the same day 0I-MN. The stable identifiers keep the sequence number (D-07 $\rightarrow$ FVD-0007) so that a citation in a commit message, an archived draft or a conversation can be followed by hand; the number encodes neither a phase nor a production record. Archived documents (paper/archive/, the dated entries of paper/NOTES.md) keep the old numbers and are not rewritten. Production’s records at the snapshot still cite the old numbers (ADR-0024, ADR-0025, formal-correspondence.md, status.md); this table resolves each. The canonical copy is docs/project/decision-id-migration.md in the formal repository.

old	new	title	production record
D-01	FVD-0001	Commitments are atomic labels; evidence is abstract	ADR-0010
D-02	FVD-0002	InterfaceRefines freezes the type and grows commitments; not logical implication	ADR-0010
D-03	FVD-0003	strengthen on a realized declaration carries a re-verification premise	ADR-0010
D-04	FVD-0004	DeclInterface.commitments is a List, not a Finset	ADR-0010
D-05	FVD-0005	References are by DeclId inside Expr (declRef), not a separate DesignExpr	ADR-0010
D-06	FVD-0006	The declRef typing rule reads $\Delta.tyView$ only	ADR-0010
D-07	FVD-0007	Realization is write-once; “detach” is not a refinement step	ADR-0009
D-08	FVD-0008	The structural order (DeclLeq, EnvRefines) is separated from the invariant (GlobalWF)	ADR-0010
D-09	FVD-0009	Evidence takes the environment as an argument	ADR-0010
D-10	FVD-0010	Evidence.Monotone is a kernel-imposed constraint on the validation layer	ADR-0010
D-11	FVD-0011	Semantics at Phase 1 is unfolding to a reference-free term	ADR-0010
D-12	FVD-0012	Acyclicity is witnessed by a rank function	ADR-0010
D-13	FVD-0013	Phase 0’s Artifact (bare list of ids) removed	FV-only
D-14	FVD-0014	Display names are not in the kernel	ADR-0008
D-15	FVD-0015	Kernel ontology migrated from holes to declarations	ADR-0010
D-16	FVD-0016	Monotone refinement is distinct from arbitrary editing	ADR-0009
D-17	FVD-0017	Commitments and validation obligations share PropertyId for now	ISS-0003
D-18	FVD-0018	Evidence.Monotone is a stability condition, not a definition of validity	ISS-0003
D-19	FVD-0019	Semantic identity lives in the type: $Ty.sem : SemanticId \rightarrow Ty$	ADR-0013

old	new	title	production record
D-20	FVD-0020	SemanticId is independent of DeclId and of display names	ADR-0013
D-21	FVD-0021	No introduction/elimination forms for semantic types in Phase 2	ADR-0013
D-22	FVD-0022	Explicit semantic mappings are ordinary declarations	ADR-0013
D-23	FVD-0023	Semantic identity change is an edit	ADR-0013
D-24	FVD-0024	Canonical closed inhabitants replaced by unresolved declarations	ADR-0013
D-25	FVD-0025	Future constraint for Phase 3: representation binding must not defeat semantic identity	ADR-0013
D-26	FVD-0026	Unrestricted symmetric mk/rep rejected	ADR-0013
D-27	FVD-0027	Representation types are sem-free (ConceptEnv.WF)	ADR-0013
D-28	FVD-0028	Observation is unrestricted; construction is licensed by the realized declaration's signature	ADR-0013
D-29	FVD-0029	Representation binding is a separate, write-once concept environment θ	ADR-0013
D-30	FVD-0030	Unfolding preserves typing under Grant.all, not under the client grant	ADR-0013
D-31	FVD-0031	Dimensions in $\text{Ty as } q \text{ d}$; algebra in Prim.ty ; no dimension rule	ADR-0011, ADR-0013
D-32	FVD-0032	Units are surface: elaborated to scaled dimensioned literals	ADR-0028
D-33	FVD-0033	Semantic identity is not indexed by dimension	ADR-0013
D-34	FVD-0034	One temporal primitive: $\text{delay init } e$	ADR-0004, ADR-0016
D-35	FVD-0035	delay is data-typed and top-level (empty context)	ADR-0004, ADR-0016
D-36	FVD-0036	Signal is not a type; Event is opt	ADR-0004, ADR-0016
D-37	FVD-0037	Causality replaces blanket acyclicity: $\text{Causal on InstDependsOn}$	ADR-0004, ADR-0016
D-38	FVD-0038	Explicit initial value on every delay	ADR-0004, ADR-0016
D-39	FVD-0039	State has no identity; state is structural	ADR-0004, ADR-0016
D-40	FVD-0040	Representation types are data (ConceptEnv.WF strengthened)	ADR-0004, ADR-0016
D-41	FVD-0041	Temporal changes are realization edits	ADR-0004, ADR-0016
D-42	FVD-0042	The reactive semantics is a relation, not yet a machine	ADR-0004, ADR-0016
D-43	FVD-0043	Time is one global tick with a schedule; no rates, no timestamps in the kernel	ADR-0004

old	new	title	production record
D-44	FVD-0044	Nominal <code>ClockId</code> , stored per declaration in <code>ClockEnv K</code> ; none = domain-agnostic pure mapping	ADR-0004
D-45	FVD-0045	One transport primitive <code>sync src init e</code> , reading strictly before	ADR-0004
D-46	FVD-0046	The clock is interface data held in a projection, not a record field	ADR-0004
D-47	FVD-0047	Rates, drift, jitter, latency, buffer capacity, value age are validation	ADR-0004
D-48	FVD-0048	Event transport = window read; buffering derived, Event still not a primitive	ISS-0001
D-49	FVD-0049	The logical relation is generic in the application relation	FV-only
D-50	FVD-0050	Physical sinks have nominal identity (<code>OutputId</code>), separate from <code>SemanticId</code> and <code>DeclId</code>	ADR-0005
D-51	FVD-0051	A drive edge is a per-declaration write-once projection β , checked by type and clock equality	ADR-0005
D-52	FVD-0052	Single-driver is a global invariant; completeness is the executable condition	ADR-0005
D-53	FVD-0053	No runtime arbitration, no implicit priority, no merge policy	ADR-0005
D-54	FVD-0054	Effect rows and action values rejected for this kernel	ADR-0005
D-55	FVD-0055	First binding is a refinement; rebinding is an edit; a second driver is invalid	ADR-0005
D-56	FVD-0056	Sinks are terminal	ADR-0005
D-57	FVD-0057	Hardware feasibility is a validation layer over $\text{Design} \times \text{Target}$, not typing	ADR-0006, ADR-0015
D-58	FVD-0058	Resources carry capabilities and per-capability units; sharing is a per-capability policy	ADR-0006, ADR-0015
D-59	FVD-0059	Requirements are independent variables with nominal <code>RequirementId</code> , optional fixed resource, optional unit relation	ADR-0006, ADR-0015
D-60	FVD-0060	Validity is unary support plus pairwise compatibility; the solver is exhaustive DFS, proved sound and complete	ADR-0006, ADR-0015
D-61	FVD-0061	Hardware extension is monotone; requirement extension, strengthening, fixing, and resource removal are revalidation triggers	ADR-0006, ADR-0015

old	new	title	production record
D-62	FVD-0062	Deployment feasibility is environment-sensitive evidence, not Evidence. Monotone	ADR-0006, ADR-0015
D-63	FVD-0063	Numeric electrical/timing constraints deferred	FV-only
D-64	FVD-0064	Behaviour components are surface objects; the kernel is unchanged	ADR-0021, ADR-0022
D-65	FVD-0065	A port is a template declaration by identity, with its public interface and clock	ADR-0021, ADR-0022
D-66	FVD-0066	Instantiation renames every identity the template owns; concepts and sinks are partitioned into internal (fresh) and global (shared)	ADR-0021, ADR-0022
D-67	FVD-0067	Binding is a Phase-1 realization step	ADR-0021, ADR-0022
D-68	FVD-0068	Composition well-formedness is stated on interfaces, never on bodies	ADR-0021, ADR-0022
D-69	FVD-0069	Causality across instances is a validation condition on the inter-instance direct-binding graph	ADR-0021, ADR-0022
D-70	FVD-0070	Clock parameters are nominal variables substituted by κ at instantiation; rates never enter	ADR-0021, ADR-0022
D-71	FVD-0071	Evidence must be equivariant and port-sound	ADR-0021, ADR-0022
D-72	FVD-0072	Hierarchy is packaging, not a tree constructor	ADR-0021, ADR-0022
D-73	FVD-0073	Theorem J is proved on the single-domain wiring fragment with direct/constant bindings	ADR-0021
D-74	FVD-0074	A behaviour group is authoring metadata; eraseGroups is a projection	ADR-0019
D-75	FVD-0075	Group operations are semantic no-ops, not refinements or edits	ADR-0019
D-76	FVD-0076	Aggregate sockets are projections of DependsOn	ADR-0019
D-77	FVD-0077	Boundary inference: required = crossing-in, provided = crossing-out, private = the rest without a sink, clocks = all clocks	ADR-0019
D-78	FVD-0078	Extraction = two restrictions of the design reconnected by Phase-8a bindings	ADR-0019
D-79	FVD-0079	Extraction causality is proved by subdividing the original graph, not by InstAcyclic	ADR-0019
D-80	FVD-0080	Template realization needs interface-local evidence	ADR-0019

old	new	title	production record
D-81	FVD-0081	Identity: templates keep original identities; instances are fresh; the group id is never a component id	ADR-0019
D-82	FVD-0082	Nested groups are a relation on the flat group list	ADR-0019, ISS-0007
D-83	FVD-0083	<code>Ty.list τ</code> is a kernel data type; the object-language buffer needs it and nothing else	ADR-0024
D-84	FVD-0084	Six list operators, registered through <code>Prim.ty: nil, cons, length, take, reverse, head</code>	ADR-0024
D-85	FVD-0085	The buffer is a surface elaboration into five declarations over <code>delay/sync</code>	ISS-0001
D-86	FVD-0086	Capacity is validation; overflow policies are explicit; only rejecting the deployment preserves semantics	ADR-0027
D-87	FVD-0087	Buffered transport is a Phase-8a binding choice, not a transport kind	ADR-0021
D-88	FVD-0088	Products enter the kernel as value composition: <code>prod, pair, fst, snd</code>	ADR-0025
D-89	FVD-0089	The list recursor <code>fold</code> is a term former, not a registered operator	ADR-0025
D-90	FVD-0090	<code>eq</code> at every data type, with the data proof in the syntax	ADR-0025
D-91	FVD-0091	<code>toList : opt τ → list τ</code> and <code>drop</code> are registered operators	ADR-0025
D-92	FVD-0092	Rank-1 polymorphism is definitional: families instantiated by matching; no type variable in the kernel	ADR-0025
D-93	FVD-0093	Constraints: the closed vocabulary <code>{Data}</code> ; no user-defined classes	ADR-0025
D-94	FVD-0094	The equation library is a set of combinators, inlined at use sites	ADR-0025
D-95	FVD-0095	Sets, intervals, records, predicates, finite quantifiers are surface	ADR-0025
D-96	FVD-0096	Sums are encoded; a kernel <code>sum</code> is deferred	ISS-0005
D-97	FVD-0097	Existentials are not needed: hiding is Phase-8a instantiation	ADR-0025
D-98	FVD-0098	Ordering is a quantity comparison; the kernel has no structural order	ADR-0025, ADR-0026
D-99	FVD-0099	The surface capability vocabulary is <code>{Data, Eq, Ord}</code> ; <code>Eq</code> = <code>Data</code> today; <code>Ord</code> is by declaration	ADR-0025, ADR-0026

old	new	title	production record
D-100	FVD-0100	Ordered library entries take Ordered evidence; comparators recover them	ADR-0025, ADR-0026
D-101	FVD-0101	Units remain entirely surface; coordinate extraction and quantity construction are elaborated quantity arithmetic	ADR-0028
D-102	FVD-0102	Unit semantics are stated exactly over an abstract scalar domain; the kernel's Nat and production's floats are models	ADR-0028
D-103	FVD-0103	A unit is an identity with a dimension and a scale; spelling is presentation	ADR-0028
D-104	FVD-0104	The Formula Composer's formal basis is typed holes with local bidirectional dimension inference — no unification	ADR-0028
D-105	FVD-0105	Preferred display units are presentation, not design	FV-only
D-106	FVD-0106	Affine conversion is complete as coordinate-change semantics; point/delta is optional physical-arithmetic validation	ISS-0004
D-107	FVD-0107	Unit coordinates are an erasure that preserves the affine coordinate change	ISS-0004
D-108	FVD-0108	Conversions form a groupoid of affine isomorphisms; differences carry the linear part	ISS-0004
D-109	FVD-0109	The exact scalar domain is a choice-free rational field built in the development	ISS-0004
D-110	FVD-0110	Unit conversion and sensor calibration are one affine-map abstraction	ISS-0004
D-111	FVD-0111	Binder syntax desugars to the Phase-9 library applied to a lambda; a binder local is the lambda parameter	ADR-0028
D-112	FVD-0112	Ranges are surface nodes desugared to inRange; no interval type or value	ADR-0028
D-113	FVD-0113	No general comprehension, no general quantifier	ADR-0028
D-114	FVD-0114	$x \ \ ?? \ d$ desugars to <code>getD</code>	ADR-0028
D-115	FVD-0115	The canonical type <code>domain(inputs) -> B</code> with <code>domain([]) = ()</code> lives above the kernel; the kernel interface type is its normalization	ADR-0029
D-116	FVD-0116	<code>() -> B</code> is realized at <code>B</code> in the empty context; no unit binder	ADR-0029
D-117	FVD-0117	<code>f</code> , <code>f()</code> , <code>f(())</code> are one reference; a reading is not a call	ADR-0029

old	new	title	production record
D-118	FVD-0118	The source role is a realization state, not a type shape and not a kind	ADR-0032
D-119	FVD-0119	A $\rightarrow$ () is not a physical sink; the drive edge is	ADR-0029, ADR-0032
D-120	FVD-0120	Transport and memory of a relationship require the unit domain – as corollaries	ADR-0029
D-121	FVD-0121	Provision is a construction over designs, not a kernel construct	PRP-0001
D-122	FVD-0122	The profile condition is purity: <code>tr.Pure</code> , i.e. typed in the empty design and delay-free	PRP-0001
D-123	FVD-0123	A channel carries the transfer function and the term that computes it	PRP-0001
D-124	FVD-0124	The profile is generic in the concept; the Source’s signature grants construction	PRP-0001
D-125	FVD-0125	Shared raw reading is primitive; the singleton is its special case	PRP-0001
D-126	FVD-0126	Trace equality needs a joint section; deployment is in general a strict refinement	PRP-0001
D-127	FVD-0127	Provision is not re-applicable; “idempotent” is the wrong word	PRP-0001
D-128	FVD-0128	A Source’s commitments are obligations on the profile	PRP-0001
D-129	FVD-0129	Independent provisions commute exactly; the assignment is a set	PRP-0001
D-130	FVD-0130	Terminology: abstract Source, provisioned Source, raw declaration; not “monomorphised”	PRP-0001

Open items

old	new	title	production record
OI-01	FVI-0001	Interface-level references	ISS-0003
OI-02	FVI-0002	Delay/temporal boundaries	FV-only
OI-03	FVI-0003	Lambda-guarded instantaneous cycles	FV-only
OI-04	FVI-0004	Higher-order <code>unfolds_</code> <code>preserves_eval</code> (closure equivalence)	FV-only
OI-05	FVI-0005	Reusable stateful components	FV-only
OI-06	FVI-0006	Theorem J for <code>MEv</code> with <code>sync</code> bindings; higher-order Theorem J; a consistent modular input	FV-only
OI-07	FVI-0007	<code>toComponent</code> realizes its chosen interface; port-level inter-instance causality	FV-only
OI-08	FVI-0008	Event multiplicity	FV-only

old	new	title	production record
OI-09	FVI-0009	StateHandler with handler-scoped clocks / independently clocked nesting	ISS-0010
OI-10	FVI-0010	Ty.list and list operators, to write the buffer in the object language	FV-only
OI-11	FVI-0011	Minimal unsat cores; numeric hardware constraints; timer and PWM compatibility	FV-only
OI-12	FVI-0012	A reusable device-component library (Phase 8 surface)	ISS-0016
OI-13	FVI-0013	Folding ClockEnv into the DeclInterface record (churn only)	FV-only
OI-14	FVI-0014	Several candidate definitions with one active, as a surface convenience	ISS-0002
OI-15	FVI-0015	Environment-sensitive evidence and invalidation tracking for edits	ISS-0003
OI-16	FVI-0016	Representation binding for concepts	FV-only
OI-17	FVI-0017	Affine units: a point/difference sort as optional validation; the display-name table	ISS-0004
OI-18	FVI-0018	Grant delegation to higher-order mappings	FV-only
OI-19	FVI-0019	Display-name table for concepts	FV-only
OI-20	FVI-0020	Source provision: stateful transducers, a device clock, commitment discharge, output provision	PRP-0001, ISS-0016
OI-21	FVI-0021	Unit-domain normalization: elim beyond canonical types; the Input narrowing	ADR-0029

Appendix F — Production snapshot

Production snapshot as of 2026-09-20, commit `de8154f5153495de2ad8a09f3ca3166c3678dc93` of `KCN-judu/BDL` (“chore(ide): satisfy clippy and rustfmt in the token tests and the perf example”, the last commit of the syntax-highlighting milestone). What was checked: `docs/README.md`’s current snapshot, `docs/project/status.md`, `docs/project/roadmap.md`, `docs/project/formal-correspondence.md`, `docs/architecture/{overview,relationship-roles,syntax-highlighting,ide-service}.md`, `docs/spec/{protocol,textual-syntax,concept-library}.md`, ADR-0032 with its two amendments, ADR-0034, ADR-0035, PRP-0001, ISS-0016, the change records of 2026-09, `library/std/concepts.toml`, and the fixture sources under `docs/fixtures/`. The formal repository is described as of the working tree of this revision; its last pushed commit is `af25567` (Phase 13, 2026-09-20). The canonical location of the production hash in the formal repository is the `snapshot` field of `docs/project/production-correspondence.md`; this appendix repeats it.

Milestones of the week before the snapshot, newest first

milestone	what it changed	records
syntax highlighting	one Rust classifier on the LSP semantic-token vocabulary; colour in Studio’s Code view and formula field; protocol 0.21	ADR-0035; <code>syntax-highlighting.md</code>
relationship roles	<code>bdL_model::RelationshipRole { Source, Rule, Value }</code> — one derived role stated by the daemon; <code>applied_by</code> ; Studio re-derives nothing; protocol 0.20	ADR-0032 amended; <code>relationship-roles.md</code>
unapplied rule	<code>reactive.rule_unapplied</code> ; the action rule. <code>apply</code> ; the hollow output socket; the three-state Source control (<i>no value yet</i>); <code>CreateMapping.definition?</code> ; protocol 0.19	change record 2026-09-unapplied-rule
reference edges	reference edges into the formula line from <code>MappingAnalysis.references</code> ; the word <i>rule</i> ; protocol 0.18	ADR-0034
generalized Standard Library	<code>LibraryItem</code> → <code>Fragment</code> → ordinary objects; <i>Concepts</i> and <i>Sources</i> ; one-transaction instantiation; localized item text; protocol 0.17	ADR-0032 first amendment
Source role	a Source as a derived presentation role; FV Phase 12 consumed; ISS-0014 resolved; ISS-0016 opened; protocol 0.16 (deprecated the same day)	ADR-0032
internationalization	English, Simplified Chinese, Japanese; locale is presentation only	ADR-0031
complete persistence	the whole authoring state saved, including text that does not build and every draft; one save guard	ADR-0030
unit-domain normalization	one canonical type per relationship; <code>()</code> → <code>B</code> preferred; <code>bdLd migrate-unit-domain</code> ; protocol 0.14	ADR-0029
natural expression surface	binders, closed ranges, <code>??</code> as one-way desugaring; the Composer draws them; protocol 0.13	ADR-0028 second amendment
PRP-0001	the Source-provision proposal, revised after the formal audit; status draft	PRP-0001

Protocol

Protobuf over framed stdio; additive minors. 0.11 structured value forms · 0.12 the Composer’s three queries · 0.13 binder and range nodes · 0.14 the unit type kind · 0.15 definition drafts in the system view · 0.16 Source template fields (deprecated) · 0.17 library items · 0.18 `MappingAnalysis.references` · 0.19 `CreateMapping.definition?/clock_id?` · **0.20** `RelationshipRole`, `MapView.role`, `MappingAnalysis.role` / `applied_by` · **0.21** `SemanticTokens` for a source file or a formula draft, with the legend in every answer. A 0.19 client ignores the role fields; Studio 0.20 requires them.

Implementation status, by area (production’s own words, abridged)

Implemented: the language core through outputs and completeness; formula language v0 with the slot; the natural forms; one canonical type per relationship; the unit registry with linear charts offered and affine charts as tested infrastructure (ISS-0004); the data core and the equation library; the compiler and simulator with product-language diagnostics and Explain; the Rust backend with differential, golden and property tests over 22 corpus cases; collections at deployment (static bounds, window capacity, bounded-memory refusal, `bdld compile --bounded-memory --period`); hardware and deployment analysis with Nano and a larger board file; behavior systems, groups, packaging, versions and substitution; the unified project format with migration and complete persistence; the layout service; text ↔ graph synchronisation; the IDE service and LSP with a VS Code extension, semantic tokens and the Composer queries; the protocol and daemon at 0.21; the Standard Library (36 Concept items, 8 Source items); localization (partial: the compiler’s diagnostic sentences are English, ISS-0015).

Partial — Studio: the Design page with Design, Code and Split views, the canvas with roles, reference edges and groups, the inspector with the Formula | Text definition editor, the Formula Composer with references, literals, slots, operators, calls, binders, ranges, boolean logic and choices structured and richer forms as text, Simulate with Sources as inputs, Deploy on the 0.5 read model, Library, the conflict banner, colour from semantic tokens. Not built: the Monitor page (a placeholder), a device binding for a Source (ISS-0016), Explain over a protocol request, domain regions and cycle emphasis on the canvas, entity hover and fixes inside a component’s source, a native menu bar.

Planned, designed, not implemented: the first platform adapter (Embassy; roadmap priority 1), build orchestration, flash, telemetry, a second embedded target; supplied Rust components (ADR-0005); the device catalogue and PRP-0001.

Not implemented by decision: user enums (ISS-0005), temporal modifiers and contexts (ISS-0010), a surface form for occurrence windows (ISS-0001), record syntax, `forall/exists` sugar, `Set/interval/sum` types, an `A -> ()` consumer form.

Facts this document relies on that could move

Protocol version numbers; the count of library items; the list of structured Composer forms (`let`, `match`, blocks and rules are text at the snapshot); the roadmap order; the two known code-side drifts production recorded while auditing its own docs (an action’s reason string; a test’s doc comment). When any of these moves, this appendix and the correspondence page move; the conceptual chapters do not.

Appendix G — Development chronology

The order in which the hypotheses were tested. Phase numbers are provenance, not structure: the conceptual chapters cite them so that a reader can find the report that holds the experiment, and the reports (docs/reports/ in the formal repository) are the local records — question, models tried, theorems with their hypotheses, executed cases, claim audit, verdicts — from which this document’s chapters were synthesized. A report is written once and corrected by strike-through when a later phase overturns a claim (Phase 9c did this to 9b’s structural order; Phase 10b to 10’s sort). Dates are the commits that added each report.

phase	date	question	main section	report
0	2026-09-14	a single persistent declaration	Part III	docs/reports/phase-00-a-single-persistent-declaration.md
1	2026-09-14	cross-declaration references	Part III	docs/reports/phase-01-cross-declaration-references.md
1 → M	2026-09-14	Migration: holes → declarations	Part III	docs/reports/phase-01m-migration-holes-to-declarations.md
2	2026-09-15	where does semantic identity live?	Part IV	docs/reports/phase-02-where-does-semantic-identity-live.md
3	2026-09-15	representation binding and physical dimensions	Part IV	docs/reports/phase-03-representation-binding-and-physical-dimensions.md
4	2026-09-15	Reactive Core	Part VII	docs/reports/phase-04-reactive-core.md
5	2026-09-15	Clock domains and synchronization	Part VII	docs/reports/phase-05-clock-domains-and-synchronization.md
6	2026-09-15	Physical outputs and the single-driver discipline	Part IX	docs/reports/phase-06-physical-outputs-and-the-single-driver-discipline.md
7	2026-09-15	Hardware constraint validation and resource allocation	Part X	docs/reports/phase-07-hardware-constraint-validation-and-resource-allocation.md
8a	2026-09-15	Behaviour as a first-class design object	Part VIII	docs/reports/phase-08a-behaviour-as-a-first-class-design-object.md
8b	2026-09-16	Behaviour grouping and component extraction	Part VIII	docs/reports/phase-08b-behaviour-grouping-and-component-extraction.md
9a	2026-09-17	List data and lossless buffered cross-domain events	Part VII	docs/reports/phase-09a-list-data-and-lossless-buffered-cross-domain-events.md
9b	2026-09-17	Minimal data abstraction and the polymorphic equation language	Part V	docs/reports/phase-09b-minimal-data-abstraction-and-the-polymorphic-equation-language.md
9c	2026-09-18	Capability boundary audit: Data vs Eq vs Ord	Part V	docs/reports/phase-09c-capability-boundary-audit-data-vs-eq-vs-ord.md
10	2026-09-18	Unit coordinates and formula-assembly semantics	Part VI	docs/reports/phase-10-unit-coordinates-and-formula-assembly-semantics.md
10b	2026-09-18	Affine coordinate erasure and conversion functoriality	Part VI	docs/reports/phase-10b-affine-coordinate-erasure-and-conversion-functoriality.md
11	2026-09-18	Natural expression surface as conservative desugaring	Part V	docs/reports/phase-11-natural-expression-surface-as-

phase	date	question	main section	report
				conservative-desugaring.md
12	2026-09-18	Unit-domain normalization and the source boundary	Part IX	docs/reports/phase-12-unit-domain-normalization-and-the-source-boundary.md
13	2026-09-20	Source provision by device transducers (PRP-0001 audit)	Part IX	docs/reports/phase-13-source-provision-by-device-transducers-prp-0001-audit.md

Notation changed once: Phases 0–1 spoke of *holes* (`HoleId`, `holeRef`); the post-Phase-1 migration (FVD-0015) replaced the vocabulary with declarations, and every later phase uses the current notation of Appendix A. Phase 9b’s structural order and Phase 10’s point/difference sort were revised by 9c and 10b; the earlier positions are kept in the reports and in Part XIV’s *decisions that changed* table.

Appendix H — Revision log

revision	what the record gained
2026-09, conference manuscript (Phases 0–7)	motivation, designer vocabulary, interaction model, the kernel through hardware validation, claim-strength discipline; the manuscript is archived beside this document
Phase 8a/8b — behavior systems and groups	components, fresh instantiation, bindings, flattening, substitutability; groups as authoring metadata and their extraction
Phase 9a — list data and lossless buffering	list τ , the object-language window, capacity as validation
Phase 9b/9c — data and equation language	products, the list recursor, rank-1 definitional polymorphism, the capability audit that reverted structural ordering
Phase 10/10b — units, charts, Formula Composer basis	unit coordinates, exact symbolic scales, local dimension inference, affine charts and the AffSort revision
production Formula Composer (P10a/P10b)	the projection, slots, compose operations, unit ownership
Phase 11 — natural expression surface	binders and ranges as conservative desugaring
the monograph rewrite (69caa3a)	the monograph structure, production architecture, correspondence and deviations, ledgers, open agenda
production P11 and ADR-0029 (3c6c8be)	the natural forms implemented; one canonical type per relationship, the empty product $()$ as the unit domain, protocol 0.13/0.14
Phase 13 — Source provision (PRP-0001 audit)	device channels and profiles, shared-row provision as an EnvRefines step, transparency, trace abstraction, joint-section exactness, the corrected claims
Phase 12 — unit-domain normalization	$() \rightarrow B$ as an interface normalization whose value is the kernel type B ; the source role as a realization state; $A \rightarrow ()$ shown unable to name a consumer
2026-09-20 — this revision	the conceptual restructuring; Phase 13 as the environment boundary (Part IX); production at de8154f; the FVD/FVI identifiers; the indexes (Appendices B–E); the two kinds of minimality; the residue of the conference form removed; paper/README.md and main.typ describing the monograph pipeline

Closing

Modern industrial products increasingly combine physical form with sensing, computation, and control, yet designers still lack a behavior medium with the immediacy that CAD provides for geometry. BDL proposes that the missing medium should not be a friendlier version of procedural programming. It should be a language in which typed product relationships are first-class design artifacts, and in which an unresolved relationship is a legal state of the design rather than a defect in a program.

What that looks like in use is a workspace in which a designer names what the product is about, draws the relationships between those things, and leaves each undefined until there is something to say; refines them locally, by formula or curve or example, without the diagram changing shape; states time as a qualifier rather than as a timer; gives situations a name and a boundary rather than a transition table; is asked for one final target where two behaviors reach for one output, and for one stated way of seeing across a boundary where two quantities move at different speeds; and chooses a board last, receiving either a pin allocation or a conflict, with the design itself untouched either way.

The kernel that supports this is small, and it is small for reasons that were checked. A declaration has a frozen type, growable public commitments, and a write-once body; clients are typed against the type view, and refinement preserves what they established while edits reopen it. Semantic concepts are nominal, represented through a write-once binding, and constructed only where a signature announces them; dimensions are carried by the types of primitive operators. One temporal primitive reads a clock domain at its previous activation, and single-domain delay is its diagonal; evaluation is deterministic and total exactly on causal designs; every designer-facing temporal operator is a shape over it. Clock domains are nominal and checked by a judgment rather than a type; crossings are explicit, initialized, and strictly earlier. Physical outputs are nominal sinks with one driver each, and every combination of behaviors is ordinary computation upstream of the drive edge. A separate validation layer decides whether the design fits a board, and its evidence is kept apart from the evidence that survives refinement.

Each of these is minimal among the designs that were tested, and this document has tried to say, for each, what was proved, what was rejected by counterexample, and what was preferred. The elaborator, the editor and the host-side firmware path have been built and are described in Parts XI–XII; the platform adapter and the device catalogue have not; the studies have not, and they are where the claims about designers would be tested. The research question is not whether designers can be taught a simpler programming language. It is whether product behavior can become a *design material* whose structure is intuitive at the surface and rigorous underneath, and the kernel presented here is the part of that question that can now be stated precisely.

References

- [1] S. Greenberg and C. Fitchett, “Phidgets: Easy Development of Physical Interfaces through Physical Widgets,” in *Proceedings of the 14th Annual ACM Symposium on User Interface Software and Technology*, ACM, 2001, pp. 209–218. doi: 10.1145/502348.502388.
- [2] B. Hartmann *et al.*, “Reflective Physical Prototyping through Integrated Design, Test, and Analysis,” in *Proceedings of the 19th Annual ACM Symposium on User Interface Software and Technology*, ACM, 2006, pp. 299–308. doi: 10.1145/1166253.1166300.
- [3] L. de Moura and S. Ullrich, “The Lean 4 Theorem Prover and Programming Language,” in *Automated Deduction – CADE 28*, in Lecture Notes in Computer Science, vol. 12699. Springer, 2021, pp. 625–635. doi: 10.1007/978-3-030-79876-5_37.
- [4] T. R. G. Green and M. Petre, “Usability Analysis of Visual Programming Environments: A 'Cognitive Dimensions' Framework,” *Journal of Visual Languages & Computing*, vol. 7, no. 2, pp. 131–174, 1996, doi: 10.1006/jvlc.1996.0009.
- [5] N. Halbwachs, P. Caspi, P. Raymond, and D. Pilaud, “The Synchronous Data Flow Programming Language LUSTRE,” *Proceedings of the IEEE*, vol. 79, no. 9, pp. 1305–1320, 1991, doi: 10.1109/5.97300.
- [6] J.-L. Colaço and M. Pouzet, “Clocks as First Class Abstract Types,” in *Proceedings of the Third ACM International Conference on Embedded Software*, ACM, 2003.
- [7] R. Dechter, *Constraint Processing*. Morgan Kaufmann, 2003.
- [8] B. Hartmann, L. Abdulla, M. Mittal, and S. R. Klemmer, “Authoring Sensor-Based Interactions by Demonstration with Direct Manipulation and Pattern Recognition,” in *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*, ACM, 2007, pp. 145–154. doi: 10.1145/1240624.1240646.
- [9] Arduino, “Arduino Language Reference.” 2024.
- [10] OpenJS Foundation, “Node-RED: Low-code programming for event-driven applications.” 2024.
- [11] M. Resnick *et al.*, “Scratch: Programming for All,” *Communications of the ACM*, vol. 52, no. 11, pp. 60–67, 2009.
- [12] A. J. Ko *et al.*, “The State of the Art in End-User Software Engineering,” *ACM Computing Surveys*, vol. 43, no. 3, p. 21:1–21:44, 2011.
- [13] J.-L. Colaço, B. Pagano, and M. Pouzet, “A Conservative Extension of Synchronous Data-flow with State Machines,” in *Proceedings of the 5th ACM International Conference on Embedded Software*, ACM, 2005. doi: 10.1145/1086228.1086261.
- [14] G. Berry and G. Gonthier, “The Esterel Synchronous Programming Language: Design, Semantics, Implementation,” *Science of Computer Programming*, vol. 19, no. 2, pp. 87–152, 1992, doi: 10.1016/0167-6423(92)90005-V.
- [15] G. Berry, “SCADE: Synchronous Design and Validation of Embedded Control Software,” in *Next Generation Design and Verification Methodologies for Distributed Embedded Control Systems*, Springer, 2007, pp. 19–33.
- [16] T. Bourke and M. Pouzet, “Zélus: A Synchronous Language with ODEs,” in *Proceedings of the 16th International Conference on Hybrid Systems: Computation and Control*, ACM, 2013, pp. 113–118. doi: 10.1145/2461328.2461348.
- [17] C. Elliott and P. Hudak, “Functional Reactive Animation,” in *Proceedings of the Second ACM SIGPLAN International Conference on Functional Programming*, ACM, 1997, pp. 263–273. doi: 10.1145/258948.258973.
- [18] G. H. Cooper and S. Krishnamurthi, “Embedding Dynamic Dataflow in a Call-by-Value Language,” in *Programming Languages and Systems: 15th European Symposium on Programming*, Springer, 2006, pp. 294–308.
- [19] National Instruments, “LabVIEW.” 2024.
- [20] The MathWorks, “Simulink and Stateflow.” 2024.

- [21] Modelica Association, “Modelica Language Specification, Version 3.6.” 2023.
- [22] D. Harel, “Statecharts: A Visual Formalism for Complex Systems,” *Science of Computer Programming*, vol. 8, no. 3, pp. 231–274, 1987, doi: 10.1016/0167-6423(87)90035-9.
- [23] Object Management Group, “OMG Systems Modeling Language (SysML) Version 2.0.” [Online]. Available: <https://www.omg.org/spec/SysML/2.0>
- [24] C. Omar, I. Voysey, M. Hilton, J. Aldrich, and M. A. Hammer, “Hazelnut: A Bidirectionally Typed Structure Editor Calculus,” in *Proceedings of the 44th ACM SIGPLAN Symposium on Principles of Programming Languages*, ACM, 2017, pp. 86–99. doi: 10.1145/3009837.3009900.
- [25] C. Omar, I. Voysey, R. Chugh, and M. A. Hammer, “Live Functional Programming with Typed Holes,” *Proceedings of the ACM on Programming Languages*, vol. 3, no. POPL, p. 14:1–14:32, 2019, doi: 10.1145/3290327.
- [26] M. Voelter, J. Siegmund, T. Berger, and B. Kolb, “Towards User-Friendly Projectional Editors,” in *Software Language Engineering (SLE)*, in LNCS, vol. 8706. 2014, pp. 41–61.
- [27] A. Kennedy, “Relational Parametricity and Units of Measure,” in *Proceedings of the 24th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, ACM, 1997, pp. 442–455. doi: 10.1145/263699.263761.
- [28] G. D. Plotkin and M. Pretnar, “Handling Algebraic Effects,” *Logical Methods in Computer Science*, vol. 9, no. 4, 2013, doi: 10.2168/LMCS-9(4:23)2013.
- [29] Z. Yang, M. Paviotti, N. Wu, B. van den Berg, and T. Schrijvers, “Structured Handling of Scoped Effects,” *Proceedings of the ACM on Programming Languages*, vol. 6, no. ICFP, 2022, doi: 10.1145/3547654.
- [30] W. M. McKeeman, “Differential Testing for Software,” *Digital Technical Journal*, vol. 10, no. 1, pp. 100–107, 1998.
- [31] X. Leroy, “Formal Verification of a Realistic Compiler,” *Communications of the ACM*, vol. 52, no. 7, pp. 107–115, 2009.